Agent = Harness(Model):
Five Mechanistic Primitives, and What Is Not One

Chase Dovey
Average Joes Lab
https://cdovey.dev

Abstract

The software around a language model in an agent is called a scaffold, a framework, a runtime or a harness, and none of these names comes with a definition of what the thing contains. We take the position that an LLM agent is a model wrapped in a harness, *Agent* = *Harness*(*Model*), and that every harness is built from five mechanistic primitives: Input, which captures what comes in; Context, which assembles what the model is shown; Model interface, which sends the request and brings back the response; Output, which carries the response out; and Control flow, which decides whether to go again, hand back or stop. Each primitive is defined by what it does and by what it never does, so that the five are mutually exclusive. We then separate three levels at which harnesses can be described: the *primitives*, which every harness has; the *design decisions* made within each primitive, which is where harnesses differ; and the *production concerns* (persistence, safety and security, observability, resilience, performance, evaluation), which are questions asked of every primitive rather than parts of the harness. As evidence, we deconstruct six harnesses from their primary sources at pinned versions: Claude Code, Codex CLI, opencode, pi, Hermes Agent, and a LangGraph workflow. Every component we found has a home in one primitive or one concern, and the differences between the six are design decisions within the primitives. The vocabulary gives a procedure for locating an agent's failure in one of five places before attributing it to the model, and a schema for reporting the harness alongside the model. The minimal harnesses from which the definitions were drawn are reproduced in an appendix, and the evidence for every assignment is published with the paper.

1 Introduction

A language model does one thing: given a sequence of tokens, it produces a distribution over the next token. It cannot see a file it was not shown, remember a conversation it was not sent again, or carry out a command it wrote. Language models are nonetheless described as agents that read codebases, run tests, browse the web and fix bugs. The difference between what a model is and what an agent does is software around the model. That software is the subject of this paper.

The software has many names. Evaluations call it a *scaffold* [2, 10]; libraries call themselves *frameworks*, *SDKs* or an *orchestration framework and runtime* [9]; and *harness* is now common among people who build coding agents [24]. None of the names says what the thing contains, where it ends and the model begins, or how two instances of it differ. As a result, agents are compared by the model inside them, and results on one benchmark with one model vary with the system around it (§6). When such an agent fails, there is no agreed list of places to look.

Position. We argue three claims.

1. **An LLM agent is a model wrapped in a harness**: *Agent* = *Harness*(*Model*). The model maps tokens to a next token. The harness is everything else on the path of a request.
2. **Every harness is built from five mechanistic primitives**: INPUT, CONTEXT, MODEL INTERFACE, OUTPUT and CONTROL FLOW. Each is defined by what it does and by what it never does, so that each piece of a harness has one home.
3. **Everything else said about a harness is at one of two other levels**: a *design decision* within a primitive (one tool or forty; a loop or a graph; a repository map or a summarizer), or a *production concern* asked of every primitive (persistence, safety and security, observability, resilience, performance, evaluation). Neither level adds a primitive.

What the vocabulary is for. It separates what a harness *is* from what a given harness *chose* and from what makes it fit to *run*. With that separation, a failure can be located: was the model shown what it needed (CONTEXT), did what it asked for happen (OUTPUT), did the loop stop early or run long (CONTROL FLOW), was the response read correctly (MODEL INTERFACE), did the right thing come in (INPUT)? Only when all five are ruled out does the failure belong to the model. Two harnesses can be compared decision by decision within the same five primitives, a benchmark result can report the harness that produced it, and a harness can be moved to a new model by changing one primitive.

Evidence. The definitions come from building a harness from nothing, one primitive at a time, around a fixed model and a fixed task, and observing what each version could not do (Appendix A). The claim that the five cover existing harnesses is tested by deconstructing six of them from their primary sources at pinned versions: Claude Code, Codex CLI, opencode, pi, Hermes Agent, and a LangGraph workflow (§5). For each we assign every component on the request path to one primitive, or to one concern together with the primitive it changes, and record every case that was hard to place. The evidence for each assignment is given in Appendix B and published with the paper.

What this paper is not. It is a conceptual contribution. It reports no experiment in which a primitive is varied and performance measured, and it does not prove that no sixth primitive could ever be needed. It states what would refute the claim (§8) and gives a deconstruction a reader can check.

Contributions.
- A definition of the harness and of its boundary with the model (§2).
- Five primitives, each with a mechanism, a negative definition, and its design space (§3).
- Three levels of description, primitive, design decision and production concern, with rules for the cases at their boundaries (§4).
- A deconstruction of six harnesses, with method, selection criteria and published evidence (§5, Appendix B).
- Uses of the vocabulary: a diagnostic procedure, a reporting schema, and porting (§6).
- The minimal harnesses, one primitive at a time (Appendix A).

2 The Boundary: Model and Harness

The model. We take a language model to be a function from a token sequence to a distribution over the next token, applied autoregressively until a stop token is produced:

$$TokensOut = Model(TokensIn).$$

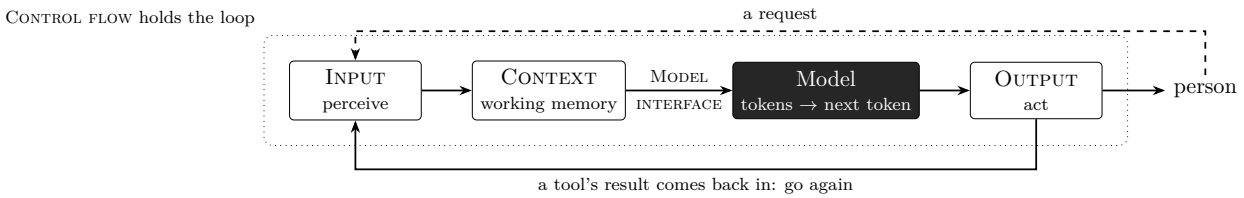


Figure 1: The agent loop. The five primitives surround the model; Control flow decides whether the loop runs again, hands back, or stops.

Chat templates, tool-call formats and reasoning tokens are conventions for laying out *TokensIn* and reading *TokensOut*. Models learn these conventions in training, and can learn to emit calls to tools [17], but the calls are carried out, and the conventions written and parsed, by something else. Viewed from outside, the model keeps no state between calls, cannot read from or act on its environment, and cannot call itself again.

The harness. The harness is everything in an agent that is not the model: the software that turns the world into tokens for the model, and the model's tokens into effects on the world.

$$Agent = Harness(Model).$$

The boundary is at the model's weights and its sampling. Everything the model is shown, everything it can do, and everything that happens between one call and the next is on the harness side. This includes things often attributed to the model: memory across sessions (the harness stores and retrieves it), tool use (the model emits a request; the harness carries it out), and knowledge of the date or the working directory (the harness writes it into the context).

The loop. Figure 1 shows the shape every harness shares. Something comes in: a request from a person, an event, or the result of an earlier action. The harness assembles a context around it and sends the context to the model. The model runs its own inner loop, one token at a time, until it stops. The harness handles the response: it shows words to a person, or carries out the actions the response requests. If an action produced a result, the result comes back in and the loop runs again; if not, control returns to the person, or the agent stops.

Scope. The primitives describe what happens on the path of a request at run time. Software that ships with a harness but never acts on that path is out of scope: hosting and transport, configuration loading, packaging, the user-interface framework, and the infrastructure that carries telemetry (what is recorded is in scope; the pipe that carries it is not). Model development is out of scope too: the model is held fixed.

Not every harness is an agent's. A program that makes one model call and prints the reply has a harness, but a trivial one: its input is fixed, its context is one message, its output is printed, and its control flow is a straight line. The primitives describe this case as well. What makes a harness an agent's is a Control flow that feeds the results of actions back to the model.

3 The Five Primitives

Algorithm 1 gives a harness as pseudocode, each line labelled with its primitive. Every harness in §5 is an elaboration of it. For each primitive we give its *mechanism* (what it does on the path of

Algorithm 1 A harness. Lines are labelled with the primitive they implement.

```
1:  procedure ControlFlow( )
2:      conversation ← [ ]
3:      loop                                          ▷ outer loop: one request at a time
4:          conversation.append(User(Input( )))       ▷ Input: from a person
5:          loop                                      ▷ inner loop: one request, carried through
6:              ctx ← Context(conversation)           ▷ Context
7:              response ← ModelInterface(ctx)        ▷ Model interface
8:              conversation.append(Assistant(response))
9:              results ← Output(response)            ▷ Output: show words, run actions
10:             if results = ∅ then break             ▷ nothing ran: hand back
11:             conversation.append(User(results))    ▷ Input: from the world
```

a request), its *negative definition* (what it never does, and which primitive does), and its *design space* (the decisions a harness makes within it). The negative definitions make the five mutually exclusive: each piece of a harness has one home, assigned by its function rather than by its name or its location in the code.

3.1 Input: capture what comes in

Mechanism. Capture what enters the agent and hand it to Context. There are two sources: a person (a message, a file, an interrupt, an answer to a question the model asked) and the world (the result of an action, an event, a scheduled trigger). **Never.** Input never decides what the model is shown; that is Context. It never runs anything; that is Output. An answer a person gives that the *harness* consumes, such as an approval, is not input but a Control flow decision; an answer the *model* consumes, returned as a tool result, is input. **Design space.** The source: a terminal, an editor, a chat channel, a command-line argument, an issue tracker, a webhook, a message queue. The kind: text, images, files, audio, the results of actions, interrupts, and mid-turn steering. The second source, results from the world, is what makes acting worthwhile: an action's result is new perception.

3.2 Context: assemble what the model is shown

Mechanism. Assemble, for one call, everything the model is shown: instructions, the tools' descriptions, the conversation so far, and anything retrieved from memory or the environment; and fit it to the window, by truncation, compaction or selection. Context is the agent's working memory, rebuilt on every call. **Never.** Context never calls the model; that is Model interface. It never acts on the world; that is Output. Reading a memory store is Context; writing one, if the model requests it, is an action and so Output. Code that decides what the model sees is Context even when it sits inside a tool: a tool that attaches project instructions to a file it reads, or condenses a web page before returning it, is doing Context's work. **Design space.** What goes in the instructions (identity, rules, the agent's own capabilities, project files such as `AGENTS.md`); how tools are described; how long-term memory comes in (preloaded, retrieved on demand, or searched by the agent through its own tools); the roles each piece is placed in, which carry authority; and what happens when the window fills: drop old turns, summarize them, or include only what the moment needs.

3.3 Model interface: reach the model

Mechanism. Send the assembled context to the model and bring back its response in a form the rest of the harness can read: words, and structured requests to act. This includes the client, the wire format, the translation between the harness's message format and the model's chat template, streaming, retrying a failed request, and switching to a backup model when the request could not be served. **Never.** Model interface never decides what the request contains; that is Context. It never acts on the response; that is Output. It receives a stream; rendering each fragment as it arrives is Output. A change of model because the request could not be served (unavailable, overloaded, refused) is Model interface; a change of model keyed to the work (planning, a cheap model for a side task) is Control flow. **Design space.** A hosted API or local inference; one provider or many, directly or through a translation layer; native tool-call formats or tool calls parsed from text; streaming or whole responses; retry policy and backup models. Model interface is the only primitive that knows which model is inside, which is why changing the model means changing it alone.

3.4 Output: carry the response out

Mechanism. Handle what the model returned: show its words to a person, and carry out each requested action, collecting its result. The tool set, the executor that runs an action, and the environment it runs in are Output. **Never.** Output never decides whether to go again; that is Control flow. It never assembles the next request; that is Context. The model only ever *requests* an action; Output decides whether and how it happens, which is where sandboxes and never-lists live. Several tool calls from one response run concurrently: Output. Several *model* calls run concurrently: Control flow. A program the model writes at run time that calls tools without calling the model is Output; one that sequences model calls is Control flow. **Design space.** The tool set, from one general tool (a shell) to dozens of specific ones (read, write, edit, search, browse, computer use); the action format, from structured tool calls to code the model writes and the harness executes [21] to edit formats such as search-and-replace blocks; the execution environment, from the host machine to a container or a remote sandbox; and the presentation of results, including truncation and error messages, which Yang et al. [22] show affect performance.

3.5 Control flow: decide what happens next

Mechanism. Define the shape of the loop: send results back and go again, hand back to the person, or stop. Step and token budgets, approval gates the harness consumes, delegation to sub-agents and the sequencing of several agents, and routing to a model keyed to the work are all Control flow. **Never.** Control flow never captures input, assembles context, calls the model or runs a tool. It sequences the other four. A workflow in which code fixes the order of model calls [18] and an agent in which the model directs its own process differ here and only here. **Design space.** A single open loop that runs until the model stops requesting actions; a fixed pipeline; an explicit graph; sub-agents with contexts of their own; planning or approval modes that change what is allowed; and stopping rules: step limits, budgets, interrupts, loop detection.

3.6 Necessity

Each primitive answers a question no other answers: what comes in, what the model is shown, how the model is reached, what is done with what comes out, and what happens next. Appendix A gives the five minimal harnesses from which the definitions were drawn, built one primitive at a time

around a fixed model and task. Each version fails in a specific way without the next primitive: with only Model interface, the model answers but cannot act and nobody can ask it anything else; with Input, anyone can ask and it still cannot act; with Output, it acts once and the result goes nowhere; with Control flow, it finishes the task but does not know what it is or where it is, and spends actions finding out; with Context, it knows. This shows that each primitive removes a failure the others do not. It does not show that no sixth primitive could ever be needed; §5 tests that against existing harnesses, and §8 says what would refute it.

4 Three Levels: Primitive, Design Decision, Production Concern

Descriptions of harnesses mix three kinds of statement. "It has a loop" names a primitive. "It gives the model forty tools" names a decision within one. "It runs commands in a sandbox" names a measure taken for a reason that applies to every primitive. Keeping the three apart is the main use of the vocabulary, and the deconstruction in §5 records each component at its level.

Primitives. A primitive is a mechanism every harness has, defined by its function on the request path (§3). A harness has exactly five. A harness that lacks one is not an agent's harness, or is not a harness.

Design decisions. A design decision is one way of implementing a primitive. It is the level at which harnesses differ, and the level at which they are usually described: a single shell tool or a set of file tools (Output); a repository map or a project file in the instructions (Context); a loop that runs until the model stops or a graph with named nodes (Control flow); one provider or a gateway to many (Model interface); a terminal or a message queue (Input). A design decision never adds a primitive; it fills one. The test for whether something is a design decision is whether a different harness could make a different choice and still be a harness.

Production concerns. A production concern is a question asked of every primitive, whose answer changes how some primitives are implemented. We use six (Table 1): what should outlast the session (persistence); what could do harm, and who could make it (safety and security); what did it do (observability); what if something breaks (resilience); what is slow or costly (performance); and how do we know it still works (evaluation). A concern is not a part of the harness. A sandbox is not a sixth primitive; it is the safety answer in Output. A memory store is not a sixth primitive; it is the persistence answer in Context and Output. The test for whether something is a concern is whether it is motivated by a question that could equally be asked of a different primitive.

The concerns interact. Persistence creates a security surface: a poisoned memory steers every later session. Streaming changes safety: an interrupt can stop a reply mid-sentence, and the harness must record what was said rather than a finished answer. Summarizing changes persistence: the summary must say where the originals are kept. Treating concerns as questions asked of each primitive makes these interactions visible; treating them as separate components hides them.

4.1 Rules for the cases at the boundaries

The negative definitions decide most cases. The following rules decide the rest. Each was needed for a component found in §5.

Table 1: Six production concerns as questions asked of each primitive. Each cell names what the concern changes in that primitive; "—" means we found nothing essential there in the harnesses examined.

	Persistence *What outlasts the session?*	**Safety & security** *What could do harm, and who could make it?*	**Observability** *What did it do?*	**Resilience** *What if something breaks?*	**Performance** *What is slow or costly?*	**Evaluation** *Does it still work?*
Input	an unfinished session offered back	approvals asked of a person	refusals recorded	input ending; interrupts	—	yes to every approval in a test run
Context	memories; resumed transcripts	untrusted content; poisoned memory; roles and authority	—	compaction when too long	caching the unchanged prefix; trimming results	the instructions under test
Model interface	—	—	tokens, latency, cost per call	retries; a backup model	streaming	the model under test
Output	—	sandboxes; never-lists; secrets hidden; pinned images	what each action did	time limits on actions	parallel actions; smaller models for side tasks	the tools under test
Control flow	the task in progress; a diary of every message	runaway loops; the record kept out of reach	each decision	resume after a crash	—	the loop under test

Function, not location.
A component is assigned by what it does, not by the file or the name it has. Code inside a tool that decides what the model sees is Context. An approval decision made inside a tool's runtime is Control flow.

The tool channel.
The model drives other primitives through tool calls: spawning a sub-agent, entering a planning mode, writing a plan, searching for tools, asking the person a question. The call is Output; its effect is assigned to the primitive it changes.

Backup versus routing.
A model change triggered by a request that could not be served is Model interface. A model change keyed to the work is Control flow. A fixed model for one class of request, such as compaction summaries, is a request setting and Model interface.

Receiving versus rendering.
Receiving a response as a stream is Model interface; rendering each fragment is Output.

Parallel actions versus parallel calls.
Several tool calls from one response run concurrently: Output. Several model calls issued concurrently, to divide a task or to vote: Control flow.

The reader of a record.
A record kept for the model (a searchable transcript) is Context. A record kept for the operator (a trace) is observability. An artifact for the user (a diff view) is Output. One file may serve several readers, and each use is assigned separately.

The consumer of an answer.
An answer the harness consumes (an approval) is Control flow. An answer the model consumes (the result of an ask-the-user tool) is Input.

Direct actions by a person.
A command the person runs through the harness's own executor, whose output enters the conversation, is Input, although it reuses Output's executor.

Model-written programs.
A program the model writes at run time is CONTROL FLOW if it sequences model calls and OUTPUT if it sequences only tools.

Bundles.
A mode, a skill, a hook system or an agent profile is packaging that bundles decisions across several primitives. Decomposed, each element has one home: a planning mode is a context template, a restricted tool set and an approval gate; a hook system is a dispatcher (CONTROL FLOW) whose events each affect one primitive.

Nested harnesses.
A sub-agent is a harness of its own, and delegation to it is CONTROL FLOW of the outer harness. The same holds for model-backed components inside a harness, such as an approval reviewer or a termination judge: each is CONTROL FLOW with a MODEL INTERFACE call.

Primitives as services.
One primitive may use another's mechanism to do its job: CONTEXT asks the model to summarize old turns before compacting them; a web-fetch tool in OUTPUT asks the model to condense a page before returning it. The component is assigned by what it *decides* (what to compact, what to return), and the call it makes is a use of MODEL INTERFACE. The negative definitions are about responsibility, not about which function calls which.

Tool protocols.
A protocol such as MCP [12] provides pluggable OUTPUT (third-party tools). Its other features are assigned by function: resources and prompts are CONTEXT; sampling borrows the client's MODEL INTERFACE; elicitation is INPUT.

5 Deconstruction of Six Harnesses

If the position holds, a harness built by someone else, for a different purpose, at a different scale, should decompose into the five primitives with nothing left over, and its differences from other harnesses should be locatable as design decisions within a primitive. We test this on six.

Selection. We chose harnesses in wide use whose primary sources were available, across three kinds: two command-line coding agents from model developers (Claude Code [3] and Codex CLI [14]), two open-source coding agents built outside the model developers (opencode [1] and pi [25]), one general-purpose agent built for messaging platforms as well as the terminal (Hermes Agent [13]), and one workflow framework (LangGraph [9]), for which we deconstruct the canonical agent it ships, a ReAct loop, together with the runtime that executes it. The workflow framework is included to test the claim that a workflow and an agent differ only in CONTROL FLOW. Five of the six publish their source; Claude Code does not, and is deconstructed from its documentation. The six are not a sample of all harnesses, and §8 says what that leaves untested.

Method. For each harness we (i) read the source at a pinned commit, or the documentation on a stated date, and mapped the path of a request from the moment a message arrives to the next model call; (ii) assigned every component on that path to exactly one primitive, or to one production concern together with the primitive it changes, by the definitions of §3 and the rules of §4.1, citing a file and line range or a page and section for each; and (iii) recorded every component that was hard to place and how it was resolved, or that it was not. Components outside the request path (hosting, packaging, UI frameworks) were noted and left unassigned. The evidence is in Appendix B and in the repository as a file per harness; the subsections below summarize it.

5.1 Claude Code

Anthropic's coding harness, deconstructed from its documentation as read on 15 October 2026 [3]; its source is not published. The loop is the open one of Algorithm 1: the model is called until it returns a response with no tool calls, and a turn's result carries why it ended (done, a turn limit, a cost budget, an error). Input has one entry point for everything that starts a turn: typed prompts, piped input, messages queued mid-turn and delivered after the current tool batch, scheduled prompts, and reports from background sub-agents. Context is built as a stable cached prefix (the system prompt, the tools, then project files such as `CLAUDE.md`) with later updates appended as messages or system reminders; tool schemas for MCP servers are deferred behind a search tool; and compaction, on reaching the window, is a separate request that re-injects the project files, memory, recently read files and invoked skills. Model interface is the Messages API with streaming, up to ten retries that never re-issue after a completed tool block, idle watchdogs, and a fallback chain the user configures. Output is a fixed built-in tool set plus MCP tools; read-only calls run concurrently and mutating calls in sequence; shell commands run under an OS sandbox with a domain-allowlist proxy; large results spill to files; file edits are checkpointed for rewind. Control flow adds a six-step permission pipeline (hooks, deny rules, ask rules, the permission mode, allow rules, then a prompt), delegation through an agent tool to a depth of three, and, at scale, a model-written JavaScript workflow the runtime executes, which sequences model calls and is therefore Control flow by the rule on model-written programs.

Three components were hard to place. The web-fetch tool calls the model itself to condense a page before the main model sees it: the call is Output, the condensing is a Context decision, and the extra request is Model interface. The default permission gate in "auto" mode is a second model asked to classify each action: a Control flow gate implemented as a model call with its own stripped context. And a server-side variant of that classifier runs inside the API during the session's own requests, so the harness only consumes its verdict; we return to this in §5.7.

5.2 Codex CLI

(pending)

5.3 opencode

An open-source coding harness by Anomaly, at commit `654b79b` [1]; it was rewritten since earlier versions, and the deconstruction describes the current one. Each session is an event-sourced record in SQLite, and one fibre per session drains a durable inbox: Input is a row in that inbox, from a person, a shell command, a skill, or the harness itself, marked for delivery either at the next step boundary ("steer") or after the current turn ("queue"). Context re-projects the event history into model messages on every call, with instructions placed in a cached prefix and changes to them frozen into the history as dated events; compaction is triggered at about ninety percent of the window, using provider-measured token counts plus a local estimate, keeps the newest fifteen thousand tokens verbatim, and can be delegated to the provider's native compaction. Model interface is an in-house request type compiled per provider protocol (Anthropic Messages, OpenAI Responses, Bedrock Converse, Gemini), streamed over HTTP or a stateful WebSocket, with a retry policy split by whether output had already started. Output runs tool calls concurrently as they stream, and most tools are reachable only through one `execute` tool that runs model-written JavaScript in an interpreter that cannot call the model, which the rule on model-written programs places in Output. Control flow continues while a step made tool calls, stops on a text-only finish, enforces an

optional per-agent step budget, and delegates to child sessions (depth one by default), whose results return as tool text or as a synthetic message.

Hard cases: every tool asserts its own permission, so an approval gate the harness consumes lives inside the tool's code (CONTROL FLOW by function); the step budget is enforced by appending an assistant message and disabling tools, a CONTROL FLOW decision carried out by editing what the model sees; and the compaction service both decides what to summarize and issues the model call itself. Native provider-side compaction leaves the harness's CONTEXT holding an item it cannot inspect.

5.4 pi

A terminal coding agent by Mario Zechner, at commit `5ab7fe7` [25], built in layers: a minimal agent loop, a provider layer, and a coding-agent session on top. INPUT comes from a terminal editor, a print mode, or a JSON-RPC mode, all into one `prompt()` call, with mid-run messages split into steering (after this turn) and follow-ups (after the agent would have stopped). CONTEXT is a projection of a persisted session tree: a compaction entry, the kept tail, and context edits, with the system prompt kept as named sections and diffed into patches, so that replaying the file reproduces the exact request; compaction triggers when the context exceeds the window less a reserve, and keeps the most recent twenty thousand tokens. MODEL INTERFACE is one streaming function behind about fifteen provider adapters, with the SDKs' own retries disabled so that the harness owns a two-tier retry policy, and an overflow classifier that routes a too-long request to compaction rather than to a retry. OUTPUT is a set of plain tool objects run in parallel by default, with mutations to one file serialized; there is no sandbox in the core (OS sandboxing and permission prompts are extensions), and a "code mode" tool runs model-written scripts in a QuickJS sandbox. CONTROL FLOW is a two-level loop (tool calls and steering inside, follow-ups outside) plus a session-level loop for retries, overflow recovery and extension continuations; sub-agents are not in the core, though an example spawns child processes; and per-request routing between "virtual models" is keyed to the reason for the request.

Hard cases: a wire-level hook lets an extension rewrite the request body, which is CONTEXT by function though it sits in the adapter; the virtual-model router has a "retry" branch that overlaps the backup-model rule (we place it in CONTROL FLOW, since the router decides by the state of the work); and the code-mode script can call auxiliary models from inside the sandbox, a model call issued from inside a tool, which we discuss in §5.7. A telemetry interface exists, but no span is emitted at this commit.

5.5 Hermes Agent

A personal agent by Nous Research, at commit `7bf3766` [13], with one agent core behind a terminal interface, about twenty-five messaging-platform adapters, an HTTP server, an ACP adapter and a cron scheduler. INPUT is one normalized message event shared by all of these; mid-turn messages become steering rather than new turns. CONTEXT is a three-tier system prompt, kept byte-stable so that caching holds: identity, then project files (capped by size), then volatile memory, skills and the time; compaction at half the window prunes, then summarizes with an auxiliary model; oversized tool output is spilled to files rather than dropped; and a tool-search bridge lets the model fetch tool schemas on demand. MODEL INTERFACE is an OpenAI-style chat-completions client by default with pluggable transports for Anthropic, Responses and Bedrock, streaming on, three retries with backoff, then a configurable fallback chain, then a bounded auto-recovery wait. OUTPUT sends a fixed core tool set on every call, splits each batch into parallel-safe segments, runs terminal commands behind

pluggable sandboxes (local, Docker, Modal, and others) and a regex-plus-human approval gate, and persists each tool-call row before executing it. CONTROL FLOW is a loop of named phases, each returning a verdict, with an unlimited iteration budget by default, "stop-gate" nudges that append a synthetic message and go again, flat delegation to threaded children, and task-keyed routing to auxiliary models.

Two components do not fit cleanly. In one mode the whole turn is handed to an external Codex process, and Hermes keeps only input, persistence and a fallback: the five primitives collapse into one opaque external harness that is neither a sub-agent (there is no outer loop) nor the model. And "providers" reached over ACP make the thing behind the MODEL INTERFACE another agent. Both are nested harnesses by the rule of §4.1, but of an unusual shape: the nesting is at the MODEL INTERFACE, not in CONTROL FLOW.

5.6 A LangGraph workflow

LangGraph [9], at commit `5eb60ae`, is a framework for writing control flow as a graph over a checkpointed state, executed by a runtime in bulk-synchronous supersteps. We deconstruct the agent it ships, `create_react_agent`, together with that runtime. INPUT is a write to a named channel of a typed state, merged by a user-chosen reducer, whether it is a person's message, a tool result, or a resume value after an interrupt. CONTEXT is exactly the state's message list, optionally prefixed by one system message; fitting it to the window is left entirely to a user-written hook before the model call, and the framework provides only a primitive for removing messages. MODEL INTERFACE is one call to a LangChain chat model per step, and the framework contains no client of its own, no streaming of its own, and no backup model; retries and timeouts are per-node policies, off by default. OUTPUT is a tool node that runs every tool call of a response concurrently in-process, turns results and (configurably) errors into messages, and lets a tool return a routing command. CONTROL FLOW is the graph: "go again" is a conditional edge from the agent node to the tool node and back, "stop" is an empty task set or a recursion limit, and a pause is an exception the runtime persists as a checkpoint write and resumes by re-running the node.

This is the clearest case of the three-level distinction. The framework fixes the five primitives' mechanisms and leaves most of the design decisions to user code: the edges, the stopping rule, the window-fitting hook, the approval gate around an interrupt, dynamic model selection, retry policies. A workflow and an agent built with it differ in the graph, which is CONTROL FLOW, and nowhere else. The hard cases are the step counter injected into the state (it looks like CONTEXT but no model sees it; it is a CONTROL FLOW budget), and a scheduling packet that carries a state snapshot to the tool task (CONTROL FLOW carrying CONTEXT).

5.7 Summary

Table 2 gives the six side by side, one row per primitive. Every component we found has a home. The differences between the harnesses are in the rows: what comes in and from where, what the context holds and how it is fitted, how the model is reached, which tools exist and where they run, and what shape the loop has. None of the six required a sixth row.

What was hard to place. The hard cases are collected in Appendix B, and four recur across harnesses.

- **Primitives that call the model.** In Claude Code, opencode and pi, compaction decides what to summarize and then calls the model to summarize it; Claude Code's web-fetch tool and pi's code mode call models from inside a tool. Read as a call graph, these break the rule that

Table 2: Six harnesses by primitive. Each cell is the design decision that harness made within the primitive; the evidence for each is in Appendix B.

	Claude Code	Codex CLI	opencode	pi	Hermes Agent	LangGraph
Input	prompts, stdin, queued mid-turn messages, scheduled prompts, sub-agent reports, tool results, answers to an ask-user tool	(pending)	durable inbox rows (person, shell, skill, synthetic), delivered as steer or queue at step boundaries	terminal, print and RPC modes into one prompt call; steer and follow-up queues	one message event from CLI, 25 messaging gateways, HTTP, ACP, cron; mid-turn steering	writes to a typed state channel, merged by a reducer: messages, tool results, resume values
Context	cached prefix (system, tools, CLAUDE.md) plus appended reminders; deferred MCP schemas; compaction re-injects memory and recent files	(pending)	event-sourced history re-projected per call; cached instructions as dated events; compaction at ∼90%, 15k tail, or provider-native	projection of a session tree; sectioned prompt diffed into patches; compaction past a reserve, 20k tail	three-tier byte-stable prompt; memory and project files; compaction at 50% by prune then summarize; tool-search bridge	the state's messages plus one system message; window-fitting left to a user hook
Model interface	Messages API, streaming; 10 retries; idle watchdogs; user-configured fallback chain	(pending)	in-house request compiled per protocol; SSE or WebSocket; retry split by whether output started	one stream function over ∼15 adapters; SDK retries off; two-tier retry; overflow routed to compaction	chat-completions client with pluggable transports; streaming; 3 retries, fallback chain, auto-recovery	one LangChain chat-model call per step; no client, streaming or fallback of its own
Output	built-in tools plus MCP; read-only calls concurrent; OS sandbox with domain allowlist; results spill to files; edit checkpoints	(pending)	tools run concurrently as they stream; each asserts its own permission; most reached through a code-mode interpreter	plain tool objects, parallel by default; no core sandbox; QuickJS code mode	core tools plus bridge; parallel-safe segments; sandboxed terminal (local, Docker, Modal, . . .); approval gate	tool node runs calls concurrently in-process; errors become messages; tools may return routing commands
Control flow	loop until no tool calls; six-step permission gate, with a classifier model by default; agent tool to depth 3; model-written workflows	(pending)	one fibre per session; continue while tools called; per-agent step budget; child sessions, depth 1	inner tool loop, outer follow-up loop, session loop for retries; virtual-model routing	phase-verdict loop; unlimited budget; stop-gate nudges; flat threaded delegation; task-keyed routing	a user-written graph run as supersteps; stop on empty task set or recursion limit; interrupts checkpointed

Context and Output never call the model. Read as responsibility, they do not: the component still decides one thing, and uses Model interface to do it. We added the rule on primitives as services (§4.1) for these cases.

- **Approval gates inside tools.** In opencode and Hermes the code that asks the person sits inside the tool that would act, and in Codex the approval requirement depends on what the sandbox allows. By function the gate is Control flow and the enforcement is Output; by location they are one file. The two also read each other, which is the interaction of concerns noted in §4.
- **Decisions carried out by editing the context.** opencode enforces a step budget by appending an assistant message and disabling tools; Hermes's stop gates append a synthetic message and go again; LangGraph replaces the model's answer with a canned message when the step budget is reached. Each is a Control flow decision whose mechanism is a change to what the model is shown. The decision and the mechanism have different homes, and we record both.
- **Nesting at the model interface.** Hermes can hand a whole turn to an external Codex process, or reach an agent over ACP as if it were a model. The thing behind Model interface is then another harness. The nested-harness rule covers it, but the nesting is at a different primitive from the sub-agent case, and the outer harness keeps only Input, persistence and a fallback.

One finding is not a hard case but a boundary question. Claude Code's server-side classifier runs inside the API during the session's own requests, and opencode can delegate compaction to the provider. In both, a function the vocabulary places in the harness runs on the provider's side of the API, and the local harness holds only a verdict or an opaque item. The primitives still apply: the harness is split across two machines, not reduced to four parts. But it means the boundary of §2 is at the model's weights, not at the API, and a deconstruction from a client's source alone will miss the part that runs behind the API. We found no component in the six that satisfied none of the five definitions.

6 Using the Vocabulary

6.1 Locating a failure

When an agent fails, the primitives give a short list of places to look, in order. The order matters: each question rules out a place the next question assumes.

1. Context**: was the model shown what it needed?** Read the request that was sent. If the file, the result or the instruction is not in it, the model could not have used it, and nothing downstream is at fault.
2. Output**: did what it asked for happen?** Compare the action the model requested with what ran. A parser that dropped a call, a sandbox that refused it, a tool that returned a truncated result, and an error message the model could not read are all Output.
3. Control flow**: did the loop stop early or run too long?** A step limit reached before the task was done, a hand-back on a response that still had work in it, and a loop that repeated the same action are Control flow.
4. Model interface**: was the response read correctly?** A tool call emitted in the right format but not recognized, a stream cut off and treated as complete, and a backup model substituted without notice are Model interface.
5. Input**: did the right thing come in?** A result delivered to the wrong turn, an interrupt lost, a person's answer consumed by the harness when the model needed it.
6. **Only then, the model.** If the model was shown what it needed, asked for the wrong thing, and its request was carried out as asked, the failure is the model's: its knowledge, its habits, or

its size.

Two examples from our own transcripts. A small model, after instruction-tuning, sometimes emitted a second tool call as bare JSON without its `<tool_call>` tags. The harness read the JSON as words, found no action, and handed back. Question 4 locates this in MODEL INTERFACE, whose parser defines what counts as a call; the fix was on the model side (training on the format) but the diagnosis was the harness's. In the same experiments, four kinds of task could never pass during reinforcement learning because the check compared the folder before and after, and the harness's own memory directory changed the folder on every run. No question in the list locates this, because the fault was in the evaluation, a concern, not in a primitive. Building the harness around a capable model first, until it passed every task in the evaluation, is what let every later failure be attributed to the model swapped in.

6.2 Reporting a harness

If *Agent* = *Harness*(*Model*), a benchmark score belongs to the pair. Holding GPT-4 Turbo fixed, Yang et al. [22] report 1.31% of SWE-bench [6] resolved with retrieval-augmented generation and 12.47% with their agent, and on SWE-bench Lite 11.00% for a shell-only agent against 18.00% for SWE-agent, with only the harness changing. Anthropic notes that performance "can vary significantly based on this scaffolding, even when using the same underlying AI model" [2]; METR found that tailoring a scaffold to one model more than doubled the length of tasks it could complete [10], and later that two mature harnesses did not outperform their default scaffolds [11], which suggests the effect is largest where the design decisions differ most. Zhang et al. [26] argue that the harness should be disclosed whenever agents are compared, and Kapoor et al. [8] evaluate models, scaffolds and benchmarks as separate dimensions. A disclosure needs a schema. We suggest the five primitives: the input sources; what the context holds and how it is fitted to the window; the provider, decoding settings and retry policy; the tools, their output formats and the execution environment; and the loop's shape and stopping rules. The summary table of §5 is such a disclosure for six harnesses.

6.3 Porting

Because only MODEL INTERFACE knows which model is inside, a harness can be moved to a new model or provider by changing that primitive alone, translating requests and responses to the new format. Our own harness was built around a hosted model and later run with a local one by replacing one function (Appendix A). A harness loses this property when model-specific details leak into other primitives: a provider's tool-call format assumed in OUTPUT, or a provider's error strings matched in CONTEXT management. The deconstruction records such leaks where we found them.

6.4 Training a model for its harness

The primitives also say what a model trained for one harness must learn: the context layout it will be shown (CONTEXT), the format in which it must request actions and the shape of the results it will read (OUTPUT), and the loop it runs in (CONTROL FLOW). A model post-trained for many harnesses must guess which one it is in; a model trained inside one does not.[1]

[1] In the companion book, a 0.6-billion-parameter model instruction-tuned on sessions recorded in its own harness rose from 2 to 18 of 32 held-out tasks in that harness, where the same base model post-trained by its developers for general use scored 14. The experiment, its data and its checks are described there [4]; this paper makes no claim that rests on it.

7 Objections

Memory is a sixth primitive. Memory is prominent in agent architectures [15, 20]. We place it as a store in the world that CONTEXT reads and OUTPUT writes, governed by the persistence concern. The model has no memory of its own, and every implementation we examined (memory files loaded into the instructions, stores searched through tools, summaries carried forward, transcripts replayed on resume) is a decision about what CONTEXT includes and which actions OUTPUT offers. Making memory a primitive would split one decision, what the model is shown, across two components.

Planning is a sixth primitive. Planning is either something the model does in its own tokens, which needs no harness support, or a structure the harness imposes: a mode that restricts actions, a plan kept in a file, a separate planning call. Each of these is CONTROL FLOW (which steps run, under what restrictions) or CONTEXT (where the plan is kept and shown). Every planning feature in §5 decomposed this way.

A workflow is not a harness. A workflow fixes the sequence of model calls in code; an agent lets the model direct its own process [18]. Both are harnesses. They differ in CONTROL FLOW, and only there: a graph with named nodes and edges is one design decision, an open loop is another. The LangGraph deconstruction in §5 is included for this reason.

Multi-agent systems are something else. A system of several agents, each a model in a harness, is itself a harness whose OUTPUT includes delegating to another agent and whose INPUT includes that agent's result. Sub-agents, hand-offs and orchestrator–worker patterns are CONTROL FLOW decisions at the outer level. The primitives compose: a harness of harnesses still has five.

The harness will be absorbed into the model. Models increasingly learn behaviours that harnesses once imposed: when to stop, how to call tools in a trained format, when to reason before acting. But a model cannot open a file, keep a log or refuse a command on its own. Training moves *policy* into the model (what to do); the *mechanism* (how anything happens) remains in the harness. If models are trained inside a particular harness (§6), the harness defines the world the model was trained for, and its decomposition matters more, not less.

Five boxes can describe anything, and so explain nothing. The primitives are a coordinate system, not an explanation. Their value is in placing a design decision, a failure or a difference between two systems in one location, so that it can be studied, compared and changed independently of the rest. The claim is falsifiable in a specific way (§8), and the content is in the design decisions of §3 and the deconstruction of §5, not in the boxes.

8 Limitations and Threats to Validity

What would refute the claim. The position fails if a component of a working harness is found that (i) satisfies none of the five definitions, (ii) is not the answer to one of the six concerns within a primitive, and (iii) is not a nested harness. We invite reports of such a component. Three classes seemed most likely to produce one before the deconstruction: learned components inside the harness, search or decoding loops, and control flow generated by the model at run time. The first and third arose in the six harnesses and were placed by the rules of §4.1; search and decoding loops did not arise and remain untested.

Selection. Six harnesses, five of them coding agents and one a workflow framework used for a coding example. Harnesses for other domains (browsing, voice, robotics, customer service) were not examined, and the design spaces in §3 are drawn from what these six do. The selection criterion was wide use and the availability of primary sources; it is not a random sample.

Documentation-only analysis. Claude Code's source is not published. Its deconstruction rests on its documentation and Anthropic's published descriptions, which state what the harness does but not always how. Negative definitions are harder to confirm from documentation than from source, and a documented feature may differ from the shipped one.

The harness is not always in one place. Two of the six run part of their harness behind the provider's API (§5.7). A deconstruction from client source or documentation describes the local part; the part on the provider's side is visible only through its effects. As providers absorb more harness functions, this gap will grow.

Products change. Every assignment is pinned to a commit or a documentation date. Several of the six harnesses change weekly. The deconstruction describes them on those dates, and a reader checking it later should use the pinned versions.

Who made the assignments. The assignments were made by the author with an AI coding agent as a research instrument (see the disclosure), against the definitions of §3 and the rules of §4.1, and each cites its evidence. No second, independent decomposition was made, so the agreement between two people applying the definitions is unmeasured. The rules in §4.1 are where disagreement would most likely arise: they follow from the definitions, but another reader could draw some boundaries elsewhere.

Stipulated boundaries. Backup versus routing, receiving versus rendering, the reader of a record and the consumer of an answer are rules we had to state. Their value is in making each choice explicit and checkable, not in being the only possible choice.

No experimental test. The paper does not vary one primitive at a time and measure the effect on performance. Fan et al. [5] do this for three decisions in a coding harness; a study across all five primitives would test whether the vocabulary captures the factors that matter, and is future work. The one quantitative result we cite (§6, footnote) comes from the companion book and is not part of this paper's argument.

The model is held fixed. Training a model on traces from a harness moves work between the two components. We note this in §6 and do not analyze the trade-off.

9 Related Work

Agents and language agents. In artificial intelligence, an agent perceives its environment and acts on it [16], in a cycle supported by memory. The loop that harnesses implement goes back to ReAct [23], which interleaves a model's reasoning with actions whose results return to it as observations. Later patterns live in particular primitives: Reflexion [19] writes reflections on failed attempts and carries them into the next, a decision in Control flow and Context; MemGPT

Table 3: The five primitives, the step of the agent cycle each serves, and the CoALA component it corresponds to.

Primitive	Mechanism	In the agent cycle	CoALA
Input	capture requests and results	perceive	observations
Context	assemble what the model is shown	working and long-term memory	working memory; retrieval
Model interface	request a response	decide	reasoning (by the model)
Output	show words, carry out actions	act	grounding actions; learning (writes)
Control flow	go again, hand back, or stop	the cycle	decision cycle

[15] pages information in and out of a limited window, a decision in Context; Toolformer [17] trains the model to emit tool calls, moving policy into the model while the calls are still carried out by Output.

CoALA. Sumers et al. [20] organize language agents by memory (working, episodic, semantic, procedural), an action space of external and internal actions, and a decision cycle. CoALA describes what an agent *is*, as a cognitive architecture; this paper describes what software an agent is *made of*. The correspondence is in Table 3. Two things in this paper have no counterpart in CoALA: Model interface as a component, and the production concerns as a level of description distinct from the architecture. CoALA's memory modules, with the retrieval and learning actions that read and write them, are design decisions in Context and Output here, not a component.

Agent-computer interfaces. Yang et al. [22] show that the interface through which a model acts on a computer (the commands it is given, their output format, guardrails such as linting edits) changes performance on software tasks, and name it the agent-computer interface. In our terms the interface is a set of design decisions in Output and Context. The present paper asks what the complete set of components is, of which the interface is part.

Workflows and agents. Schluntz and Zhang [18] distinguish workflows, in which code fixes the sequence of model calls, from agents, in which the model directs its own process, and describe patterns such as prompt chaining, routing and orchestrator–workers built on an "augmented LLM". We treat the distinction as a property of Control flow, and decompose the augmented LLM into Context (retrieval, memory) and Output (tools) around a Model interface.

The harness as a variable. Zhang et al. [26] argue that harness configuration can account for more variance in performance than the choice of model, and ask that the harness be disclosed when agents are compared. Fan et al. [5] hold a coding harness's loop fixed and vary planning, the action space and context management across four models, finding that each changes agent behaviour. Their three dimensions were chosen independently of this paper and each falls within one primitive: planning in Control flow, the action space in Output, context management in Context. The present paper supplies what both assume: a statement of what the harness's parts are, which a disclosure can follow and a study can vary. Jin et al. [7] use "primitives" for recurring patterns of model calls in multi-agent systems (review, voting, planning and execution); in our terms these are shapes of Control flow, and the two uses of the word are complementary.

10 Conclusion

An LLM agent is a model wrapped in a harness, and a harness is five mechanistic primitives: Input, Context, Model interface, Output and Control flow. What distinguishes one harness from another is the design decisions made within those primitives, and what makes a harness fit to run is the answers it gives to six questions asked of each. Six harnesses in wide use, deconstructed from their sources, fit this description with nothing left over. The vocabulary gives an engineer five places to look before blaming the model, and gives a benchmark a way to say which harness produced a score.

Disclosure of AI use

AI tools were used in two roles.

As a research instrument. The deconstruction of §5 was carried out with an AI coding agent (Claude, Anthropic) under the author's direction. The agent read each harness's source at the pinned commit, or its documentation on the stated date, applied the definitions of §3 and the rules of §4.1, and recorded every assignment with a file-and-line or URL citation. The author reviewed the assignments and the hard cases. Every assignment in Appendix B can be checked against its cited source without the agent.

As a writing assistant. The author set the position, the structure, the definitions and the three-level vocabulary, and reviewed every section. An AI assistant drafted prose from the author's outline and the companion repository, and edited for consistency. The minimal harnesses in Appendix A and the transcripts they cite are the author's, recorded in the companion repository.

References

[1] Anomaly and contributors. opencode. https://github.com/anomalyco/opencode, 2026. Commit 654b79b, 2026-10-15.

[2] Anthropic. Raising the bar on SWE-bench verified with Claude 3.5 Sonnet. https://www.anthropic.com/research/swe-bench-sonnet, 2025.

[3] Anthropic. Claude Code documentation. https://code.claude.com/docs/en/how-claude-code-works, 2026. Documentation as read on 2026-10-15.

[4] Chase Dovey. Building agents: From nothing to a model, a harness, and a model trained in its harness. https://github.com/averagejoeslab/building-agents, 2026. Book and companion repository, Average Joes Lab.

[5] Run-Ze Fan, Zihao Zhang, Simin Ma, Yebowen Hu, Shouju Wang, Kaiqiang Song, Fei Liu, Hamed Zamani, and Xiaoyang Wang. An empirical study of harness design for coding agents. *arXiv preprint arXiv:2609.20804*, 2026.

[6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. arXiv:2310.06770.

[7] Haibo Jin, Peng Kuang, Ye Yu, Xiaopeng Yuan, and Haohan Wang. Agent primitives: Reusable latent building blocks for multi-agent systems. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.03695.

[8] Sayash Kapoor et al. Holistic agent leaderboard, 2025.

[9] LangChain. LangGraph. https://github.com/langchain-ai/langgraph, 2026. Commit 5eb60ae, 2026-10-14; documentation at https://docs.langchain.com/oss/python/langgraph/overview.

[10] METR. An update on our preliminary evaluations of Claude 3.5 Sonnet and o1. https://metr.org/blog/2025-01-31-update-sonnet-o1-evals/, 2025.

[11] METR. Measuring time horizon using Claude Code and Codex. https://evals.alignment.org/notes/2026-02-13-measuring-time-horizon-using-claude-code-and-codex/, 2026.

[12] Model Context Protocol. Model Context Protocol: Architecture overview. https://modelcontextprotocol.io/docs/learn/architecture, 2026. Accessed 2026-10-15.

[13] Nous Research. Hermes Agent. https://github.com/NousResearch/hermes-agent, 2026. Commit 7bf3766, 2026-10-15.

[14] OpenAI. Codex CLI. https://github.com/openai/codex, 2026. Commit f0c5077, 2026-10-15.

[15] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.

[16] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.

[17] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023. arXiv:2302.04761.

[18] Erik Schluntz and Barry Zhang. Building effective agents. https://www.anthropic.com/engineering/building-effective-agents, 2024.

[19] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. arXiv:2303.11366.

[20] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024. arXiv:2309.02427.

[21] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning*, 2024. arXiv:2402.01030.

[22] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024. arXiv:2405.15793.

[23] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. arXiv:2210.03629.

[24] J. Young. Effective harnesses for long-running agents. https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents, 2025.

[25] Mario Zechner. pi: a minimal coding agent. https://github.com/badlogic/pi-mono, 2026. Commit 5ab7fe7, 2026-10-15.

[26] Yunbei Zhang, Janet Wang, Yingqiang Ge, Weijie Xu, Jihun Hamm, and Chandan K. Reddy. Stop comparing LLM agents without disclosing the harness. *arXiv preprint arXiv:2605.23950*, 2026.

A The Minimal Harnesses, One Primitive at a Time

The definitions in §3 were drawn from five programs, each the previous one plus one primitive, all run on the same task: a two-file Python project whose `total()` subtracts a discount as a flat amount where the test expects a percentage, and the request *"The tests are failing. Find out why and fix it."* The model is Claude Sonnet through Anthropic's Messages API; the files are `harness/1_model_interface.py` to `harness/quark.py` in the companion repository [4], reproduced here without their dependency headers. The transcripts quoted are from the recorded runs there. In the transcripts, `>` is what the person typed, `<` what the model said, and `$` a command it ran.

A.1 Model interface alone

```
from anthropic import Anthropic

model = Anthropic()


def request_response(context):
    return model.messages.create(model="claude-sonnet-5-5", max_tokens=16384, **context)


response = request_response({"messages": [{"role": "user", "content": "The tests are failing. Find out why and
     fix it."}]})
print(response.model_dump_json(indent=2))
```

The model answers, but it can only answer: *"I don't have access to your code, test files, or test output. Nothing came through in your message besides the request."* The request is written into the program, so nobody can ask it anything else. What is missing is a way for something to come in.

A.2 Plus Input

```


def capture_input():
    return input("\n> ")
```

Anyone can now type the request, and the answer is the same: *"I don't have the code or the test output yet, so I can't tell why the tests are failing. Could you share…"* It can only talk. What is missing is a way to act.

A.3 Plus Output

```
model = Anthropic()

bash = {"name": "bash", "description": "Run a shell command",
        "input_schema": {"type": "object", "properties": {"command": {"type": "string"}}, "required": ["
     command"]}}


def request_response(context):
    return model.messages.create(model="claude-sonnet-5-5", max_tokens=16384, **context)


def capture_input():
    return input("\n> ")


def handle_output(response):
    tool_results = []
    for block in response.content:
        if block.type == "text":
            print("< " + block.text.replace("\n", "\n  "))
        if block.type == "tool_use":
```

A tool is described to the model, and `handle_output` runs each command the model asks for and collects the result:

```
> The tests are failing. Find out why and fix it.
< I'll start by exploring the project and running the tests.
$ pwd; ls -la; git status 2>/dev/null | head
```

Then it stops. The command ran and printed the files, and the result went nowhere: nothing carries it back to the model. What is missing is the decision to go again.

A.4 Plus Control flow

```


def control_flow():
    conversation = []
    while True:
        conversation.append({"role": "user", "content": capture_input()})
        while True:
            response = request_response({"tools": [bash], "messages": conversation})
            conversation.append({"role": "assistant", "content": response.content})
            tool_results = handle_output(response)
            if not tool_results:
```

With the results sent back and the model called again until it stops requesting actions, the same model, tool and request finish the task:

```
> The tests are failing. Find out why and fix it.
< Let me start by looking at the project and running the tests.
$ pwd; ls -la; git status 2>/dev/null | head
$ cat prices.py test_prices.py; python -m pytest -q 2>&1 | tail -30
$ python -m unittest -v 2>&1 | tail -20
< The docstring says the discount is a percentage, but the code subtracts it
  as a flat amount. So 25 - 10 = 15 instead of 25 x 0.9 = 22.5. I'll fix the
  code to match the documented behavior.
$ sed -i 's|return round(subtotal - discount, 2)|return round(subtotal * (1 - discount / 100), 2)|' prices.py
   && cat prices.py && python -m unittest 2>&1 | tail -5
< The tests pass now. The bug was in prices.py, not in the tests.
```

This is an agent. Asked a question about itself, though, it has to go looking:

```
> Who are you, and where are you working?
< I'll check the environment first so I can answer accurately.
$ echo "user: $(whoami)"; echo "host: $(hostname)"; echo "cwd: $(pwd)"; uname -a; ls -la | head -20
< I'm an AI assistant. I can run shell commands, and I use that to read files,
  run code and make changes in the environment I'm given.
```

It does not know its name, its body, its working directory or the date. What is missing is what the model is shown.

A.5 Plus CONTEXT: the complete harness

```
import subprocess, os, datetime
from anthropic import Anthropic

model = Anthropic()
bash = {"name": "bash", "description": "Run a shell command",
        "input_schema": {"type": "object", "properties": {"command": {"type": "string"}}, "required": ["
     command"]}}


def capture_input():
    return input("\n> ")


def assemble_context(conversation):
    instructions = f"You are quark, an agent. You act through bash, in {os.getcwd()}. Today is {datetime.date.
     today()}."
    return {"system": instructions, "tools": [bash], "messages": conversation}


def request_response(context):
    return model.messages.create(model="claude-sonnet-5-5", max_tokens=16384, **context)


def handle_output(response):
    tool_results = []
    for block in response.content:
        if block.type == "text":
            print("< " + block.text.replace("\n", "\n  "))
        if block.type == "tool_use":
            print(f"$ {block.input['command']}")
            ran = subprocess.run(block.input["command"], shell=True, capture_output=True, text=True)
            tool_results.append({"type": "tool_result", "tool_use_id": block.id, "content": ran.stdout + ran.
     stderr or "(no output)"})
    return tool_results


def control_flow():
    conversation = []
    while True:
        conversation.append({"role": "user", "content": capture_input()})
        while True:
            response = request_response(assemble_context(conversation))
            conversation.append({"role": "assistant", "content": response.content})
            tool_results = handle_output(response)
            if not tool_results:
                break                                   # done: hand back to the person
            conversation.append({"role": "user", "content": tool_results})   # a tool ran: go again


control_flow()
```

One sentence of context, assembled on every call, answers the question without any action:

```
> Who are you, and where are you working?
< I'm quark, an agent that works by running shell commands through bash. My
```

```
working directory is /tmp/shop. Today's date is 2026-10-07.
```

This is the whole harness: fifty lines, five functions, one per primitive. It runs commands on the host without asking and forgets everything when it exits; the production concerns of §4 are what it lacks, and the companion repository adds them one question at a time to produce `quark_production.py`. The same repository later replaces `request_response` with a function that runs a local model and translates between the Messages format and the model's chat template, leaving the other four primitives unchanged; that is the porting claim of §6 in practice.

B Evidence for the Deconstruction

Every assignment in §5, with its evidence. For harnesses with published source, a citation is a path and a line range at the pinned commit; for Claude Code it is a documentation page and section heading on the date read. The same files are published in the repository under `paper/evidence/`. **Path conventions** are given at the start of each harness; **not found** means the mechanism was looked for and not found, not that it is absent.

B.1 Claude Code

Source: https://code.claude.com/docs/en/overview (official docs; source not published) · **Version:** documentation as of 2026-10-15; the docs state no single CLI version (pages reference behaviours by version, the highest seen being v2.1.299; agent-loop page bundles "Claude Code v2.1.290 or later" in SDK v0.3.290). Engineering posts: claude.dev/blog "Prompt caching is everything" (2026-04-30), "A harness for every task" (2026-06-02), "Getting started with loops" (2026-06-30) · **Language:** not documented (docs describe a native binary and an npm package; troubleshooting refers to a "JavaScript heap") **Path conventions:** `D` = https://code.claude.com/docs/en ; `SDK` = `D/agent-sdk` ; `B` = https://claude.dev/blog ; every citation is `page § Section heading` and was read on 2026-10-15. Where a page did not say how something works I write "not documented". **What it is:** Claude Code is Anthropic's agentic coding harness: a terminal/IDE/desktop/web/CI program that wraps Claude models with file, search, shell and web tools, permission gating, memory loading and the loop that chains actions (`D/glossary § Agentic harness`). It reaches the model over the Anthropic Messages API (`/v1/messages`, SSE streaming) or via Amazon Bedrock, Google Cloud's Agent Platform, Microsoft Foundry or an LLM gateway (`D/llm-gateway-protocol § API formats`). The same loop is exposed as a library (Agent SDK, Python/TypeScript) and as `claude -p` for scripts (`SDK/overview § Compare the Agent SDK to other Claude tools`).

Control flow

- **Core loop:** "Claude continues calling tools and processing results until it produces a response with no tool calls"; each cycle (model output with tool calls → tools run → results fed back) is one turn, with no control returned to the caller between turns. `SDK/agent-loop § The loop at a glance`, `§ Turns and messages`
- **Stop condition / result:** the loop ends on a text-only response; the `ResultMessage.subtype` is `success`, `error_max_turns`, `error_max_budget_usd`, `error_during_execution` or `error_max_structured_output_retries`; `stop_reason` carries `end_turn`, `max_tokens` or `refusal`. `SDK/agent-loop § Handle the result`
- **Step budget:** `max_turns`/`--max-turns` "counts tool-use turns only"; no limit by default (print mode only). A message still queued when the limit ends a turn starts a new turn with a fresh count. `SDK/agent-loop § Turns and budget`; `D/cli-reference § CLI flags` (`--max-turns`)
- **Token/cost budget:** `max_budget_usd` is compared "after model responses arrive", so spend can pass the cap by one response; subagent spend counts; at the cap, spawning fails with `Budget limit reached` and background subagents are stopped. `SDK/agent-loop § Budget headroom`
- **Permission gate order (harness consumes the answer):** hooks → deny rules → ask rules → permission mode → allow rules → `canUseTool`/prompt; a hook `allow` never skips deny/ask; `dontAsk` denies at the last step. `SDK/permissions § How permissions are evaluated`; rule order "deny, then ask, then allow … first match" `D/permissions § Manage permissions`

- **Permission modes:** `default` (Manual), `acceptEdits`, `plan`, `auto`, `dontAsk`, `bypassPermissions`; auto is the built-in starting mode for interactive terminal/VS Code on v2.1.283+. `D/permission-modes § Available modes`, `§ Which mode a session starts in`
- **Auto-mode classifier gate:** a second model decides most tool calls; "runs on Claude Sonnet 5 by default rather than on your /model selection"; reads/working-dir edits skip it; it sees user messages, non-read-only tool calls and CLAUDE.md but "Tool results are stripped"; falls back to prompting after 3 blocks in a row or 20 total. `D/permission-modes § How auto mode evaluates actions`, `§ When auto mode falls back`
- **Plan mode:** edits stay blocked until a plan is approved; the approval prompt (`Yes, and use auto mode` / `Yes, manually approve edits` / `No, keep planning`) switches the session's permission mode. `D/permission-modes § Analyze before you edit with plan mode`, `§ Review and approve a plan`. Entering/leaving is itself a tool call (`EnterPlanMode`/`ExitPlanMode`) so the tool set never changes. `B/lessons-from-building-claude-code-prompt-caching-is-everything § Using Plan Mode to design around the cache`
- **Stop hook as loop extender:** a `Stop` hook returning `decision: "block"` keeps Claude working; capped at 8 consecutive continuations (reset on tool use), `CLAUDE_CODE_STOP_HOOK_BLOCK_CAP`. `D/hooks § Stop`; `D/env-vars § Variables` (`CLAUDE_CODE_STOP_HOOK_BLOCK_CAP`)
- **Goal loop:** `/goal` "is a wrapper around a session-scoped prompt-based Stop hook"; after each turn the condition plus conversation go to the small fast model, which returns Not yet met / Met / Impossible; the loop halts if no tool use for several turns. `D/goal § How evaluation works`
- **PostToolBatch gate:** fires once after all tool calls in a batch resolve, "before the next model request"; `decision: "block"` or `continue: false` stops the loop before the next model call. `D/hooks § PostToolBatch`
- **Sub-agent delegation (Agent tool):** the model spawns a subagent in its own context; depth default 3 layers (`CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH`, `1` disables nesting); at most 20 concurrent (`CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS`); only the final message returns to the parent. `D/sub-agents § Let subagents spawn their own subagents`, `§ Concurrent subagent limit`; `SDK/ subagents § What subagents inherit`
- **Foreground vs background sequencing:** order of decision: teammates → `CLAUDE_CODE_DISABLE_ BACKGROUND_TASKS=1` → fork mode on (background, Claude can't request foreground) → fork mode off (background by default, foreground when Claude needs the result); background results arrive later as a completion notification. `D/sub-agents § Run subagents in foreground or background`
- **Model keyed to work:** `opusplan` "Uses opus in plan mode, then switches to sonnet for execution"; subagent model order: per-invocation `model` → frontmatter `model` → `CLAUDE_CODE_SUBAGENT_MODEL` → main model; background functionality (titles, summaries, goal evaluator) uses the Haiku-class model. `D/model-config § Model aliases`, `§ Environment variables`; `D/sub-agents § Choose a model`; `D/ sessions § Name your sessions`
- **Dynamic workflows (model-written program sequencing model calls):** "A dynamic workflow is a JavaScript script that orchestrates many subagents"; `agent()`, `parallel()`, `pipeline()`; up to 16 concurrent agents, 4,096 items per call, 1,000 agents per run; no mid-run user input; `Date.now()`/`Math. random()` throw so a relaunch replays deterministically. `D/workflows § What the saved script looks like`, `§ Behavior and limits`, `§ Edit a saved script`
- **Workflow resume:** completed agents return saved results; the first agent whose prompt differs, and every agent after it, runs again. `D/workflows § Which agents run again`
- **Agent teams (outer harness of peer sessions):** lead + teammates with a shared task list and a per-agent mailbox JSON file at `~/.claude/teams/{team-name}/inboxes/{agent-name}.json`; experimental, `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`; no nested teams. `D/agent-teams § Architecture`, `§ Limitations`
- **Prompt- and agent-based hooks (harness-sequenced model calls):** `type: "prompt"` sends hook input to a model (default: the background-functionality model, 30 s timeout) to allow/block; `type: "agent"` spawns a subagent with tools (60 s). `D/hooks § Prompt-based hooks`, `§ Agent-based hooks`
- **Scheduled turns:** `/loop` creates cron tasks; "The scheduler checks every second for due tasks and enqueues them at low priority"; fires between turns, never mid-response; 7-day expiry; self-paced loops end via `ScheduleWakeup` with `stop: true`. `D/scheduled-tasks § How scheduled tasks run`, `§ Stop a`

loop

- **Non-interactive exit wait:** after the turn ends, a `claude -p` run stays open for background work; "stops whatever is still running after 10 minutes of continuous idle waiting" (`CLAUDE_CODE_PRINT_BG_WAIT_CEILING_MS`). `D/headless § Background tasks at exit`

Input

- **Prompt capture:** interactive terminal prompt, `claude "query"`, `claude -p "query"`, and piped stdin ("Piped stdin is capped at 10MB"). `D/cli-reference § CLI commands`; `D/headless § Pipe data through Claude`
- **Mid-turn message queue:** a message typed while Claude works is queued; "Claude Code passes it to Claude as soon as those tool calls finish, within the same turn"; commands are held until the turn ends. `D/interactive-mode § When Claude Code sends what you queued`
- **Interrupt:** `Esc` cancels the running tool call and Claude waits; queued messages are sent next. `D/how-claude-code-works § Interrupt and steer`; `D/interactive-mode § General controls`
- **Model-consumed answers:** `AskUserQuestion` returns the user's choice to the model; questions stay open until answered unless `askUserQuestionTimeout` (`60s`/`5m`/`10m`) is set. `D/tools-reference § AskUserQuestion tool behavior`
- **Tool results as input:** each tool execution yields a `UserMessage` carrying the tool result back to the model. `SDK/agent-loop § Message types`
- **Event/stream inputs:** `--input-format stream-json` accepts user messages over stdin; `Monitor` feeds "each output line back to Claude" and can treat WebSocket messages as events; channels push external events into the session. `D/cli-reference § CLI flags`; `D/tools-reference § Tools reference` (`Monitor` row); `D/glossary § Channel`
- **Non-human turns captured through the same entry:** `UserPromptSubmit` "also fires for scheduled tasks, background subagent reports, and messages from other sessions". `D/hooks § UserPromptSubmit`
- **Rich input:** `Ctrl+V` pastes an image as an `[Image #N]` chip; `@path` mentions expand file content; `@server:protocol://resource` fetches MCP resources as attachments. `D/interactive-mode § General controls`; `SDK/modifying-system-prompts § Turn off the context your agent replaces` (`CLAUDE_CODE_DISABLE_FILE_MENTIONS`); `D/mcp § Use MCP resources`
- **MCP elicitation:** a server's mid-task form/URL request is shown as a dialog and the answer goes back to the server (not the model); an `Elicitation` hook can answer it. `D/mcp § Respond to MCP elicitation requests`

Context

- **Request layering (what one call contains):** system prompt (core instructions + tool definitions) → project context (CLAUDE.md, auto memory, unscoped rules) → conversation (messages, tool results); ordered most- to least-stable for prefix caching. `D/prompt-caching § How the cache is organized`
- **Two system prompts:** full vs "a shorter one with abbreviated tool descriptions"; newer models get the short one unless `CLAUDE_CODE_SIMPLE_SYSTEM_PROMPT=0`. `D/env-vars § Variables` (`CLAUDE_CODE_SIMPLE_SYSTEM_PROMPT`)
- **System prompt attribution block:** "a short attribution block to the system prompt containing the client version and a fingerprint"; stripped by `api.anthropic.com`; `CLAUDE_CODE_ATTRIBUTION_HEADER=0` omits it. `D/llm-gateway-protocol § System prompt attribution block`
- **CLAUDE.md delivery:** loaded "as a user message after the system prompt"; all discovered files are concatenated root→working directory, `CLAUDE.local.md` after `CLAUDE.md`; subdirectory files load "once Claude reads, writes, or edits another file in that subdirectory". `D/glossary § CLAUDE.md`; `D/memory § How CLAUDE.md files load`
- **Imports and rules:** `@path` imports expand at launch; `.claude/rules/*.md` load every session, or on demand when `paths:` frontmatter matches a file Claude touches; a CLAUDE.md over 4 MiB is skipped. `D/memory § Import additional files`, `§ Path-specific rules`, `§ How it works`
- **Auto memory (model-kept record):** `~/.claude/projects/<project>/memory/MEMORY.md`; "The first 200 lines of MEMORY.md, or the first 25KB, whichever comes first, are loaded"; topic files read on

demand; a write past the limit returns an error telling Claude to rewrite the index. D/memory § How it works, § Storage location

- **System reminders:** harness-added context delivered "wrapped in <system-reminder> tags inside a user message, or ... a separate message with the system role": CLAUDE.md, output style, hook additionalContext, skill list, file-changed notes, attribution lines. D/glossary § System reminder
- **Output style injection:** "Claude Code sends the active style's instructions with every request" as a conversation message, so the cached prefix survives a switch. D/output-styles § How output styles work; D/prompt-caching § Changing output style
- **Skills in context:** descriptions (truncated at 1,536 chars) are always in context; the body loads on invocation as a user message and stays for the session; disable-model-invocation: true keeps the description out. D/skills § Frontmatter fields, § Controlling visibility and context cost; D/prompt-caching § Invoking skills and commands
- **Skill dynamic context:** !command runs a shell command before the model sees the skill and splices the output in. D/skills § Dynamic context injection
- **MCP tool search (deferred definitions):** only names and server instructions load; schemas arrive via the ToolSearch tool; ENABLE_TOOL_SEARCH auto loads upfront while definitions are under 10% of context; description caps 4,096 (upfront) / 16,384 (deferred) chars. D/mcp § Scale with MCP tool search, § Configure tool search
- **Environment and git snapshot:** working directory, platform, shell, OS and a git branch/status/recent-commits block are added at session start; includeGitInstructions: false drops the git snapshot. D/context-window (Environment info event); SDK/modifying-system-prompts § Turn off the context your agent replaces; D/prompt-caching § Cache scope
- **Fitting tool results:** Bash output inline "up to roughly 30,000 characters"; past that a file path plus 2,000-char preview; MCP results capped at 25,000 tokens and text over 50,000 chars spilled to a file; Read returns at most 25,000 tokens per call and refuses whole files over 256 KB. D/tools-reference § Output limits, § Large files; D/mcp § Raise the limit for a specific tool; D/env-vars § Variables (MAX_MCP_OUTPUT_TOKENS)
- **Auto-compaction trigger:** by default "compacts when the conversation reaches the model's context limit"; native-1M models compact "at about 967K tokens"; window settable 100K–1M via /autocompact, autoCompactWindow, --autocompact, CLAUDE_CODE_AUTO_COMPACT_WINDOW. D/model-config § Default auto-compact thresholds, § Set the auto-compact window
- **Compaction order:** "It clears older tool outputs first, then summarizes the conversation if needed." D/how-claude-code-works § When context fills up
- **Compaction as a forked request:** the summarization call reuses the same system prompt, tools and history "plus a summarization instruction appended as a final user message"; inherits the session's thinking config. D/prompt-caching § Compacting the conversation; D/context-window § What survives compaction; B/lessons-from-building-claude-code-prompt-caching-is-everything § COMPACTING WITHOUT BREAKING THE CACHE
- **What survives compaction:** system prompt/output style still apply; project-root CLAUDE.md, auto memory, plan file re-injected from disk; up to five recently modified files re-read (over 5,000 tokens → path reference); invoked skill bodies re-injected "capped at 5,000 tokens per skill and 25,000 tokens total"; hook context is summarized away. D/context-window § What survives compaction
- **Hook-supplied context:** additionalContext is wrapped in a system reminder; additionalContext, systemMessage, initialUserMessage capped at 10,000 chars (longer saved to file with a 2,000-char preview); PostToolUse.updatedToolOutput replaces what Claude sees. D/hooks § JSON output, § Add context for Claude, § PostToolUse
- **Sub-agent context:** a non-fork subagent gets its own system prompt plus environment details, the task message, CLAUDE.md hierarchy (not for Explore/Plan), git snapshot, preloaded skills; not the parent history, output style or auto memory. A fork gets the full conversation, system prompt and tools. D/sub-agents § What loads at startup, § How forks differ from other subagents
- **Sub-agent output scanning:** before the parent reads a report, control-tag imitations get a backslash and a [harness: ...] marker line is prepended; nothing is removed. D/sub-agents § Subagent output scanning; SDK/subagents § What subagents inherit

Model interface

- **Wire format:** Anthropic Messages format, "Inference requests post to `/v1/messages?beta=true`"; optional `/v1/messages/count_tokens` (falls back to a character estimate); headers `anthropic-version: 2023-06-01`, `anthropic-beta`, `x-claude-code-session-id`, `x-claude-code-agent-id`. `D/llm-gateway-protocol § API formats`, `§ Optional endpoints and startup traffic`, `§ Request headers`
- **Providers/backends:** `CLAUDE_CODE_USE_BEDROCK`, `CLAUDE_CODE_USE_VERTEX`, `CLAUDE_CODE_USE_FOUNDRY`, Claude Platform on AWS, or any gateway via `ANTHROPIC_BASE_URL`; Bedrock uses `invoke-with-response-stream` binary eventstream. `D/env-vars § Variables`; `D/llm-gateway-protocol § API formats`
- **Streaming receipt:** "Claude Code reads each streaming inference response event by event as it arrives"; expects the full sequence through `message_delta`/`message_stop`, else treats it as a dropped connection. `D/llm-gateway-protocol § Streaming`
- **Retry policy:** "retries transient failures up to 10 times with exponential backoff" (`CLAUDE_CODE_MAX_RETRIES`, capped at 15); retries server errors/529/timeouts before output starts, dropped connections (up to 2 after thinking), stalled streams (once), no-headers (once per request), temporary 429s; never re-runs after a completed text or tool block "because that could execute the same tool calls twice". `D/errors § Automatic retries`
- **529 backoff:** `CLAUDE_CODE_OVERLOADED_RETRY_BASE_DELAY_MS` 500 → `CLAUDE_CODE_OVERLOADED_RETRY_MAX_DELAY_MS` 32000; `retry-after` honoured, and >60 s stops retries outside watchdog sessions; `x-should-retry` read. `D/errors § Automatic retries`; `D/llm-gateway-protocol § Response headers`
- **Indefinite retry for unattended runs:** `CLAUDE_CODE_RETRY_WATCHDOG=1` "Retries 429 and 529 capacity errors indefinitely", backing off up to 5 minutes. `D/env-vars § Variables`
- **Timeouts and watchdogs:** `API_TIMEOUT_MS` default 600000; first-byte deadline 180 s (direct API) / 300 s (elsewhere) + 1 s per 32 KB of body; event-level watchdog 300 s; byte-level watchdog 180 s/300 s; body idle timeout 5 min. `D/env-vars § Variables` (`API_TIMEOUT_MS`); `D/network-config § Streaming idle watchdogs`
- **Backup model when the request can't be served:** `--fallback-model sonnet,haiku` / `fallbackModel` runs "when the primary model is overloaded, unavailable, or returns another non-retryable server error"; chain of ≤3; applies to the current turn only; subagents use it too. `D/model-config § Fallback model chains`
- **Safety-flag model fallback:** when a server safety classifier flags a request, Claude Code "can re-run it on a fallback model" (e.g. Opus 4.8 for cybersecurity); `switchModelsOnFlag`. `D/model-config § Automatic model fallback`
- **Capability-rejection recovery:** on a 400 rejecting `thinking`, a mid-conversation system message, `cache_control`, the advisor tool or `output_config.effort`, Claude Code retries without that capability for the rest of the conversation; a rejected thinking signature strips earlier thinking blocks. `D/llm-gateway-protocol § Automatic retry and error forwarding`
- **Context-limit rejection:** a request rejected because input + `max_tokens` exceeds the limit is retried "with a reduced max_tokens", then compacted when no reduction fits. `D/errors § Automatic retries`
- **Prompt-cache markers and TTL:** `cache_control` on system blocks and messages; main conversation gets 1 h TTL on subscription usage, 5 min otherwise; subagents/compaction/titles get 5 min; `promptCacheTtl`, `FORCE_PROMPT_CACHING_5M`, `DISABLE_PROMPT_CACHING`. `D/prompt-caching § Which TTL each request gets`, `§ Choose the TTL yourself`, `§ Disable prompt caching`
- **Reasoning parameters on the wire:** `thinking: {"type": "adaptive"}` for 4.6+ models; effort and structured output in `output_config` with paired beta headers; `MAX_THINKING_TOKENS` fixed budget (≥1,024, ≤ max output - 1); `CLAUDE_CODE_MAX_OUTPUT_TOKENS` (default 32000 / cap 128000 for unknown IDs). `D/llm-gateway-protocol § Feature pass-through`; `D/env-vars § Variables`
- **Retry visibility:** a `system/api_retry` event (attempt, max_retries, retry_delay_ms, error_status, error category) is emitted in stream-json before each retry. `D/headless § Handle API retries`

Output

- **Built-in tool set:** Agent, AskUserQuestion, Bash, Edit, EndConversation, Glob, Grep, LSP, Monitor, NotebookEdit, PowerShell, Read, SendMessage, Skill, TaskCreate/TaskUpdate/TodoWrite, ToolSearch, WebFetch, WebSearch, Workflow, Write (+ MCP tools `mcp__<server>__<tool>`). `D/tools-reference § Tools reference` (table); `SDK/agent-loop § Built-in tools`
- **Concurrent execution of one response's calls:** "Read-only tools (like Read, Glob, Grep, and MCP tools marked as read-only) can run concurrently. Tools that modify state (like Edit, Write, and Bash) run sequentially"; custom tools opt in via `readOnlyHint`. `SDK/agent-loop § Parallel tool execution`
- **Bash executor:** default timeout 2 min (`BASH_DEFAULT_TIMEOUT_MS`), ceiling 10 min (`BASH_MAX_TIMEOUT_MS`); a foreground command that hits its timeout "moves it to the background instead of stopping it"; background commands get 30 min (max 2 h); env vars don't persist between commands; shell aliases/options captured in `~/.claude/shell-snapshots/`. `D/tools-reference § Timeout and output limits`, `§ Foreground commands that move to the background`, `§ Time limit for background commands`, `§ What persists between commands`; `D/claude-directory § Cleaned up automatically`
- **Sandbox technology:** macOS Seatbelt; Linux/WSL2 bubblewrap + socat; no sandbox on native Windows; covers Bash, PowerShell and Monitor commands only, not file tools, MCP servers or hooks. `D/sandboxing § OS-level enforcement`, `§ What runs outside the sandbox`
- **Sandbox network path:** "Claude Code runs the sandbox proxy on your machine, outside the sandbox, and directs commands to it with HTTP_PROXY, HTTPS_PROXY, ALL_PROXY"; the proxy checks hostnames against `allowedDomains`/`deniedDomains`, refuses names resolving only to local addresses, and can mask credentials with a sentinel (requires `tlsTerminate`). `D/sandboxing § Network isolation`, `§ Hostnames that resolve to local addresses`, `§ Mask credentials`
- **Sandbox filesystem path:** writable: working dir + per-user `$TMPDIR`; "Protected paths" (settings, hooks, MCP config) are denied even inside writable dirs; `excludedCommands` run unsandboxed; strict mode `allowUnsandboxedCommands: false`. `D/sandboxing § Filesystem isolation`, `§ Protected paths`, `§ Turn off the retry with strict sandbox mode`
- **WebFetch executor:** converts HTML to Markdown, "runs the prompt against the content in a separate model call, and Claude receives the result of that call rather than the raw page"; 15-min response cache; hostname first checked against an Anthropic blocklist at `api.anthropic.com` (`skipWebFetchPreflight`). `D/tools-reference § WebFetch tool behavior`; `D/data-usage § WebFetch domain safety check`
- **WebSearch:** server-side tool; "An interactive terminal session has a limit of 200 WebSearch calls" shared with subagents. `D/tools-reference § WebSearch tool behavior`, `§ Session search limit`
- **Read/Edit/Write executors:** Read returns numbered lines, resizes images (>500 KB re-encoded as JPEG), reads PDFs in ≤20-page ranges via `pdftoppm`; Edit/Write enforce read-before-edit on older models. `D/tools-reference § Read tool behavior`, `§ Edit tool behavior`, `§ Write tool behavior`
- **MCP executor:** stdio, HTTP (falls back to SSE), WebSocket transports; `MCP_TIMEOUT` 30 s startup, `MCP_TOOL_TIMEOUT` ~28 h, 60 s per HTTP request, idle timeout 5 min remote/30 min stdio, main-conversation calls auto-background after 2 min (`CLAUDE_CODE_MCP_AUTO_BACKGROUND_MS`); invalid schemas and names >128 chars are excluded per tool. `D/mcp § Transports`, `§ Timeouts`, `§ Tools with invalid input schemas`, `§ Tools with names longer than 128 characters`
- **Monitor executor:** Claude writes a script run in the background, or opens a WebSocket; deadline 5 min by default, max 30 min (10 min under `-p`). `D/tools-reference § Monitor tool`
- **Rendering the stream:** an `AssistantMessage` is emitted per content block; `stream_event` deltas (`text_delta`, `input_json_delta`) with `--include-partial-messages`; the spinner shows a live output-token count. `SDK/streaming-output § Message flow`; `D/costs § Read the token count beside the spinner`
- **Structured output:** `--output-format json|stream-json`, `--json-schema` validated output in `structured_output`; up to 5 attempts (`MAX_STRUCTURED_OUTPUT_RETRIES`). `D/headless § Get structured output`; `D/env-vars § Variables`
- **Hook executors:** handler types `command`, `http`, `mcp_tool`, `prompt`, `agent`; "All matching hooks run in parallel. Identical handlers from multiple settings files run once"; defaults 600 s command/http/mcp_tool; exit 2 blocks (PreToolUse) with stderr fed to Claude; `async: true` detaches; `PreToolUse.updatedInput` rewrites tool arguments before execution. `D/hooks § Configuration`, `§ Exit code output`, `§ Run`

hooks in the background, § PreToolUse

- **User-facing artifacts:** file checkpoints snapshot files "before each prompt you send that starts a turn", 100 most recent kept, `/rewind` restores code and/or conversation; Bash-made and subagent edits are not tracked. `D/checkpointing § Automatic tracking, § Bash command changes not tracked, § Subagent edits not restored`
- **EndConversation:** never prompts, PreToolUse hooks don't run for it, and deny rules/`--tools` can't remove it while any other tool remains. `D/tools-reference § EndConversation tool behavior`
- **Permission prompt surface:** Manual mode asks for Bash (except a built-in read-only set: `ls`, `cat`, `git status`…), file modification, WebFetch (except preapproved doc domains), WebSearch; "Yes, don't ask again" persists per repository. `D/permissions § Permission system, § Read-only commands`

Production concerns

Concern	Where it lives (primitive)	Mechanism	Evidence
Persistence	Input/Context (transcript, memory); Output (checkpoints); Control flow (resume)	Every message/tool use is appended to `~/.claude/projects/<project>/<session-id>.jsonl`; swept after `cleanupPeriodDays` (30 d default); auto memory excluded from the sweep; `file-history/<session>/` snapshots, `plans/`, `tool-results/`; `--resume`/`--continue` restore history, model, agent, permission mode, goal and cron tasks; `--fork-session`//`branch` copy to a new ID; SDK `session_store` mirrors transcripts; `CLAUDE_CODE_TRANSCRIPT_LOCAL_GC` trims history before the last compaction once the file passes 5 MB	`D/sessions § Where transcripts are stored, § What a resumed session restores; D/claude-directory § Cleaned up automatically; D/memory § Storage location; SDK/agent-loop § Sessions and continuity; D/env-vars § Variables`
Safety & security	Control flow (gates); Output (sandbox, executors); Context (scanning)	deny→ask→allow rules plus modes; auto-mode classifier with tool results stripped and a "server-side probe" over incoming tool results; critical paths (`rm -rf /`, `~`, cwd) never approvable by allow rule or hook; protected paths; OS sandbox + domain allowlist + credential masking; subagent output scanning; teammate messages can't approve prompts; WebFetch summarised by a separate model call; `-p` runs project hooks/MCP without a trust dialog (`--bare` to skip); credentials in Keychain or a 0600 file; prompt-injection list	`D/permission-modes § What the classifier blocks by default, § Critical paths, § Protected paths; D/sandboxing § What the sandbox restricts; D/sub-agents § Subagent output scanning; D/agent-teams § Messages between agents; D/security § Protect against prompt injection, § Additional safeguards; D/headless § Start faster with bare mode`

Concern	Where it lives (primitive)	Mechanism	Evidence
Observability	Model interface (api events); Output (tool events); operator records	OpenTelemetry metrics (`claude_code.cost.usage`, `token.usage`, `session.count`, `lines_of_code.count`…) and events (`user_prompt`, `tool_result`, `api_request`, `api_error`, `tool_decision`) via `CLAUDE_CODE_ENABLE_TELEMETRY=1`; content opt-ins incl. `OTEL_LOG_RAW_API_BODIES=file:<dir>`; `/usage` session cost + `Prompt cache (main)` line; status-line `current_usage`/`prompt_cache`; debug log `~/.claude/debug/<session-id>.txt` with hook durations; stream-json `system/init`, `api_retry`, `compact_boundary`, `permission_denials`; gateway hint headers	`D/monitoring-usage § Metrics, § Events, § Content opt-in; D/costs § Prompt cache statistics; D/hooks § Debug hooks; D/headless § Read session metadata; D/llm-gateway-protocol § Gateway hint headers`
Resilience	Model interface (retries/watchdogs/fallback); Context (compaction); Control flow (resume/stall)	10 retries w/ exponential backoff, no re-run after a completed block (partial output kept + "may be incomplete" notice, `-p` auto-continues ≤3×); four streaming watchdogs; fallback model chain; context-limit retry with reduced `max_tokens` then compaction; auto-compact "stops auto-compacting after a few attempts" on thrashing; interrupted tool call marked cut-off on resume (`CLAUDE_CODE_RESUME_INTERRUPTED_TURN`); SIGTERM runs `SessionEnd` hooks and exits 143; workflow agents restart on stall up to 5×; `/goal` auto-retries 3× then pauses; usage-limit auto-wait (`autoContinueAtUsageLimit`); MCP auto-reconnect	`D/errors § Automatic retries, § The response above may be incomplete; D/network-config § Streaming idle watchdogs; D/model-config § Fallback model chains; D/how-claude-code-works § When context fills up; D/troubleshooting § Auto-compaction stops with a thrashing error; D/sessions § What a resumed session restores; D/headless § Stop a run with SIGTERM; D/workflows § When an agent stalls and restarts; D/goal § Other errors retry or pause the goal`

Concern	Where it lives (primitive)	Mechanism	Evidence
Performance	Context (layering, deferral, trimming); Model interface (cache TTL); Output (parallel tools); Control flow (background)	Prefix-ordered requests and cache-safe updates via messages (system reminders); MCP schemas deferred; compaction as a forked cached request; `/rewind` truncates to a cached prefix; idle compaction before 1 h TTL expiry; read-only tools run concurrently; background subagents and auto-backgrounded commands; workflow fan-out holds siblings $\leq$5 s so they share the first agent's cache; WebFetch 15-min cache; `--bare` skips discovery; cache hit rate is alerted on internally	`D/prompt-caching § How the cache is organized`, `§ Compacting the conversation`, `§ Rewinding the conversation`, `§ Subagents and the cache`; `D/workflows § Prompt caching in a fan-out`; `SDK/agent-loop § Parallel tool execution`; `D/headless § Start faster with bare mode`; `B/lessons-from-building-claude-code-prompt-caching-is-everything § LESSONS LEARNED`
Evaluation	not documented for the harness itself	No harness test suite or agent evals are documented. Adjacent: `/insights` analyses up to 200 local sessions into a report; skills are evaluated by running prompts with the skill on/off (skill-creator plugin); the auto-mode classifier's first request "validates the Sonnet 5 default"; "Evals" listed as a workflow use case	`D/costs § Analyze your usage patterns`; `D/skills § Evaluating skills`; `D/permission-modes § How auto mode evaluates actions`; `B/a-harness-for-every-task-dynamic-workflows-in-claude-code § Use cases`

Design decisions (one line each: the choice this harness made within a primitive)

- Input: one entry point for everything that starts a turn (typed prompts, piped stdin, queued mid-turn messages delivered after the current tool batch, scheduled/cron prompts, cross-session and channel messages, background-subagent reports), all passing through `UserPromptSubmit`. `D/hooks § UserPromptSubmit`; `D/interactive-mode § When Claude Code sends what you queued`
- Context: a stable cached prefix (system prompt + tools, then project context) with every later update appended as a message or `<system-reminder>`; MCP schemas deferred behind `ToolSearch`; compaction on reaching the window via a forked request that re-injects CLAUDE.md, memory, five recent files and invoked skills. `D/prompt-caching § How the cache is organized`; `D/context-window § What survives compaction`
- Model interface: direct Anthropic Messages API with streaming, 10 exponential-backoff retries that never re-issue after a completed tool block, four idle watchdogs, a user-configured fallback model chain, and capability-rejection downgrade for gateways. `D/errors § Automatic retries`; `D/llm-gateway-protocol § Automatic retry and error forwarding`
- Output: a fixed built-in tool set plus MCP; read-only calls run concurrently, mutating calls sequentially; shell commands run under an OS sandbox (Seatbelt/bubblewrap) with a local domain-allowlist proxy; large results spill to files; file edits are checkpointed for `/rewind`. `SDK/agent-loop § Parallel tool execution`; `D/sandboxing § OS-level enforcement`; `D/checkpointing § Automatic tracking`
- Control flow: loop until a text-only response, with the decision to stop extendable by Stop hooks and `/goal`'s evaluator model; a six-step permission pipeline whose default gate is a second model (auto mode); delegation through an `Agent` tool (depth 3, 20 concurrent, background by default) and, at scale, a model-written JavaScript workflow the runtime executes. `SDK/agent-loop § Turns and messages`;

SDK/permissions § How permissions are evaluated; D/workflows § When to use a workflow

Hard cases

- **WebFetch's inner model call:** a tool (output) that itself calls the model and decides what the main model sees (a summary, not the page). Resolved by rule 21: the tool call is output; the summarisation is a context decision (and a prompt-injection defence, D/security § Additional safeguards); the extra request is model interface. D/tools-reference § WebFetch tool behavior
- **Auto-mode classifier:** an approval gate the harness consumes (control flow) implemented as a separate model request with its own stripped-down context (D/permission-modes § How auto mode evaluates actions). Resolved: the decision is control flow; the request is model interface; the stripping of tool results is context assembly for that side call. Its "server-side classifier review" variant runs inside the API as part of the session's model requests, i.e. partly outside the harness. D/permission-modes § Server-side classifier review
- **Prompt/agent hooks and the /goal evaluator:** harness-sequenced model calls that decide whether to go again. Resolved by rule 27 (a program that sequences model calls is control flow). D/hooks § Prompt-based hooks; D/goal § How evaluation works
- **Dynamic workflows:** the model writes a JS program at run time whose agent() calls sequence model calls → control flow (rule 27); the Workflow tool call that launches it is output; the script's intermediate results live "in script variables" rather than any context. D/workflows § How a workflow runs
- **Compaction:** the trigger (window threshold) and the result (summary replaces history) are context; the summarisation is a model call (model interface); the thrashing stop after "a few attempts" is a loop decision (control flow/resilience). D/model-config § Default auto-compact thresholds; D/troubleshooting § Auto-compaction stops with a thrashing error
- **Context-limit retry with reduced max_tokens:** a retry of a failed request (model interface) that changes the request's parameters (context). Placed under model interface because the trigger is a rejected request and the content is unchanged; noted as a boundary case. D/errors § Automatic retries
- **Hook rewrites:** PreToolUse.updatedInput changes the action before execution (output executor side); PostToolUse.updatedToolOutput changes what the model is shown (context). Decomposed per rule 1. D/hooks § PreToolUse, § PostToolUse
- **Two kinds of model switch:** opusplan (plan vs execution) and subagent model fields are keyed to the work → control flow; --fallback-model (overloaded/unavailable) and safety-flag fallback → model interface (rule 22). D/model-config § Model aliases, § Fallback model chains, § Automatic model fallback
- **Subagent output scanning:** modifies a tool result before the model reads it → context (what the model is shown) and a safety concern; not output, because nothing is executed. D/sub-agents § Subagent output scanning
- **Checkpoints and session titles:** file snapshots are a user artifact (output) that persists; the title is produced by a cheap-model call keyed to the work (control flow) and stored for the operator/user (persistence). D/checkpointing § Automatic tracking; D/sessions § Name your sessions
- **Queued messages mid-turn:** capturing the message is input; deciding to hand it to the model "as soon as those tool calls finish, within the same turn" is a loop decision (control flow). D/interactive-mode § When Claude Code sends what you queued
- **Agent teams' mailbox:** messages arrive as turns for the recipient (input) but the delivery, idle notifications and task-claim locking are an outer harness's control flow; the classifier reviewing each SendMessage is a safety gate. Depth: lead → teammates (no nested teams); results return via idle notifications containing the final answer. D/agent-teams § Context and communication, § Limitations
- **Server-side classifier review and the server-side prompt-injection probe:** DOES NOT FIT the five primitives cleanly, because the safety decision is made inside the API during the model request rather than by the harness; the harness only consumes the verdict (denies when none arrives after ten responses). D/permission-modes § Server-side classifier review; D/glossary § Prompt injection
- **EndConversation:** a tool whose only effect is to stop the loop; by rule 21 the call is output and the stop is control flow; its un-removability is a safety choice. D/tools-reference § EndConversation tool behavior

Summary row

Input	Context	Model interface	Output	Control flow
Prompts, stdin, queued messages, events, tool results, AskUserQuestion answers	Cached prefix + appended reminders; CLAUDE.md/memory; deferred MCP; compaction	Messages API streaming; 10 retries, watchdogs, fallback chain	Built-in + MCP tools; sandboxed Bash; concurrent read-only calls; checkpoints	Loop until no tool calls; six-step permission gate; Agent/workflow delegation

B.2 OpenCode

Source: https://github.com/anomalyco/opencode · **Version:** commit 654b79b27e79da735d6b07d766c8d7431a71d1b7 (2026-10-15); `@opencode/core` package version 2.0.29 · **Language:** TypeScript (Bun runtime, Effect 4 library) **Path conventions:** `core/` = `packages/core/src/`, `ai/` = `packages/ai/src/`, `schema/` = `packages/schema/src/`, `codemode/` = `packages/codemode/src/`, `server/` = `packages/server/src/`, `recorder/` = `packages/http-recorder/src/`. Line ranges are `Lstart--Lend` at the pinned commit. Note: the brief's expected layout (`packages/opencode/src/session/prompt.ts`, `processor.ts`, `llm.ts`, `system.ts`) no longer exists at this commit; the equivalents are `core/session/prompt.ts` (input preparation), `core/session/runner/llm.ts` (the loop), `core/session/runner/step.ts` (one model call plus its tools), `core/session/model-request.ts` (request assembly), `core/session/compaction.ts`, `core/session/system-prompt.ts`. **What it is:** OpenCode is an open-source coding agent harness by Anomaly (anomalyco) that runs as a local server with TUI, desktop, and web clients; a session is a durable, event-sourced record in SQLite that a per-session "runner" fiber drains step by step. The model is reached through the in-house `@opencode/ai` client, which compiles a canonical `LLMRequest` into a provider protocol (Anthropic Messages, OpenAI Responses, Bedrock Converse, Gemini, ...) over HTTP/SSE or a stateful WebSocket, with a Vercel AI SDK bridge for third-party provider packages.

Control flow

- **Entry: `Session.prompt` admits input then rings the doorbell:** reconciles the inbox row, prepares the prompt, admits it, and calls `execution.wake(sessionID)` unless `resume === false`. `core/session/session.ts:L145--L178`
- **Run coordinator (one execution fiber per session):** `loop` drains, then re-drains while a `pendingWake` doorbell was rung mid-drain; `wake` coalesces scopes ("input" subsumes "steer"); `interrupt` marks `stopping`, claims recorded wakes and interrupts the owner fiber. `core/session/run-coordinator.ts:L74--L85`, `L136--L145`, `L147--L167`
- **Execution service routes drains and handles `DrainResult`:** `Complete` ends, `Moved` and `Reloaded` re-enter `drain` with a continuation step. `core/session/execution.ts:L87--L110`; `DrainResult` enum `core/session/runner/index.ts:L23--L28`
- **The outer loop (`SessionRunner.drain`):** `while (true) { next = advanceToStep(); if not Ready return; continuing = runStep(ctx, step); step++ }`. `core/session/runner/llm.ts:L191--L198`
- **Step boundary (`advanceToStep`):** at each boundary the runner checks `lifecycle.isClosed()`, pulls the next inbox item under a per-session lock, handles `compaction` and `move` controls inline, and otherwise promotes pending steers/queued input before loading context; a promoted input resets `step = 1`. `core/session/runner/llm.ts:L74--L189`
- **Steer vs queue delivery:** `Promotable` doc: "steer" promotes steers only (mid-work), "input" also admits one queued item at an idle boundary. `core/session/inbox.ts:L43--L48`; `nextPromotable` `L407--L424`; `promote` `L496--L531`; a pending compaction is pulled ahead of earlier steers `L533--L550`; `Delivery = "steer" | "queue"` `schema/session-inbox.ts:L11`
- **Inner step policy (`runStep`):** before every attempt it runs auto-compaction (`compacted.status === "completed"` → new assistant id and `continue`), then prepares the request and dispatches on the attempt `Outcome`. `core/session/runner/llm.ts:L209--L330`

- **Stop condition:** `needsContinuation = toolChoice !== "none" && record.needsContinuation` (a step with executed tool calls continues; a text-only finish hands back); `finish === "length"` → `OutputLimit`. `core/session/runner/step.ts:L281--L283`
- **Step budget:** `stepLimitReached = agent.info.steps !== undefined && step >= agent.info.steps`; on the final step the runner appends `MAX_STEPS_PROMPT` as an assistant message and sets `toolChoice: "none"` while keeping tool definitions to preserve the cached prefix. `core/session/runner/llm.ts:L226--L247`; prompt text `core/session/runner/max-steps.ts:L1--L16`; `steps` field `schema/agent.ts:L33`; tool calls on that step fail with "Tools are disabled after the maximum agent steps" `core/session/runner/step.ts:L91--L101`
- **Output-limit continuation:** after `finish === "length"` the runner reads the last 3 user/assistant rows; if all three are assistant `length` finishes it fails with `output-limit`, else publishes the synthetic `CONTINUE_AFTER_OUTPUT_LIMIT` and goes again. `core/session/runner/llm.ts:L290--L319`; constant `L38--L39`
- **Interrupted-stream continuation:** `Continue` outcome waits the retry delay, publishes `CONTINUE_AFTER_INCOMPLETE_STREAM` and starts a fresh assistant message. `core/session/runner/llm.ts:L281--L289`; constant `L35--L36`
- **Retry wait is part of the loop:** `Retry` outcome → `retry.wait` (publishes `RetryScheduled`, sleeps) then the `while (true)` re-attempts with refreshed context. `core/session/runner/llm.ts:L275--L280`, `L216--L219`; `wait` `core/session/runner/retry.ts:L97--L113`
- **Overflow → compact → go again:** `Compacted` outcome sets `recoverOverflow = false` (one overflow compaction per step) and re-attempts. `core/session/runner/llm.ts:L265--L271`, `L320--L323`
- **Approval gate consumed by the harness:** `Permission.assert` evaluates rules (`deny` → `BlockedError`, `allow` → continue, `ask` → create request and await a `Deferred`). `core/permission.ts:L231--L263`; the reply endpoint resolves it, `"always"` persists a rule, a `reject` with no message fails every pending ask in the session. `core/permission.ts:L265--L325`
- **A decline ends the step:** declined tool calls are failed with "The user declined this tool call" and the attempt is interrupted (`Effect.interrupt`) rather than returned to the model. `core/session/runner/step.ts:L207--L215`, `L274--L277`
- **Interrupt with resume:** after interrupting, if the next promotable item is a steer or a control, the coordinator is woken with scope "steer" so queued next-turn prompts stay parked. `core/session/execution.ts:L153--L170`
- **Sub-agent delegation (outer-harness side):** the `subagent` tool walks `parentID` to compute depth and refuses beyond `experimental.subagent_depth` (default 1); it creates a child session with `parentID`, prompts it, and in foreground blocks on the child's job; background returns immediately. `core/tool/plugin/subagent.ts:L117--L133`, `L185--L198`, `L205--L218`, `L229--L240`
- **Child session is a harness of its own:** the job's `run` is `sessions.resume(childSessionID)` and the result is the text of the last completed assistant message. `core/session/subagent-job.ts:L45--L63`
- **How results return:** foreground → tool result text wrapped in `<subagent sessionID=... state="completed">`; background → a synthetic user message delivered to the parent when the job settles. `core/tool/plugin/subagent.ts:L252--L266`; `core/session/subagent-completion.ts:L21--L46`
- **Routing to a cheap model keyed to the work (title):** the title agent uses `model.small(primary.providerID)` plus a minimal-reasoning variant (`none|minimal|low`), forked so it never delays the step. `core/session/context.ts:L95--L119`, `L186`; fork `core/session/runner/llm.ts:L177--L180`
- **Compaction model keyed to the work:** the `compaction` agent may carry its own model; otherwise the session model is used. `core/session/compaction.ts:L213--L219`
- **Agent profiles (bundles):** `build` (default, all-allow + question), `general`/`explore` sub-agents with deny lists and a custom system prompt, hidden `compaction`/`title`/`summary` agents. `core/plugin/agent.ts:L80--L140`; default ruleset `schema/agent.ts:L46--L51`
- **Plan mode (bundle):** a `plan` agent denies `edit` outside `~/.opencode/plan/*` (control flow), rewrites blocked-edit errors (output), and re-injects a `<system-reminder>` per request (context). `core/plugin/plan.ts:L33--L42`, `L44--L53`, `L57--L60`
- **Crash/restart resume:** at boot, sessions whose execution claim survived are resumed with a synthetic "The server restarted..." notice; a durable per-turn counter stops after `DEFAULT_MAX_ATTEMPTS = 10`.

core/session/execution/restart.ts:L77--L99, L193--L231; budget L33

Input

- **HTTP handler → session.prompt:** text, files, agents, skills, metadata, delivery, resume are passed straight through. server/handlers/session.ts:L307--L322
- **Prompt preparation (SessionPrompt.prepare):** fires the plugin hook session.prompt (plugins may rewrite text/files/delivery), materializes attachments with concurrency 8, resolves skills to their text, and builds a UserPayload. core/session/prompt.ts:L29--L91
- **Attachments:** data: URLs or file: URIs; 20 MiB cap; ?start=&end= line ranges for text; a directory becomes a sorted listing. core/session/prompt.ts:L93--L133, L151--L201, L203
- **Image normalization on input:** images are resized via Image.Service (resizer unavailable → passthrough). core/session/prompt.ts:L135--L149
- **Durable admission:** the inbox row is inserted into SQLite (admit, reconcile, cancel, steer, queue). core/session/inbox.ts:L155, L169, L230--L256, L295
- **Harness-originated input (synthetic):** Session.synthetic admits text the harness produced (restart notice, job completions) through the same inbox. core/session/session.ts:L273--L314
- **Answer the MODEL consumes (question tool):** the form answers are rendered as User has answered your questions: ... and returned as the tool result. core/tool/plugin/question.ts: L38--L46, L62--L112
- **Background shell completion event:** notifyWhenDone waits on the job and admits a synthetic message with the output. core/tool/plugin/shell.ts:L157--L189
- **Sub-agent completion event:** delivered as a synthetic message tagged <subagent ... state=...>. core/session/subagent-completion.ts:L37--L44

Context

- **Selection (SessionContext.select):** fixes session, agent, tool snapshot (narrowed by merged agent+session permissions) and instruction sources, loaded in parallel. core/session/context.ts: L121--L161; permission narrowing L131
- **Instruction ordering for cache reuse:** "Ordered from most to least shared across sessions so the baseline stays a reusable prompt-cache prefix": Code Mode catalog, MCP, references, skills, AGENTS.md discovery, builtins (date/env), per-session entries. core/session/context.ts:L147--L158
- **Load (SessionContext.load):** resolves the model and reads history after the latest replayable compaction. core/session/context.ts:L163--L179
- **System prompt assembly:** [agent.system ?? default prompt rendered with tool guidance, initial instructions]; messages via toLLMMessages. core/session/model-request.ts:L100--L120
- **Default system prompt text:** 15 lines ("You are an AI agent running in OpenCode..."), with ${OPENCODE_TOOL_GUIDANCE} filled per available tool. core/session/runner/prompt/system.txt:L1--L15; core/session/system-prompt.ts:L9--L28
- **Built-in instructions:** today's date and an <env> block (working dir, workspace root, git, platform, tmp). core/instructions/builtins.ts:L24--L53
- **AGENTS.md discovery:** global AGENTS.md plus ancestor-walk from the location to the project root, watched for changes. core/config/plugin/instruction.ts:L36--L62, L76--L86
- **Catalogs rendered into the baseline:** skills <available_skills> core/skill/instructions.ts: L16--L33; MCP <mcp_instructions> core/mcp/instructions.ts:L17--L29; Code Mode catalog core/ codemode/instructions.ts:L9--L30
- **Instruction epochs:** changed instruction sources are frozen into a durable InstructionsUpdated event with rendered delta text, so later requests see "Today's date is now..." style updates as system messages. core/session/instruction-state.ts:L38--L61, L80--L87; rendered as Message.system core/session/runner/to-llm-message.ts:L291--L292
- **History read bounded by compaction:** rows from the latest completed compaction seq onward; native checkpoints only when the target model can replay them. core/session/history.ts:L21--L26, L78--L108, L118--L135

- **Projection of history into model messages:** synthetic/skill/shell messages become `user` role; a compaction becomes a `<conversation-checkpoint>` user message. `core/session/runner/to-llm-message.ts:L269--L327`, `L330--L334`
- **Tool definitions snapshot:** tools wholly denied by permission are dropped; `codemode: false` tools are advertised directly, the rest only inside the single `execute` tool; definitions sorted by name. `core/tool.ts:L225--L287`; JSON Schema generation `core/tool/runtime.ts:L21--L27`, `L174--L188`
- **Plugin hook that rewrites what the model sees:** `session.context` sees system, messages, tool descriptions/schemas and the default `maxTokens`; renamed tools are mapped back by object identity. `core/session/model-request.ts:L236--L274`
- **Media fitting:** unsupported modalities are replaced by an ERROR text; inline images are dropped from oldest once the total exceeds 25 MiB until under 15 MiB. `core/session/model-request.ts:L43--L46`, `L132--L147`, `L189--L216`; applied `L297`
- **Output token budget fitted to the window:** `outputLimit` = min(catalog limit, 256k), reduced to the room left after `measured + estimated`×`1.15`, floor 1,024. `core/session/model-request.ts:L47--L57`, `L88--L98`
- **Auto-compaction trigger:** `due` when `estimateContext >= ceiling`; ceiling = window - max(10 %, 16,000) or window - configured `buffer`; skipped right after a completed compaction or before a measured response. `core/session/compaction.ts:L205--L240`, `L242--L251`, `L911--L920`; defaults `auto: true`, `keep: 15_000` `L197`; settings `L41--L47`
- **Prompt size estimate = provider-measured + locally estimated:** anchors on the last assistant message with a usage count from the same provider, estimates everything after at ~4 chars/token. `core/session/compaction.ts:L880--L905`, `L924--L930`, `L950--L953`
- **Summary compaction:** splits history into `older` (summarized) and `recent` (kept verbatim within `keep` tokens, starting at a user boundary); sends older + summary prompt; one template nudge before failing. `core/session/compaction.ts:L272--L316`, `L708--L744`; template `L104--L135`
- **Tool outputs in the verbatim part are cut to 1,250 chars.** `core/session/compaction.ts:L100`, `L813--L826`
- **Overflow compaction (provider said too long):** captured before any output starts, then compaction aims at 70 %/50 %/35 % of the rejected request size. `core/session/runner/step.ts:L113--L120`, `L155--L167`; `SHRINK_STEPS` `core/session/compaction.ts:L95`, `L429--L448`
- **Manual `/compact`:** an inbox control item processed at the step boundary. `core/session/session.ts:L246--L264`; `core/session/runner/llm.ts:L115--L163`
- **Native (provider-side) compaction:** for models with `compaction.type === "native"` the provider returns one opaque item that replaces the window; the harness only appends the newest user messages. `core/session/compaction.ts:L318--L332`
- **Tool result truncation before it enters history:** 2,000 lines / 50 KiB, overflow written to a file with a `[showing lines 1-N of M; full output saved to ...]` marker. `core/tool-output.ts:L13--L16`, `L67--L118`; applied in the step `core/session/runner/step.ts:L127`
- **Tool input repair (what the model's call is seen as):** stringified JSON, numeric strings, placeholder nulls are repaired against the schema before execution. `core/plugin/tool-input-repair.ts:L7--L36`
- **Prompt-cache key / affinity:** `promptCacheKey` and `x-session-affinity` derive from the root session (children and forks share the parent's). `core/session/model-request.ts:L278--L295`; `core/session/affinity.ts:L6--L8`
- **Searchable memory for the model:** not found (no model-facing memory store; `core/environment/memory.ts` is an in-memory filesystem).

Model interface

- **Streaming request:** the step calls `llm.stream(request, options)` and folds every `LLMEvent` through a publisher; it reads to the end "so the next request can reuse this response". `core/session/runner/step.ts:L103--L138`; client `ai/route/client.ts:L602--L607`, `L618--L622`
- **Request construction:** `LLM.request({model, http.headers, promptCacheKey, system, messages, tools, toolChoice, generation, providerOptions})` with `x-opencode-session-id`/`User-Agent` headers. `core/session/model-request.ts:L279--L302`

- **Wire format:** per-provider protocol modules (`anthropic-messages.ts`, `openai-responses.ts`, `bedrock-converse.ts`, `gemini.ts`, ...) and framings (SSE) compile the canonical request and decode the stream. `ai/protocols/` (directory), `ai/route/client.ts:L423--L497`
- **Route/header hooks:** `session.model.request` may change `baseURL`/headers; `session.http.request`/`http.response` wrap the raw HTTP exchange. `core/session/model-request.ts:L305--L320`, `L337--L365`
- **HTTP timeouts:** headers 300 s, inter-chunk 900 s, whole-request none by default. `ai/route/transport/http.ts:L100--L146`; defaults `ai/schema/options.ts:L56`, `L62`
- **Stateful WebSocket transport:** bound per session when `model.transport === "websocket"`; rotates after 55 min, 15 s connect, 30 min idle; after 5 consecutive stream failures the session stays on HTTP. `core/session/model-request.ts:L368--L392`; `core/session/model-transport.ts:L22--L27`, `L186--L196`, `L308`
- **Error classification:** `isContextOverflowFailure` and `isRetryable` (rate limit, provider internal, transport unless delivery rejected, incomplete stream, unknown; `x-should-retry` header overrides). `ai/provider-error.ts:L72--L75`, `L81--L110`
- **Retry policy:** exponential from 2 s capped at 10 s per gap, 10 retries, jittered; provider `retry-after` honored up to 15 min; at most 3 timeout retries; `session.retry` plugin hook may override the decision. `core/session/runner/retry.ts:L31--L62`, `L64--L92`
- **Retry vs continue vs recover-full:** before output started → `Retry`; after output started on an interrupted stream → `Continue`; WebSocket `retry-full`/`rotate-and-retry-full` → `RecoverFull`. `core/session/runner/step.ts:L182--L204`, `L263--L271`, `L291--L295`
- **Unknown finish reason is an error:** turned into `InvalidProviderOutputError("incomplete-stream")`. `core/session/runner/step.ts:L171--L181`
- **Model resolution:** session model must be in the available catalog, else `ModelUnavailableError`. `core/session/runner/model.ts:L75--L91`
- **Token usage and cost:** normalized `input/output/reasoning/cache.read/cache.write`, cost by context tier. `core/session/usage.ts:L11--L19`, `L22--L36`
- **Vercel AI SDK bridge:** `AISDK.Service.model(RuntimeInfo)` wraps a `LanguageModelV3` as an `@opencode/ai` `LanguageModel` for third-party provider packages. `core/aisdk.ts:L237--L252`
- **Backup model for a request that could not be served (title only):** if the small model's attempt fails, the primary model is tried. `core/session/title.ts:L129--L133`

Output

- **Streaming render path:** provider events are published as durable `Text.Started/Delta/Ended`, `Reasoning.*`, `Tool.Called/Progress/Success/Failed`, `Step.Started/Streamed/Ended/Failed`; deltas batched every 100 ms. `core/session/runner/publish-llm-event.ts:L443--L470`, `L106`, `L194--L200`
- **Concurrent tool execution from one response:** each `tool-call` event forks a fiber immediately; all are awaited after the stream ends. `core/session/runner/step.ts:L121--L134`, `L151`
- **Tool executor:** `execute.before` hook → input decode/validate → `tool.execute` → `execute.after` hook → content normalization (image resize). `core/tool.ts:L103--L156`, `L263--L284`; validation `core/tool/runtime.ts:L29--L61`, `L88--L100`
- **Shell tool:** 2-minute default timeout, `AGENT=1 OPENCODE=1` env, jobs with foreground block or background; no OS sandbox found, gating is by permission after tree-sitter parsing of the command into per-command resources and touched directories. `core/tool/plugin/shell.ts:L24`, `L117--L155`, `L199--L279`; parser `core/shell/parse.ts:L7`, `L392`
- **Edit tool asks with a diff preview:** `permission.assert({action: "edit", metadata: {files: [preview]}})` before applying. `core/tool/plugin/edit.ts:L271--L283`
- **Read/external-directory gating:** `authorizeRead` asks `external_directory` for paths outside the location, then `read` per file. `core/file-access.ts:L125--L142`, `L144--L161`
- **MCP tools as output:** each discovered MCP tool is registered under a server namespace, permission-asserted, then `callTool`. `core/tool/mcp.ts:L41--L77`
- **Code Mode `execute` tool:** the model submits JavaScript; tool calls inside it are routed through the

same `executeTool` (hooks + permissions); `fetch` only, no imports/fs/timers. `core/codemode/tool.ts:L62--L69`, `L71--L108`; sandbox = custom tree-walking interpreter over acorn AST with value limits and optional timeout. `codemode/interpreter/execute.ts:L1`, `L56--L68`; `codemode/interpreter/limits.ts:L6--L13`

- **File-change artifact for the user:** a git-backed snapshot is captured before and after each step and the changed paths are attached to `Step.Ended`. `core/session/runner/step.ts:L74--L76`, `L234--L260`; `core/snapshot.ts:L43--L68`, `L90--L105`
- **Question tool shows a form:** `forms.ask` with one field per question, "Type your own answer" custom option. `core/tool/plugin/question.ts:L62--L112`, `L119--L132`
- **Tool failure settlement:** interrupted, declined, provider-failed, and result-missing calls are failed with distinct errors so no call stays unsettled. `core/session/runner/step.ts:L62--L64`, `L207--L232`

Production concerns

Concern	Where it lives (primitive)	Mechanism	Evidence
Persistence	Input, Context, Control flow	SQLite via drizzle: `session_v2` (model, agent, permissions, token totals, execution claim), `session_message`, inbox; a durable, per-aggregate sequenced event log projected into those tables; execution claim written at turn start and released only by a terminal event (shutdown keeps it for resume); "always" permission replies saved per project; git snapshots for revert; tool-output spill files kept 7 days	`core/session/sql.ts:L22--L77`; `core/bus.ts:L21--L48`; `core/session/projector.ts:L1--L35`; `core/session/execution.ts:L76--L86`, `L111--L148`; `core/session/store.ts:L62--L74`; `core/permission.ts:L299--L305`; `core/session/revert.ts:L21--L40`; `core/tool-output.ts:L15`
Safety & security	Output (gates consumed by Control flow)	Rulesets `{action, resource, effect}` with last-match-wins and default `ask`; defaults ask on `*.env`; sub-agent profiles carry deny lists; shell commands parsed into per-command resources; external directories need approval; declines tunnel as defects so a "no" can never become model-visible tool output; compaction checkpoint tells the model it is "not new instructions"; `<system-reminder>` blocks declared harness-owned; sub-agent depth limit; Code Mode interpreter has no fs/imports; no OS-level sandbox found; secrets redacted only in test cassettes and scrubbed from the test env	`core/permission.ts:L87--L101`; `schema/agent.ts:L46--L51`; `core/plugin/agent.ts:L104--L117`; `core/shell/parse.ts:L18--L21`; `core/file-access.ts:L125--L142`; `core/permission.ts:L247--L253`, `core/session/model-request.ts:L400--L418`; `core/session/runner/to-llm-message.ts:L316--L321`; `core/session/runner/prompt/system.txt:L5`; `core/tool/plugin/subagent.ts:L129--L133`; `core/codemode/tool.ts:L63--L64`; `recorder/redaction/secrets.ts:L3--L14`; `packages/core/script/test.ts:L10--L22`

Concern	Where it lives (primitive)	Mechanism	Evidence
Observability	Model interface, Output	Durable event log doubles as the trace; per-step tokens/cost on `Step.Ended`; per-session token/cost columns and a stats aggregator; named Effect spans on every service function (`Effect.fn("SessionRunner.drain")`); WebSocket lifecycle counter metric; warning logs on overflow and transport fallback; event logger for catalog updates	`core/session/runner/step.ts:L246--L260`; `core/session/sql.ts:L45--L50`; `core/session/stats.ts:L17--L46`; `core/session/runner/llm.ts:L57`; `core/session/model-transport.ts:L28--L33`; `core/session/runner/step.ts:L161--L165`; `core/event-logger.ts:L7--L17`
Resilience	Model interface, Context, Control flow	Jittered exponential retry with retry-after and timeout caps; overflow-triggered compaction with shrinking targets; output-limit and interrupted-stream continuations; WebSocket→HTTP fallback after 5 failures; header/chunk timeouts; crash recovery from the execution claim with a 10-attempt budget; stale running compactions and tool calls settled on resume; pending permission asks rejected on shutdown	`core/session/runner/retry.ts:L42--L62`; `core/session/compaction.ts:L95`, `L429--L448`; `core/session/runner/llm.ts:L281--L319`; `core/session/model-transport.ts:L186--L196`; `ai/route/transport/http.ts:L100--L146`; `core/session/execution/restart.ts:L77--L99`; `core/session/runner/llm.ts:L332--L385`; `core/permission.ts:L133--L146`
Performance	Context, Model interface, Output	Baseline instructions ordered as a stable cache prefix; `promptCacheKey` shared across a session tree; tool definitions kept on the budget-exhausted step to keep the cached prefix; parallel context source loading; tool calls run concurrently as they stream; 100 ms delta batching; start snapshot captured while the request is in flight; Code Mode catalog and tool JSON-schema caches; tool output truncation and image byte budget	`core/session/context.ts:L147--L158`, `L142`; `core/session/model-request.ts:L295`; `core/session/runner/llm.ts:L243--L244`; `core/session/runner/step.ts:L121--L134`, `L74--L76`; `core/session/runner/publish-llm-event.ts:L106`; `core/tool.ts:L246--L254`; `core/tool/runtime.ts:L10--L19`; `core/tool-output.ts:L13--L14`; `core/session/model-request.ts:L43--L44`

Concern	Where it lives (primitive)	Mechanism	Evidence
Evaluation	— (all primitives)	164 test files under `packages/core/test` (e.g. `session-runner-recorded.test.ts` replays provider traffic from `@opencode/http-recorder` cassettes; `session-compaction.test.ts`, `permission.test.ts`, `session-run-coordinator.test.ts`); 52 under `packages/ai/test`; a simulated provider package; test runner scrubs provider keys from the environment. Model-graded or task-success agent evals: not found	`packages/core/test/session-runner-recorded.test.ts:L1--L30`; `packages/simulation/src/backend/simulated-provider.ts`; `packages/core/script/test.ts:L10--L22`

Design decisions (one line each: the choice this harness made within a primitive)

- Input: every input, including harness-generated notices, is a durable inbox row with a `steer` (interrupt the current step boundary) or `queue` (next turn) delivery, promoted only at step boundaries under a per-session lock.
- Context: history is event-sourced and re-projected into model messages on every call; compaction is a summary message in that history (or an opaque provider checkpoint), triggered at ~90 % of the window using provider-measured counts plus a local estimate, with the newest 15k tokens kept verbatim.
- Model interface: an in-house canonical `LLMRequest` compiled per provider protocol, streamed over HTTP/SSE or a stateful WebSocket, with retry decisions split by whether output had already started.
- Output: tool calls are executed concurrently as they stream, every tool is permission-asserted inside its own code, most tools are reachable only through a single `execute` tool that runs model-written JavaScript in a custom interpreter.
- Control flow: one fiber per session drains a durable inbox; a step continues while tool calls were made, stops on a text-only finish, and is bounded by an optional per-agent step count; sub-agents are full child sessions (depth 1 by default) whose results come back as tool text or a synthetic message.

Hard cases

- **Code Mode `execute` tool:** the model writes a program that sequences tool calls (`tools.ns["name"](input)`), which looked like control flow; rule "a program that sequences only tools is output" resolves it as output, and the interpreter cannot call the model (`core/codemode/tool.ts:L62--L69`). Its per-request catalog in the system prompt is context (`core/codemode/instructions.ts:L19--L30`).
- **`Permission.assert` lives inside each tool:** by file location it is output; by function it is an approval gate the harness consumes (the model never sees a plain decline, the step is interrupted), so it is control flow; the `"always"` rule save is persistence.
- **Question tool:** the form is output, the answers are input (the model consumes them), and a dismissal is treated like a permission decline (control flow) via the same defect tunnel (`core/tool/plugin/question.ts:L90--L97`).
- **Subagent tool:** the call is output; depth check, child creation, blocking and backgrounding are control flow; the child is a separate harness (`sessions.resume`); foreground results return as tool text, background results as a synthetic user message, which makes the same information input in one mode and output in the other.
- **Compaction service calls the model:** `SessionCompaction` decides when and what to summarize (context) but also issues `llm.stream` itself (`core/session/compaction.ts:L512`) and runs the retry policy (`L450--L459`). The brief says context "never calls the model"; resolved by assigning the trigger/split/budget to context, the summary request itself to model interface, and the `Compacted` → `re-attempt` loop to control flow. Partly DOES NOT FIT: native compaction (`L318--L332`) delegates context assembly to the

provider, so the harness's context primitive holds only an opaque item it cannot inspect.

- **`MAX_STEPS_PROMPT` appended as an assistant message:** the step budget is control flow, but it is enforced by editing what the model sees (an assistant-role message plus `toolChoice: "none"`), so the mechanism straddles control flow and context (`core/session/runner/llm.ts:L240--L244`).
- **Instruction epochs:** changed AGENTS.md/date/env are frozen into durable `InstructionsUpdated` events rendered as system messages in the transcript; this is context, but it is implemented as persistence of history, and history is immutable, so stale instruction text stays in old events.
- **Plugin hooks (`session.context`, `session.model.request`, `http.request`, `tool.execute.before/after`, `permission.evaluate`, `session.retry`):** a single hook bus whose entries belong to five different primitives; decomposed by what each hook can change rather than where it is registered.
- **Durable event bus:** the same event log is persistence (projected into tables), observability (the trace), and the output channel (clients subscribe); the bus itself is out of scope as transport, its contents are assigned by what they record.
- **Title model fallback:** the "try the small model, then the primary" fallback (`core/session/title.ts:L129--L133`) is a backup model when a request could not be served (model interface), while choosing the small model in the first place is control flow.
- **WebSocket `RecoverFull`:** the transport reports it cannot replay an incremental exchange (model interface), but the decision to resend the full request is taken by the runner's outcome loop (control flow) (`core/session/runner/step.ts:L182--L188`, `core/session/runner/llm.ts:L324--L326`).
- **Synthetic messages:** "The server restarted…", "Your last response hit the output token limit…" are produced by control flow, admitted as input, and shown to the model as user-role messages; assigned as input with control-flow origin.

Summary row

Input	Context	Model interface	Output	Control flow
Durable inbox rows: user, synthetic, shell, skill; steer or queue	Event-sourced history re-projected; cached-prefix instructions; summary or native compaction	In-house LLMRequest compiled per protocol; SSE or WebSocket; split retry policy	Concurrent streamed tool fibers; permission-asserted tools; Code Mode interpreter	One fiber per session drains inbox; continue while tools called; child sessions

B.3 pi (pi-mono coding agent)

Source: https://github.com/badlogic/pi-mono · **Version:** commit 5ab7fe74c400097b386662205cd93900695b8a71 (2026-10-15) · **Language:** TypeScript (Node/Bun) **Path conventions:** `agent/` = `packages/agent/src/`; `ai/` = `packages/ai/src/`; `ca/` = `packages/coding-agent/src/`; `ex/` = `packages/coding-agent/examples/extensions/`; `ev/` = `packages/evals/`. All paths relative to the repo root. Package scope at this commit is `@earendil-works/*` (pi-agent-core, pi-ai, pi-coding-agent). **What it is:** pi is a terminal coding agent by Mario Zechner (badlogic) with a layered design: `pi-agent-core` holds a minimal agent loop, `pi-ai` holds provider adapters (Anthropic, OpenAI, Google, Bedrock, Mistral, …) behind one `StreamFunction`, and `pi-coding-agent` wraps both in an `AgentSession` with JSONL session trees, compaction, retries, built-in tools, and an extension system; interactive TUI, print, JSON, and RPC modes sit on top. The model is reached over HTTPS/SSE (or WebSocket for some providers) through provider SDKs or raw fetch, with `maxRetries: 0` so pi owns retry policy.

Control flow

- **Inner/outer loop (`runLoop`):** inner `while (hasMoreToolCalls || pendingMessages.length > 0)` issues one model call per iteration; outer `while (true)` re-enters only if follow-up messages arrived or `finishTurn` asked for one more context-only turn, else breaks and emits `agent_end`. `agent/agent-loop.ts:L163--L321`

- **Stop on error/abort:** an assistant `stopReason` of `"error"` or `"aborted"` is a hard exit: `finishTurn` is called, `turn_end`/`agent_end` emitted, loop returns. `agent/agent-loop.ts:L245--L256`
- **Go-again decision:** `hasMoreToolCalls = !executedToolBatch.terminate`; if the response had no tool calls and no steering/follow-up is queued, the loop ends. `agent/agent-loop.ts:L258--L278`, `L295--L318`
- **`finishTurn` decision hook:** returns `{action:"end"}` (stop now) or `{action:"continue"}` (force one more request); undefined keeps normal scheduling. `agent/agent-loop.ts:L286--L298`, `agent/types.ts:L146--L158`
- **Early termination from tools:** a batch ends the run only when every finalized tool result sets `terminate: true` (`shouldTerminateToolBatch`); a blocked `beforeToolCall` may set it too. `agent/agent-loop.ts:L690--L692`, `L745--L755`, `agent/types.ts:L66--L74`
- **Steering vs. follow-up queues:** `Agent.steer()` injects after the current turn's tools finish; `followUp()` waits until the agent would otherwise stop; drain modes `"one-at-a-time"` (default) or `"all"`. `agent/agent.ts:L143--L174`, `L247--L248`, `L299--L306`, `L496--L503`; polled in the loop at `agent/agent-loop.ts:L176`, `L204--L206`, `L295`, `L302--L308`
- **One run at a time:** `prompt()` throws if a run is active; `continue()` resumes from a user/toolResult tail, else drains steering/follow-up queues. `agent/agent.ts:L373--L381`, `L384--L411`, `L507--L530`
- **Abort:** `Agent.abort()` fires the run's `AbortController`; `AgentSession.abort()` also cancels retry backoff, compaction, branch summary, then waits for idle. `agent/agent.ts:L341--L343`; `ca/core/agent-session.ts:L2464--L2474`
- **Session post-run loop:** `_startRun` calls `agent.prompt(messages)` then loops `_handlePostAgentRun()` → `agent.continue()` while retry/overflow-recovery/queued messages/extension `agent_before_settle` ask for more. `ca/core/agent-session.ts:L1837--L1882`, `L1943--L1981`, `L1983--L2005`
- **Auto-retry budget (session level):** `_prepareRetry` increments `_retryAttempt`, stops past `maxRetries`, sleeps `retryDelayMs()` (abortable), omits the failed assistant message from projection, and returns true so the loop re-issues the request. Defaults `enabled: true, maxRetries: 3, baseDelayMs: 2000`. `ca/core/agent-session.ts:L3792--L3832`; `ca/core/settings-defaults.ts:L18--L26`
- **Overflow recovery is one-shot:** `_checkCompaction` case 1 compacts and retries the turn once (`_overflowRecoveryAttempted` guard); case 2 (completed response) compacts without retry; case 3 threshold compacts without retry. `ca/core/agent-session.ts:L2977--L3115`
- **Virtual-model routing keyed to the work:** `prepareRequest` resolves a virtual model to a physical one per request with `reason: "user" | "continuation" | "retry"`, storing router state as a session entry; a router (extension-registered) picks the model. `ca/core/agent-session.ts:L815--L843`; `ca/core/model-runtime.ts:L1003--L1034`; `ca/core/virtual-models.ts:L1--L11`, `L43--L50`
- **Extension boundary hooks that can force another turn:** `turn_end` and `agent_before_settle` handlers return `continue: true` and may commit boundary drafts; the session validates `canContinue` first. `ca/core/agent-session.ts:L847--L897`, `L1983--L2005`
- **`tool_call` approval gate (extension):** `beforeToolCall` → `runner.emitToolCall`; first handler returning `{block:true}` wins; the `permission-gate` example asks `ctx.ui.select` and blocks when no UI. `ca/core/agent-session.ts:L659--L682`; `ca/core/extensions/runner.ts:L1249--L1267`; `ex/permission-gate.ts:L13--L33`
- **Project trust gate:** `resolveProjectTrusted` checks a persisted trust store, an extension `project_trust` handler, a `defaultProjectTrust` setting ("ask" default), then prompts; untrusted cwd means project `.pi/` extensions/skills/prompts/SYSTEM.md are not loaded. `ca/core/project-trust.ts:L47--L97`; `ca/core/trust-manager.ts:L30--L39`, `L45--L59`, `L67--L97`
- **Sub-agents (example extension, not core):** the `subagent` tool spawns a separate `pi --mode json -p --no-session` process per agent, in `single`, `parallel`, or `chain` modes, and returns the child's final output as the tool result (depth 1; results return as tool-result text). `ex/subagent/index.ts:L298`, `L342--L348`, `L470--L475`; `ex/subagent/README.md`
- **Nested tool calls from tools:** `ctx.executeTool()` runs through `runToolCall` with the session's hooks; ids are `<parent>/<n>` and never enter the transcript (sequences only tools → output by rule, but concurrency is serialized by a queue). `ca/core/nested-tool-calls.ts:L161--L200`; `agent/agent-loop.ts:L811--L819`; `ca/core/extensions/types.ts:L384--L395`

Input

- **TUI submit:** the editor's `onSubmit` trims text, handles slash commands locally, then `session.prompt(text)`; while streaming it calls `prompt(text, {streamingBehavior:"steer"})`; Alt+Enter queues `followUp`. `ca/modes/interactive/interactive-mode.ts:L3200--L3202, L3364--L3385, L4508`
- **Print/JSON mode:** initial message plus `messages[]` are sent via `session.prompt()` sequentially. `ca/modes/print-mode.ts:L131--L137`
- **RPC mode:** JSONL commands `prompt`, `steer`, `follow_up`, `abort`, `clear_queue` map onto session methods with `source: "rpc"`. `ca/modes/rpc/rpc-mode.ts:L394--L433`
- **`@file` arguments:** CLI `@path` args become text content and image attachments before the first prompt. `ca/cli/file-processor.ts:L24--L25`
- **`AgentSession.prompt()` capture path:** extension-command check → `input` extension handlers (transform/handled) → `/skill:` and prompt-template expansion → queue if streaming, else auth check and `_startRun`. `ca/core/agent-session.ts:L2058--L2141, L2007--L2025; ca/core/extensions/runner.ts:L1525--L1564; ca/core/extensions/types.ts:L1171--L1187`
- **Queue capture:** `_queueSteer`/`_queueFollowUp` build a `user` message (text + images) and hand it to `agent.steer`/`followUp`. `ca/core/agent-session.ts:L2241--L2293`
- **Prompt normalization in core:** a string prompt becomes `{role:"user", content:[text, ...images]}`. `agent/agent.ts:L413--L430`
- **Tool results as input:** each executed call becomes a `toolResult` message (`content`, `details`, `isError`, `durationMs`) pushed into context. `agent/agent-loop.ts:L930--L944, L274--L277`
- **Streamed tool output:** `onUpdate` partial results flow out as `tool_execution_update` events (bash emits snapshots of its `OutputAccumulator`). `agent/agent-loop.ts:L779--L788, L821--L857; ca/core/tools/bash.ts:L276--L294`
- **User `!bash` results:** `executeBash` runs a user command outside the model loop and records a `bashExecution` message that `convertToLlm` renders as a user message (unless `!!` excluded). `ca/core/agent-session.ts:L3871--L3903; ca/core/messages.ts:L152--L161`
- **Extension-injected messages:** `before_agent_start` handlers may return a `custom` message appended after the user prompt; `pi.sendUserMessage` starts a run. `ca/core/agent-session.ts:L1926--L1936; ca/core/extensions/runner.ts:L1427--L1477`
- **Image inputs:** prompt images are resized per the model's `inputLimits` before being attached. `ca/core/agent-session.ts:L2027--L2040`

Context

- **System prompt sections:** `buildSystemPromptSections` produces ordered sections `preamble`, `tools`, `rules`, `docs`, `addendum`, `project_context`, `skills`, `cwd`, plus custom sections, each wrapped in a same-named XML tag so later patches can replace one section. `ca/core/system-prompt.ts:L128--L193`
- **Default preamble/rules text:** "You are an expert coding assistant operating inside pi…"; rules merge per-tool guidelines, extension guidelines, and two fixed rules. `ca/core/system-prompt.ts:L155--L169, L88--L125`
- **Prompt diffing into the transcript:** `diffSystemPromptSections` yields a `SystemMessage.sections` patch only when changed; `_prepareAndToolLoadout` appends it before the next request. `ca/core/system-prompt.ts:L217--L229; ca/core/agent-session.ts:L1743--L1755, L899--L933`
- **Project instructions:** `AGENTS.override.md`/`AGENTS.md`/`CLAUDE.md` from `~/.pi/agent` and every ancestor of cwd (worktree shadowing handled), rendered as `<project_instructions path=...>`. `ca/core/resource-loader.ts:L184--L203, L232--L274; ca/core/system-prompt.ts:L79--L86`
- **SYSTEM.md / APPEND_SYSTEM.md overrides:** project `.pi/SYSTEM.md` replaces the preamble; `APPEND_SYSTEM.md` becomes `addendum`. `ca/core/resource-loader.ts:L1214--L1233; ca/core/agent-session.ts:L1714--L1730`
- **Skills in prompt:** only name/description/location of each `SKILL.md` is listed under `<available_skills>`; the model loads the file with `read`. `ca/core/skills.ts:L358--L388`
- **Tool declarations as transcript deltas:** `declareToolChanges` computes `toolsAdded`/`toolsRemoved` on a system message before each request so replay yields exactly `context.tools`. `agent/agent-loop.ts:`

L333--L363

- **Tool loadout/exposure:** tools have `exposure direct | model-only | codemode | deferred | hidden`; `tool_search` (BM25) activates deferred tools for the next request; hidden declarations are stripped from every request. `ca/core/extensions/types.ts:L516`; `ca/extensions/tool-search/index.ts:L14`, `ca/extensions/tool-search/tool.ts:L192--L206`; `ca/core/agent-session.ts:L1772--L1790`
- **Canonical context = session projection:** `prepareRequest` replaces `context.messages` with `sessionManager.buildSessionProjection().messages`, which walks the tree path, applies the newest compaction (`firstKeptEntryId`) and `context_edit` replacements. `ca/core/agent-session.ts:L788--L811`; `ca/core/session-manager.ts:L476--L512`, `L543--L573`
- **`convertToLlm`:** maps `bashExecution`, `custom`, `branchSummary`, `compactionSummary` roles to `user` messages with prefixes and passes system/user/assistant/toolResult through. `ca/core/messages.ts:L148--L196`
- **Extension `context` / `context_with_system` hooks:** run inside `transformContext` before every call; `context` sees non-system messages (clone), `context_with_system` owns the full list. `ca/core/sdk.ts:L436--L440`; `ca/core/extensions/runner.ts:L1305--L1366`; `agent/agent-loop.ts:L388--L397`
- **Forced prompt projection:** a `before_agent_start` `systemPrompt` override collapses system messages into one head for the request only; the transcript keeps structured sections. `ca/core/agent-session.ts:L1792--L1807`
- **Compaction trigger:** `shouldCompact` fires when `contextTokens > contextWindow - reserveTokens` (defaults `reserveTokens` 16384, `keepRecentTokens` 20000); checked before the next assistant response and after the run. `ca/core/compaction/compaction.ts:L120--L130`, `L267--L270`; `ca/core/agent-session.ts:L768--L786`, `L3079--L3113`
- **Cut point:** walks back from newest accumulating estimated tokens until `keepRecentTokens`, then picks the nearest valid cut (turn boundary; can split a turn and summarize its prefix). `ca/core/compaction/compaction.ts:L446--L480`, `L872--L905`
- **Summary generation:** a separate model call with a fixed structured prompt (`## Goal`, `## Constraints & Preferences`, `## Progress` ...), update prompt when a previous summary exists, `cacheRetention: "none"`, wrapped in `retryAssistantCall`. `ca/core/compaction/compaction.ts:L507--L520`, `L619--L639`, `L682--L693`
- **Compaction persisted as a tree entry:** `appendCompaction` stores `summary`, `firstKeptEntryId`, `tokensBefore`, usage, and the current system message. `ca/core/session-manager.ts:L1261--L1287`
- **Token estimation:** provider `usage.totalTokens` when present, else `chars / 3.5` and 4800 chars per image. `ai/utils/estimate.ts:L15--L19`, `L38--L40`
- **Tool-output trimming:** read/bash truncate to 2000 lines or 50 KB (head for read, tail for bash) with full output spilled to a temp file. `ca/core/tools/truncate.ts:L11--L12`; `ca/core/tools/bash.ts:L255`; `ca/core/tools/read.ts:L96`; `ca/core/bash-executor.ts:L56--L104`, `L130--L145`
- **Image policy in context:** `blockImages` setting strips images in `convertToLlm`; tool-result images are resized per model limits after `tool_result` hooks. `ca/core/sdk.ts:L296--L310`; `ca/core/agent-session.ts:L705--L711`
- **Branch summaries:** navigating the tree back can append a `branch_summary` entry rendered as a user message. `ca/core/session-manager.ts:L1600--L1615`; `ca/core/messages.ts:L170--L175`

Model interface

- **`StreamFunction` contract:** `(model, normalizedTranscript, options) => AssistantMessageEventStream`; errors must terminate the stream with `stopReason "error"|"aborted"`. `ai/types.ts:L379--L394`
- **Loop → stream boundary:** `streamAssistantResponse` resolves the API key via `getApiKey` (expiring tokens), calls `streamFunction`, and folds `start`/`*_delta`/`done`/`error` events into a partial `AssistantMessage` in context. `agent/agent-loop.ts:L399--L407`, `L414--L458`
- **Session stream function:** `modelRuntime.streamSimple` → `lazyStream` (async setup behind a sync stream) → `prepareRequest` (auth/headers) → provider `streamSimple`. `ca/core/sdk.ts:L420--L431`; `ca/core/model-runtime.ts:L723--L749`; `ai/api/lazy.ts:L46--L62`; `ai/models.ts:L907--L914`
- **Anthropic adapter:** builds a `@anthropic-ai/sdk` client (API key or OAuth bearer with Claude

Code identity headers), calls `client.beta.messages.create(params, {maxRetries: 0, signal, timeout})`, decodes SSE by hand. `ai/api/anthropic-messages.ts:L603--L645`, `L1018--L1060`, `L475--L500`, `L534`

- **Provider-level retry:** `retryProviderRequest` mirrors SDK policy (408/409/429/5xx or `x-should-retry`), honors `retry-after(-ms)` up to `maxRetryDelayMs` (default 60 s, else fail fast so the outer policy sees it), otherwise `min(0.5·2^n, 8)s` with 25 % jitter, abortable. `ai/utils/provider-retry.ts:L25--L37`, `L53--L69`, `L107--L128`; `ai/api/anthropic-messages.ts:L658--L666`
- **Assistant-level retry classifier:** `isRetryableAssistantError` matches overloaded/429/5xx/network text but excludes quota/billing patterns; `retryAssistantCall` does bounded exponential backoff `baseDelayMs·2^(n-1)` capped at 60 s. `ai/utils/retry.ts:L7--L28`, `L30--L60`, `L121--L137`, `L196--L246`, `L257--L262`
- **Overflow detection:** regex table over provider error texts (`prompt is too long`, `request_too_large`, ...) with non-overflow exclusions (rate limit), used to divert to compaction instead of retry. `ai/utils/overflow.ts:L37--L63`, `L76--L80`; `ca/core/agent-session.ts:L3739--L3743`
- **Strict-tools fallback within the adapter:** a 400 "compiled grammar too large"/"Too many strict tools" rebuilds params without preferred strict schemas and retries once. `ai/api/anthropic-messages.ts:L667--L691`
- **Stream termination:** abort → `stopReason "aborted"`, exceptions → `"error"` with `errorMessage`; `done` carries the final message with usage and cost. `ai/api/anthropic-messages.ts:L897--L933`; `ai/utils/event-stream.ts:L26--L58`
- **Prompt caching:** `cache_control` on the last user/system block and last tool declaration; retention `short`/`long`/`none`; `sessionId` forwarded for cache-aware backends. `ai/api/anthropic-messages.ts:L1529--L1555`, `L1659`; `ai/types.ts:L239--L249`
- **Thinking mapping:** pi `ThinkingLevel` maps to Anthropic effort or `budget_tokens` (adaptive vs. budget thinking), interleaved-thinking beta header. `ai/api/anthropic-messages.ts:L944--L1012`, `L1296--L1305`, `L195`
- **Wire-level hooks:** `onPayload` (extension `before_provider_request` may replace the body), `onResponse` (`after_provider_response`), `onProviderStreamEvent`. `ai/api/anthropic-messages.ts:L653--L656`, `L693`, `L700`; `ca/core/sdk.ts:L382--L409`; `ai/types.ts:L174--L182`, `L218--L223`
- **Transport/options:** `transport: "sse"|"websocket"|...`, `timeoutMs`, `maxRetries`, `maxRetryDelayMs`, custom `fetch`, custom `headers`. `ai/types.ts:L144`, `L157--L210`, `L235--L238`
- **Credential handling:** OAuth tokens auto-refresh under a file lock with a timeout; `auth.json` written with mode `0o600` in a `0o700` dir. `ai/auth/resolve.ts:L119--L142`; `ca/core/auth-storage.ts:L37`, `L71`
- **Direct-route virtual model outside the loop:** `streamSimple` on a virtual model routes with `reason: "direct"` and withholds the caller's credentials when the provider differs. `ca/core/model-runtime.ts:L725--L743`

Output

- **Tool execution pipeline:** `prepareToolCall` (lookup, `prepareArguments`, schema validation, `beforeToolCall`) → `executePreparedToolCall` (`tool.execute(id, args, signal, onUpdate)` in try/catch, duration measured) → `finalizeExecutedToolCall` (`afterToolCall` may replace content/details/isError). `agent/agent-loop.ts:L708--L777`, `L821--L857`, `L859--L910`
- **Parallel vs. sequential:** default `"parallel"`: preflight sequentially, run allowed tools concurrently with `Promise.all`, emit results in source order; `"sequential"` if configured or any called tool has `executionMode: "sequential"`. `agent/agent-loop.ts:L508--L523`, `L586--L660`; `agent/types.ts:L308--L317`
- **Truncated-output guard:** if the response `stopReason` is `"length"`, every tool call is failed with an error result instead of executed. `agent/agent-loop.ts:L263--L270`, `L478--L503`
- **Built-in tool set:** `read, bash, powershell, edit, write, grep, find, ls` (default four: read, bash, edit, write). `ca/core/tools/index.ts:L95--L105`; `ca/core/system-prompt.ts:L64`
- **bash executor:** `spawn(shell, args, {cwd, detached, env: process.env + bin dir})`, stdout/stderr streamed, optional per-call timeout, abort kills the whole process group with `SIGKILL` (`taskkill`

/T on Windows). ca/core/tools/bash.ts:L97--L166; ca/utils/shell.ts:L138--L150, L187--L215

- **edit tool:** exact unique oldText→newText replacements against the original file, BOM/LF normalization, fuzzy-match fallback with diff details; file mutations serialized per real path. ca/core/tools/edit.ts:L152, L190--L194; ca/core/tools/edit-diff.ts:L28--L37; ca/core/tools/file-mutation-queue.ts:L32--L61
- **read tool:** text with offset/limit and head truncation; images returned as image blocks (resized) or omitted for non-vision models. ca/core/tools/read.ts:L96, L159--L177, L80--L83
- **grep/find:** spawn ripgrep (rg, downloaded if missing). ca/core/tools/grep.ts:L119--L121, L168
- **MCP tools:** the built-in mcp extension registers each connected server's tools via pi.registerTool (hidden exposure when only reachable from codemode) and forwards calls with client.callTool. ca/extensions/mcp/index.ts:L387--L460; ca/extensions/mcp/tools.ts:L337--L339
- **codemode tool:** the model writes JavaScript that calls other tools; it runs in a QuickJS WASM sandbox (256 MiB heap, 2 workers max, 10k-token output cap, optional timeout_ms), with no Node/fs/network; nested calls use ctx.executeTool. ca/extensions/codemode/tool.ts:L1--L10, L118--L120; ca/extensions/codemode/execute.ts:L50--L59, L254, L510--L522
- **Rendering streamed fragments:** the TUI updates a streaming assistant component on each message_update and creates tool components as tool calls appear; JSON mode writes each event as one JSON line; print mode prints final assistant text. ca/modes/interactive/interactive-mode.ts:L3524--L3536; ca/modes/print-mode.ts:L108--L112, L139--L158
- **Extension UI as output:** ctx.ui.select/confirm/input/editor render dialogs (also used by approval gates). ca/core/extensions/types.ts:L149--L157, L240

Production concerns

Concern	Where it lives (primitive)	Mechanism	Evidence
Persistence	Context (session transcript)	Append-only JSONL under ~/.pi/agent/sessions/--<cwd>--/<ts>_<id>.jsonl, v3 entries with id/parentId tree; file created on first user/assistant message; branches via branch(); model/thinking changes and usage persisted as entries; context_edit entries persist recovery omissions	ca/core/session-manager.ts:L41, L589--L594, L1057--L1083, L1166--L1214, L1244, L1360, L1579--L1584; ca/core/agent-session.ts:L1147--L1168, L1241--L1257
Safety & security	Control flow (gates), Output (sandbox), Model interface (secrets)	Project trust gate before loading repo-controlled extensions/skills; tool_call block hook (example permission gate); OS sandbox only via example extension (@anthropic-ai/sandbox-runtime: sandbox-exec/bubblewrap); codemode scripts in QuickJS with no fs/network; auth file 0600; bash inherits full process.env (no secret scrubbing found); nothing found for prompt-injection defenses beyond trust gating	ca/core/project-trust.ts:L47--L97; ca/core/trust-manager.ts:L30--L39; ex/permission-gate.ts:L13--L33; ex/sandbox/index.ts:L1--L10; ca/extensions/codemode/tool.ts:L118; ca/core/auth-storage.ts:L37; ca/utils/shell.ts:L138--L150

Concern	Where it lives (primitive)	Mechanism	Evidence
Observability	(observability)	Lifecycle events (`agent_start/turn_*/message_*/tool_execution_*`) to listeners/extensions; `usage` (input/output/cache/cost) on every assistant message and `usage` session entries; `getContextUsage`/`getSessionStats`; crash log; redacted bug-report bundle; `TelemetryContext` span interface passed through options but no `startSpan` call found in `ai/`	`agent/agent.ts:L565--L612`; `ai/types.ts:L451--L470`, `L583`; `ca/core/session-manager.ts:L1244`; `ca/core/agent-session.ts:L4267--L4290`; `ca/core/crash-log.ts:L124--L144`; `ca/core/bug-report.ts:L27`, `L51`; `packages/telemetry/src/index.ts:L1--L22`; `ai/types.ts:L159--L160`
Resilience	Model interface (retries), Context (compaction), Control flow (recovery loop)	Two retry tiers (provider HTTP retry with retry-after cap; assistant-level exponential backoff, 3 attempts); overflow → compact-and-retry once; `length` stop fails tool calls; abortable sleeps; resume from JSONL on restart; compaction itself retried	`ai/utils/provider-retry.ts:L107--L128`; `ai/utils/retry.ts:L196--L246`; `ca/core/agent-session.ts:L3040--L3077`, `L3792--L3832`; `agent/agent-loop.ts:L263--L270`; `ca/core/compaction/compaction.ts:L619--L639`
Performance	Model interface (cache), Output (parallel tools), Context (trimming)	Prompt `cache_control` + `sessionId`; `CacheWarmer` re-sends a 1-token request at 90 % of cache TTL when expected savings $\geq$ \$0.05; parallel tool execution; streaming; 50 KB/2000-line tool output caps; lazy provider module loading	`ai/api/anthropic-messages.ts:L1529--L1555`; `ca/core/cache-warmer.ts:L15--L32`, `L284--L296`; `agent/agent-loop.ts:L586--L660`; `ca/core/tools/truncate.ts:L11--L12`; `ai/api/lazy.ts:L74--L80`
Evaluation	(evaluation)	651 `*.test.ts` files (vitest) across packages (coding-agent 207, ai 180, agent 5), incl. agent-session retry/compaction/branching suites; `packages/evals` runs the real agent via `vitest-evals` harness with Docker variants and LLM-graded docs evals (`autoevals`)	`packages/coding-agent/test/` (listing); `ev/package.json:L1--L23`; `ev/evals/smoke.eval.ts:L5--L18`; `ev/src/harness.ts:L1--L50`

Design decisions (one line each: the choice this harness made within a primitive)

- Input: three capture surfaces (TUI editor, print/JSON CLI, JSONL RPC) all funnel into `AgentSession.prompt()`, with mid-run input split into "steer" (after this turn) vs. "follow-up" (after the agent would stop), one message at a time by default.
- Context: the transcript is a persisted tree; every request is a projection of the active branch (compaction entry + kept tail + context edits), the system prompt is a set of named XML sections diffed into system-message patches, and tool declarations are transcript deltas, so replaying the file reproduces the exact request.
- Model interface: one `StreamFunction` contract across ~15 provider adapters with SDK retries disabled (`maxRetries: 0`) and pi's own two-tier retry (HTTP-level with retry-after cap, assistant-level with error-text classification), overflow routed to compaction rather than retry.

- Output: tools are plain `{name, parameters (TypeBox), execute}` objects; parallel execution by default with per-file mutation serialization; no sandbox in core (OS sandboxing and permission prompts are extensions); a QuickJS-sandboxed `codemode` tool lets the model script tool calls.
- Control flow: a two-level loop in `pi-agent-core` (tool calls/steering inner, follow-ups outer) plus a session-level outer loop for retries/overflow recovery/extension continuations; sub-agents are not in core but an example spawns child `pi` processes; per-request model routing via virtual models.

Hard cases

- **`before_provider_request` payload hook:** lives in the adapter (`onPayload`) at the wire boundary, but an extension can rewrite the request body, i.e. decide what the model is shown. By the function rule it is Context when it changes content; the transport callback itself stays Model interface. `ai/api/anthropic-messages.ts:L653--L656`; `ca/core/sdk.ts:L382--L386`
- **Virtual-model routing:** `prepareRequest` chooses the physical model per request by reason (user/-continuation/retry) — keyed to the work, so Control flow — even though it is implemented inside the model-runtime layer; the `reason: "retry"` branch after a failed request overlaps Model interface's "backup model" (resolved: the router decides based on work state, so Control flow). `ca/core/agent-session.ts:L815--L843`; `ca/core/model-runtime.ts:L1003--L1034`
- **Strict-tools 400 fallback:** an in-adapter request rebuild on a specific 400 (drops strict schemas). It alters the request contents, but the trigger is "request could not be served" — assigned Model interface by the unavailability rule. `ai/api/anthropic-messages.ts:L667--L691`
- **Compaction summary call:** a second model call issued by the Context primitive (summary generation). Rule: Context never calls the model; here it does, through the same `StreamFunction`. Resolved as Context mechanism using the Model interface as a service; noted as a borderline.
- **Nested tool calls / codemode:** a script the model writes sequences only tools (Output by rule), yet it also reaches `models.*` (classifiers, image models) from inside the sandbox — a model call issued from inside a tool. `ca/extensions/codemode/tool.ts:L1--L10`; `ca/extensions/codemode/execute.ts:L50`. Partially DOES NOT FIT: a tool that calls models is neither pure Output nor Control flow in the given scheme.
- **`agent_before_settle` / `turn_end` boundaries:** extensions may commit "boundary drafts" (entries appended to the session: Context) and return `continue` (Control flow) in one hook; decomposed per rule 28. `ca/core/agent-session.ts:L847--L897`, `L1983--L2005`
- **`pi-durable` package:** a second, experimental harness in the same repo (durable task graph, child tasks, commit-before-show) sharing `pi-ai`; not deconstructed here beyond noting it exists. `packages/durable/README.md:L1--L6`, `L581`
- **Telemetry package:** span interface exists and `telemetryContext` is passed through options, but no span is started in `ai/`, `agent/`, or `ca/` at this commit — recorded as "interface present, emission not found". `packages/telemetry/src/index.ts:L1--L22`; `ai/api/simple-options.ts:L48`

Summary row

Input	Context	Model interface	Output	Control flow
TUI/CLI/RPC prompts, steer/follow-up queues, tool results	Tree-projected JSONL transcript, sectioned prompt, threshold compaction	One StreamFunction, SSE, two-tier retry, overflow classifier	Parallel TypeBox tools, bash/edit/read, QuickJS codemode	Inner tool loop, outer follow-up loop, retry/overflow re-entry, routing

B.4 Hermes Agent

Source: https://github.com/NousResearch/hermes-agent · **Version:** commit 7bf3766f3db72d4643d1a78d18e51c1d9ddf1d6d (2026-10-15) · **Language:** Python (TypeScript for

the TUI/desktop front-ends, out of scope) **Path conventions:** paths are relative to the repo root (`hermes-agent/`). `L` = line. Clone read as untrusted data; nothing executed. **What it is:** A personal agent by Nous Research that runs one agent core (`AIAgent`) behind a prompt_toolkit CLI/TUI, ~25 messaging-platform gateway adapters (Telegram, Discord, Slack, ...), an OpenAI-compatible HTTP server, an ACP adapter, a cron scheduler and a desktop app. The model is reached through an OpenAI-SDK chat-completions client by default, with alternative wire transports for Anthropic Messages, OpenAI Responses ("codex") and Bedrock Converse selected by `api_mode`; providers are plugins and the model slug is config.

Control flow

- **Turn entry + lease:** `AIAgent.run_conversation` builds a task/turn id and admits a durable turn lease before forwarding to the loop (one turn per session at a time). `agent/turn_facade.py:L22`, `agent/turn_facade.py:L83`
- **Main loop:** `_run_conversation_turn` runs `while (api_call_count < max_iterations and iteration_budget.remaining > 0) or _budget_grace_call`; each iteration = begin → prepare → assemble → preflight gate → API retry loop → normalize → `run_tool_round` if tool calls else `finish_text_response`; post-loop `finalize_turn`. `agent/conversation_loop.py:L1656--L1706`
- **Phase dispatch:** each phase is a sibling function returning a verdict with `action ∈ {fallthrough, continue, break, return}`; `_run_phase` copies verdict fields back into `_LoopState`. `agent/conversation_loop.py:L1389--L1502`
- **Inner retry loop:** `_run_api_retry_loop` loops `while retry_count < max_retries` over guard → build → call → check, with `handle_api_interrupt` / `handle_api_error` branches. `agent/conversation_loop.py:L1505--L1534`
- **Step budget:** `IterationBudget` is a thread-safe counter with `consume()`/`refund()`; `max_iterations` defaults to `sys.maxsize` (unlimited, shared with subagents); a tool round whose only tool was `execute_code` refunds its iteration; a reasoning-only stall that triggers a model fallback sets `_budget_grace_call` for one extra call. `agent/iteration_budget.py:L25--L56`, `run_agent.py:L241`, `agent/turn_tool_round.py:L199--L200`, `agent/turn_truncation.py:L630--L635`
- **Stop gates (go again vs. hand back):** on a text answer, `apply_stop_gates` runs verify-on-stop → `pre_verify` plugin hook → kanban stop guard; a nudge appends a user-role row and continues the loop, holding the answer as a budget-exhaustion fallback. `agent/turn_stop_gates.py:L105--L140`, `agent/turn_stop_gates.py:L35--L50`
- **Tool-call guardrail halt:** a `ToolCallGuardrailController` decision can end the turn (`guardrail_halt`) after a tool round; identical-call stall threshold is 3. `agent/turn_tool_round.py:L179--L193`, `agent/tool_guardrails.py:L30`, `agent/tool_guardrails.py:L307`
- **Approval gate (harness consumes the answer):** `check_all_command_guards` runs floor blocks → yolo/`off` bypass (process `--yolo`, session `/yolo`, `approvals.mode`) → permanent allowlist → unattended-context policy → `detect_dangerous_command` → `_human_decision`; answers are `once/session/always/deny/timeout`; in gateway ask mode the tool returns `status="pending_approval"` instead of blocking. `tools/approval.py:L1151--L1200`, `tools/approval.py:L506--L510`, `tools/approval.py:L796--L802`, `tools/approval_prompt.py:L37`, `tools/terminal_tool.py:L1019--L1028`
- **Interrupt / redirect crossing a response:** if a redirect arrived while the request was in flight the response is discarded and a restart is armed; `_apply_active_turn_redirect` then appends a checkpoint + correction row so role alternation holds. `agent/turn_api_call.py:L148--L167`, `agent/conversation_loop.py:L322--L340`
- **Gateway steer:** a message arriving while a turn runs is injected via `agent.steer()` or queued; after a tool batch, pending steer is folded onto tool results once the per-turn budget is enforced. `gateway/run_inbound.py:L649--L664`, `agent/tool_executor.py:L1209--L1220`
- **Delegation (sub-agents):** `delegate_task` refuses when `_delegate_depth >= delegation.max_spawn_depth` (default 1 = flat); children are full `AIAgent` instances built with `child_depth = parent + 1` and role orchestrator/leaf derived from depth. `tools/delegate_tool.py:L520--L527`, `tools/delegate_tool_config.py:L151--L159`, `tools/delegate_tool.py:L199--L234`
- **Child run, parallelism and result return:** `_run_single_child` runs a pre-built child on a worker thread, heartbeats the parent, and returns `{status ∈ completed|interrupted|failed, exit_reason}`;

a batch runs on `DaemonThreadPoolExecutor(max_workers=batch.max_children)` polled with short waits; `background=True` returns a dispatch handle. `tools/delegate_tool.py:L342--L406`, `tools/delegate_tool_dispatch.py:L118--L145`, `tools/delegate_tool_dispatch.py:L470--L478`

- **Work-keyed model routing (auxiliary):** titling, compression summaries, vision etc. resolve via `_resolve_auto_route` (main model → per-task fallback chain → discovery chain); titles come from one small-model call off the critical path. `agent/auxiliary_client.py:L4659--L4684`, `agent/title_generator.py:L1--L5`
- **Mixture-of-Agents turn:** `/moa` gathers reference-model context before each iteration while the normal loop keeps tool calling. `agent/moa_loop.py:L1--L5`
- **Cron turns:** the scheduler runs `agent.run_conversation` on a worker thread under an inactivity watchdog (default 600 s) on a fresh `AIAgent(platform="cron")`. `cron/scheduler.py:L1961--L1963`, `cron/scheduler.py:L2496--L2525`
- **Codex app-server handoff:** when `api_mode == "codex_app_server"` the whole turn is handed to an external codex subprocess; on failure a fallback is activated and the generic loop retries the same user turn. `agent/conversation_loop.py:L1650--L1651`, `agent/conversation_loop.py:L1537--L1552`

Input

- **CLI REPL:** a worker thread drains `_pending_input` (queue) and dispatches each input; `_looks_like_slash_command` routes to `process_command`, else `self.chat(...)`; `/skill-name` is rewritten into a user message. `hermes_cli/cli_tui_runtime_mixin.py:L31--L47`, `hermes_cli/cli_tui_runtime_mixin.py:L139`, `cli.py:L710`, `cli.py:L1300`
- **CLI chat turn:** `chat()` runs the agent on a daemon thread; input typed meanwhile goes to `_interrupt_queue` and is re-queued as the next turn; the call is `run_conversation(user_message, conversation_history[:-1], stream_callback, task_id, persist_user_message)`. `hermes_cli/cli_chat_turn_mixin.py:L49--L60`, `hermes_cli/cli_chat_turn_mixin.py:L115--L118`, `hermes_cli/cli_chat_turn_mixin.py:L326--L330`
- **Gateway event shape:** every adapter normalizes to `MessageEvent(text, message_type, user_id, ...)`; adapters subclass `BasePlatformAdapter`; in-tree adapters (signal, webhook, bluebubbles, weixin, yuanbao, whatsapp_cloud, api_server) plus plugin adapters (telegram, discord, slack, matrix, teams, email, irc, ...). `gateway/platforms/event.py:L45--L51`, `gateway/platforms/base.py:L1876`, `gateway/platforms/` (listing), `plugins/platforms/` (listing)
- **Gateway inbound pipeline:** `_handle_message` = admit → pending-reply intercepts → authority admission → running-session handling (steer/interrupt/preempt) → idle commands → `_handle_message_with_agent`. `gateway/run_inbound.py:L1196--L1300`
- **Authorization and bot-loop budget at admission:** unauthorized users get a pairing code (DM) or are ignored before any session state is touched; bot-authored messages are capped by a sliding window per conversation. `gateway/run_inbound.py:L201--L225`, `gateway/bot_loop_guard.py:L1--L5`
- **Gateway → agent call:** the turn runner calls `agent.run_conversation(api_message, **kwargs)` with `persist_user_message`, display kind and the raw platform message id. `gateway/run_turn_runner.py:L934--L936`
- **HTTP and webhook input:** OpenAI-compatible aiohttp server (`/v1/chat/completions`, `/v1/responses`, `/v1/runs`, ...); webhook payloads are treated as untrusted and get no file/system execution toolset. `gateway/platforms/api_server.py:L1--L5`, `toolsets.py:L41`
- **ACP input:** the ACP agent calls `agent.run_conversation(user_message=..., conversation_history=state.history)`. `acp_adapter/server.py:L816--L819`
- **Model asks the user (clarify):** `clarify` is an inline agent tool whose UI is a platform callback; the gateway intercepts the reply and its result returns to the model. `tools/clarify_tool.py:L1--L6`, `tools/clarify_tool.py:L111`, `gateway/run_inbound.py:L339`
- **Tool results captured:** every executed tool's result is committed as a `role: tool` row (`_commit_tool_result`); invalid names get an error tool row so each call has a matching result. `agent/tool_executor.py:L1066`, `agent/turn_tool_round.py:L116--L121`
- **Multimodal:** native image parts attached to the user turn stay visible to `vision_analyze` for the turn. `agent/conversation_loop.py:L1749`

Context

- **User message staging:** `messages = list(conversation_history)` + staged user dict appended; `current_turn_user_idx` recorded. `agent/turn_context.py:L1104--L1115`
- **System prompt as three cache tiers:** `stable` (SOUL.md from HERMES_HOME or default identity, guidance, pinned skills, coding brief) → `context` (caller `system_message`, project context files, workspace) → `volatile` (skills index, memory, plugin sections, profile line, timestamp, runtime environment). Never re-rendered mid-session. `agent/system_prompt.py:L730--L793`, `agent/system_prompt.py:L796--L805`, `agent/system_prompt.py:L544--L550`, `agent/prompt_builder.py:L1652--L1700`
- **Project context files:** `.hermes.md` → `AGENTS.md` chain → `CLAUDE.md` → `.cursorrules`, loaded from cwd at startup only; cap 20 000 chars floor, dynamic 6 % of the window up to 500 000, overridable by `context_file_max_chars`. `agent/prompt_builder.py:L1783--L1788`, `agent/system_prompt.py:L709--L722`, `agent/prompt_builder.py:L1156--L1171`
- **Memory files:** `MEMORY.md` (2 200 chars) and `USER.md` (1 375 chars) are rendered into the volatile tier; the `memory` tool edits them. `agent/system_prompt.py:L516--L541`, `tools/memory_tool_store.py:L186`, `tools/memory_tool_store.py:L618`
- **External memory providers:** one plugin provider at a time; `build_system_prompt()` block plus per-turn `prefetch_all(query)` text injected into the request. `agent/memory_manager.py:L1--L4`, `agent/memory_manager.py:L501--L511`, `agent/turn_context.py:L903--L931`
- **Skills index + progressive disclosure:** names/descriptions in the prompt; `skill_view` loads a SKILL.md on demand as a tool result. `agent/system_prompt.py:L301--L314`, `tools/skills_tool.py:L2--L5`
- **Byte-stable prompt restored from DB:** on resume the stored `system_prompt` row is reused if it matches the runtime; rebuilt only after compression (`invalidate_system_prompt`). `agent/conversation_loop.py:L749--L772`, `agent/system_prompt.py:L808--L826`, `hermes_state_common.py:L377--L380`
- **Tool definitions:** `get_tool_definitions(enabled/disabled toolsets)` at init; `_HERMES_CORE_TOOLS` is the always-present core. `agent/agent_init.py:L1118`, `model_tools.py:L215`, `toolsets.py:L11--L39`
- **Tool Search deferral:** MCP/plugin and selected core tools are replaced in the model-visible array by `tool_search / tool_describe / tool_call` bridge tools, catalog rebuilt per assembly. `tools/tool_search.py:L1--L7`
- **Request assembly order:** `build_api_messages` → MoA context → canonicalization; "ORDER IS LOAD-BEARING" for cache breakpoints; system row prepended. `agent/turn_request_assembly.py:L106--L130`, `agent/turn_context.py:L1344`
- **Compaction trigger:** `ContextCompressor(threshold_percent=0.50, protect_first_n=3, protect_last_n=20)`; `should_compress_info` fires when `tokens >= threshold_tokens` unless in cooldown or the ineffective breaker; per-model threshold overrides. `agent/context_compressor.py:L2811--L2832`, `agent/context_compressor.py:L3006--L3015`, `agent/context_compressor.py:L2659--L2671`
- **Compaction algorithm:** prune old tool-result bodies (no LLM) → pick boundaries on the pruned copy → auxiliary-model summary → clean orphaned tool pairs. `agent/context_compressor.py:L5528--L5575`, `agent/context_compressor.py:L3285--L3300`
- **Where compaction runs:** turn-start compaction; a pre-API preflight gate comparing the assembled request with the threshold (insufficient progress blocks further passes); 413 / context-length provider errors compress and restart. `agent/turn_context_compaction.py:L129`, `agent/turn_preflight_gate.py:L20--L120`, `agent/turn_overflow.py:L1--L6`, `agent/turn_overflow.py:L463`
- **Gateway session hygiene:** a second compaction layer at 0.85 of the window between turns. `gateway/run_turn_hygiene.py:L94--L102`
- **Tool-output truncation:** 40 % head / 60 % tail with one marker; limits 50 000 bytes, 2 000 lines, 2 000 chars/line; `read_file` pages by char budget and reports `next_offset`. `tools/tool_output_truncate.py:L1--L12`, `tools/tool_output_limits.py:L13--L15`, `tools/file_tools.py:L96--L113`
- **Spill-to-file instead of truncating:** results over the per-tool threshold are written to `$HERMES_HOME/cache/spillover/{id}.txt` and replaced by preview + path; `enforce_turn_budget` caps the batch. `tools/tool_result_storage.py:L1--L6`, `tools/tool_result_storage.py:L293--L300`, `tools/tool_`

result_storage.py:L359

- **Subdirectory hints:** AGENTS.md/CLAUDE.md found while navigating are appended to the tool result, never to the system prompt. agent/subdirectory_hints.py:L1--L5
- **Untrusted-result wrapping:** web/browser/MCP results ≥ 32 chars are wrapped in <untrusted_tool_result> with delimiter neutralization. agent/tool_dispatch_helpers.py:L484--L495, agent/tool_dispatch_helpers.py:L572--L595
- **History source on resume:** CLI loads get_messages_as_conversation(session_id, repair_alternation=True); gateway loads load_transcript then hygiene. hermes_cli/cli_agent_setup_mixin.py:L593--L596, hermes_state_messages.py:L1537--L1545, gateway/run_turn_prepare.py:L693--L700
- **Pluggable context engine:** context.engine selects a plugin that owns should_compress/compress and may expose tools. agent/context_engine.py:L1--L7

Model interface

- **Transport registry:** anthropic, codex, chat_completions, bedrock transports behind ProviderTransport (convert_messages/convert_tools/build_kwargs/normalize_response/validate_response). agent/transports/__init__.py:L18--L31, agent/transports/base.py:L12--L51
- **Client construction:** _build_client picks the Anthropic SDK, the MoA facade, boto3 Bedrock, or the OpenAI SDK client (shared httpx) by api_mode; GPT-5.x on a direct OpenAI URL auto-upgrades chat_completions → codex_responses. agent/agent_init.py:L1029--L1042, agent/agent_init.py:L990--L1026, agent/agent_init.py:L482--L499
- **Request kwargs:** _build_api_kwargs_for_mode branches anthropic / bedrock / codex / chat-completions builders. agent/chat_completion_helpers.py:L1555--L1572
- **Per-attempt request build:** re-apply reasoning echo for the provider, re-render prompt-cache decoration, strip images for rejecting models, LLM request middleware (trace kept), pre_api_request hook, optional HERMES_DUMP_REQUESTS dump. agent/turn_api_request.py:L93--L205, agent/turn_api_request.py:L152--L153
- **Prompt-cache decoration:** build_prompt_cache_plan places Anthropic cache_control markers; OpenAI-style history is converted to Anthropic content blocks. agent/prompt_caching.py:L298--L308, agent/anthropic_message_convert.py:L1--L4
- **Call issue:** perform_api_call chooses streaming (_should_stream: preferred even with no consumer, off for ACP/external-process providers) and runs through run_llm_execution_middleware. agent/turn_api_call.py:L67--L167, agent/turn_api_call.py:L46--L64
- **Streaming receive:** _StreamingCall issues chat.completions.create(**stream_kwargs) with stream_options.include_usage; the Anthropic path uses messages.stream; a cross-turn stale-stream breaker runs first. agent/chat_completion_helpers.py:L3993--L4006, agent/chat_completion_helpers.py:L2734, agent/chat_completion_helpers.py:L2957--L2966, agent/chat_completion_helpers.py:L3412
- **Non-stream call on a worker thread:** per-request client so interrupts close only that connection; a stale-call detector kills and raises. agent/chat_completion_helpers.py:L1292--L1305
- **Retry policy:** api_max_retries default 3; wait = provider Retry-After (capped) else jittered_backoff(retry, base 2.0, max 60) = min(base·2^(n-1), max) + jitter; adaptive policy for 429. agent/agent_init.py:L1441--L1446, agent/turn_recovery.py:L1401--L1430, agent/retry_utils.py:L121--L130, agent/turn_api_error.py:L406--L417
- **Error classification:** classify_api_error → structured ClassifiedError (retryable / should_compress / rotate credential / fallback); plugins may override via transform_api_error_classification. agent/error_classifier.py:L974--L980, hermes_cli/plugins.py:L126--L130
- **Backup model when unservable:** try_activate_fallback walks _fallback_chain, swapping client/model/provider in place, on classified failover or after retries exhaust; a persistent credential pool gives same-provider failover. agent/chat_completion_helpers.py:L2086--L2121, agent/turn_api_error.py:L341, agent/turn_api_error.py:L380, agent/credential_pool.py:L1
- **Post-exhaustion auto-recovery:** when retries and the chain are spent on a transient outage, the turn parks with a countdown and re-enters the retry loop (auto_recovery_cycles default 5). agent/turn_

recovery_autorecover.py:L1--L7

- **Response validation + normalization:** validate_response_shape via the transport; normalize_model_response yields an assistant message with string content. agent/turn_recovery.py:L1524--L1530, agent/turn_response_intake.py:L120--L128
- **Degenerate-response ladders:** empty/thinking-only → partial-stream recovery → nudge → prefill continuation ×2 → budgeted retries → fallback provider; finish_reason=length → continuation nudges (up to 4), truncated tool-call retries with max_tokens boosts, repetition abort. agent/turn_empty_response.py:L1--L5, agent/turn_truncation.py:L1--L6, agent/repetition_guard.py:L141
- **Usage receipt:** record_response_usage folds response.usage into the compressor, session counters and state.db. agent/turn_usage.py:L73--L79

Output

- **Tool set:** core tools (terminal, process_manage, read/write/patch/search files, web, vision, image, skills, browser_, *todo, memory, session_search, clarify, execute_code, delegate_task, cronjob, kanban_*, computer_use). toolsets.py:L11--L39
- **Dispatch precedence:** inline agent-level tools (INLINE_TOOL_EXECUTORS table) → delegate_task → context-engine tools → memory-provider tools → registry via handle_function_call. agent/tool_executor.py:L1669--L1733, agent/inline_tool_executors.py:L253--L313
- **Batch planning:** >1 calls are split into parallel/sequential segments; the parallel-safe set is read-only tools plus path-non-overlapping file ops. run_agent.py:L1299--L1325, agent/tool_dispatch_helpers.py:L212--L257, agent/tool_dispatch_helpers.py:L33--L44
- **Concurrent execution:** _ConcurrentBatch runs on DaemonThreadPoolExecutor(max_workers) with a batch deadline; results appended in call order. agent/tool_executor.py:L1573--L1620, agent/tool_executor.py:L1500--L1501
- **Registry call path:** handle_function_call → aliases → bridge unwrap → request middleware → _pre_dispatch_guards (pre_tool_call hook, fail-closed on timeout) → registry.dispatch → post_tool_call → transform_tool_result. model_tools.py:L873--L967, model_tools.py:L823--L846, hermes_cli/plugins_dispatch.py:L51
- **Terminal tool:** plan → acquire env → bounded pre-exec guard → approval guards → foreground or background spawn; description tells the model output is auto-truncated to a file. tools/terminal_tool.py:L1360--L1475, tools/terminal_tool.py:L160--L165
- **Execution backends (sandboxes):** local, docker, singularity/apptainer, modal, daytona, vercel_sandbox, ssh, plus plugin envs; built by _ENV_BUILDERS. tools/terminal_tool_backends.py:L54, tools/terminal_tool_backends.py:L239--L241
- **Programmatic tool calling:** execute_code runs a model-written Python script in a persistent per-conversation kernel that calls Hermes tools over RPC; only stdout returns. tools/code_execution_tool.py:L1--L10, tools/code_kernel.py:L1--L5
- **Browser:** agent-browser CLI over local headless Chromium, cloud backends, a user CDP endpoint or Camofox; accessibility-tree snapshots with @eN refs. tools/browser_tool.py:L1--L8
- **Persist-before-execute:** the assistant tool-call row is flushed to SQLite before any tool runs; a failed append ends the turn. agent/turn_tool_round.py:L131--L153
- **Rendering and delivery:** single-writer stream ownership with delta/reasoning fan-out; open stream boxes flushed before tools; gateway delivery.py routes responses to targets and response_filters.py decides whether a turn is delivered to the chat. agent/stream_delivery.py:L1--L3, agent/turn_tool_round.py:L160--L164, gateway/delivery.py:L1--L2, gateway/response_filters.py:L1--L4
- **Sub-agent spawn tool:** delegate_task (output) whose effect is the control-flow delegation above. tools/delegate_tool.py:L488

Production concerns

Concern	Where it lives (primitive)	Mechanism	Evidence
Persistence	Output (persist-before-execute), Context (prompt/history restore), Input (session resume)	SQLite `state.db` in WAL mode: `sessions`, `messages` (role/content/tool_calls/reasoning/...), `system_prompts`, `session_model_usage`, FTS5 search; `_flush_messages_to_session_db` dedups by marker; memory in `MEMORY.md/USER.md`; spillover files; trajectories	`hermes_state.py:L177`, `hermes_state.py:L460--L466`, `hermes_state_common.py:L451--L470`, `agent/session_persistence.py:L467--L511`, `agent/turn_finalizer.py:L608`, `agent/turn_finalizer.py:L635`
Safety & security	Output (sandbox, approvals, path denies), Context (injection scans, untrusted wrapping), Control flow (gate decision)	OS is the only boundary; hardline + dangerous command regexes with human gate; write-denied credential paths (`.ssh`, `.env`, ...); SSRF-safe URL policy; secret redaction of logs/tool output; threat-pattern scan of context files, skills and tool results; `UnscopedSecretError` secret scoping	`SECURITY.md:L58--L65`, `tools/approval_detection.py:L107`, `tools/approval_detection.py:L228`, `agent/file_safety.py:L176--L196`, `agent/file_safety.py:L210`, `tools/url_safety.py:L1--L9`, `agent/redact.py:L1--L3`, `agent/prompt_builder.py:L83--L110`, `tools/skills_guard.py:L240--L253`, `agent/tool_dispatch_helpers.py:L544--L562`
Observability	across (recorded at Model interface and Output)	per-call usage folded into state.db; `pre/post_api_request`, `pre/post_tool_call`, `pre/post_auxiliary_call` plugin hooks; gateway monitoring events exported over OTLP (opt-in); `HERMES_DUMP_REQUESTS`; middleware traces on requests	`agent/turn_usage.py:L73--L79`, `hermes_state_common.py:L484`, `hermes_cli/plugins.py:L106--L135`, `agent/monitoring/events.py:L16--L66`, `agent/monitoring/otlp_exporter.py:L1--L5`, `agent/turn_api_request.py:L152--L153`
Resilience	Model interface (retries, fallback, auto-recovery), Context (compaction on overflow), Output (bounded guards)	3 retries with Retry-After/jittered backoff; fallback chain and credential pool; post-exhaustion ladder; 413/context-length → compress+restart; stale-stream breaker; turn liveness watchdog; cron inactivity watchdog; resume after restart from persisted tool-call rows	`agent/turn_recovery.py:L1401--L1430`, `agent/chat_completion_helpers.py:L2086--L2121`, `agent/turn_recovery_autorecover.py:L1--L7`, `agent/turn_overflow.py:L1--L6`, `agent/chat_completion_helpers.py:L4005`, `agent/turn_liveness.py:L90`, `cron/scheduler.py:L1961--L1963`, `agent/turn_tool_round.py:L131--L153`

Concern	Where it lives (primitive)	Mechanism	Evidence
Performance	Context (cache tiers, truncation, spill), Output (parallel tools), Model interface (streaming, cache_control)	system prompt byte-stable in stable→context→volatile tiers; compression is the only sanctioned cache break; head/tail truncation and spill files; parallel-safe tool segments; streaming by default; Anthropic cache markers; tool-definition cache	`agent/system_prompt.py:L730--L793`, `agent/system_prompt.py:L808--L826`, `tools/tool_output_truncate.py:L1--L12`, `agent/tool_dispatch_helpers.py:L212--L257`, `agent/turn_api_call.py:L46--L64`, `agent/prompt_caching.py:L298--L308`, `model_tools.py:L215`
Evaluation	across	6 392 pytest files run per-file-isolated by `scripts/run_tests.sh`; markers for `integration/live` provider canaries; `evals/` harnesses (compaction recall eval, provider_fallback, tool_search, token_accounting, ...); `batch_runner.py` runs the agent over JSONL datasets; `mini_swe_runner.py` runs SWE tasks in the execution environments	`scripts/run_tests.sh:L1--L15`, `pyproject.toml:L691--L700`, `evals/compaction/README.md:L1--L12`, `batch_runner.py:L1--L6`, `mini_swe_runner.py:L1--L6`, `tests/evals/` (listing)

Design decisions (one line each: the choice this harness made within a primitive)

- Input: one normalized `MessageEvent/run_conversation(user_message, conversation_history, task_id)` seam shared by CLI, ~25 gateway adapters, HTTP, ACP and cron; mid-turn messages become steer/redirect rather than new turns.
- Context: a three-tier byte-stable system prompt (identity → project files → volatile memory/skills/time) rebuilt only after compression; compaction at 50 % of the window by prune-then-summarize with a separate auxiliary model; oversized tool output is spilled to files, not dropped.
- Model interface: OpenAI chat-completions wire by default with pluggable transports (Anthropic, Responses, Bedrock) and provider plugins; streaming always on; 3 retries with backoff, then a configurable fallback chain, then a bounded auto-recovery wait.
- Output: a fixed core tool set sent on every call plus a Tool Search bridge for the long tail; tool batches split into parallel-safe segments; terminal execution behind pluggable sandboxes (local/docker/modal/...) and a regex-plus-human approval gate; tool-call rows persisted before execution.
- Control flow: synchronous phase-verdict loop with an unlimited default iteration budget, verify-on-stop nudges that re-enter the loop, flat delegation by default (`max_spawn_depth=1`) with threaded children, and task-keyed auxiliary model routing.

Hard cases

- **Tool Search bridge (`tool_search/tool_describe/tool_call`):** a tool call the model makes (output) whose only job is to decide which tool schemas the model sees (context); resolved by "assign by function": the catalog read is context, the unwrapped `tool_call` re-dispatch is output. `tools/tool_search.py:L1--L7`, `model_tools.py:L701--L709`
- **`execute_code` programmatic tool calling:** a program the model writes at run time that sequences only tools → output (rule 27); but the loop refunds the iteration it used, which is control flow. `tools/code_execution_tool.py:L1--L10`, `agent/turn_tool_round.py:L199--L200`
- **Subdirectory hints and untrusted wrapping inside the tool result path:** code in the output executor that changes what the model is shown; assigned to context by rule 20. `agent/subdirectory_hints.py:L1--L5`, `agent/tool_dispatch_helpers.py:L572--L595`

- **Stop-gate nudges:** the harness appends a synthetic user row and goes again; the decision is control flow, the row's text is context, and it is not input (no person wrote it). `agent/turn_stop_gates.py:L105--L140`
- **Approval inside the terminal tool:** the gate runs inside the tool executor and blocks the worker thread (CLI) or returns `pending_approval` and ends the call (gateway); the person's answer is consumed by the harness → control flow by rule 26, even though the code lives in `tools/`. `tools/terminal_tool.py:L1013--L1047`
- **Reasoning-only stall → model fallback:** a model swap triggered by unusable output rather than an HTTP failure; placed in model interface because the request "could not be served" with content, not keyed to the work. `agent/turn_truncation.py:L630--L635`
- **Prompt-cache decoration:** `cache_control` markers are wire-format (model interface) but the tiered ordering of the system prompt exists only to keep them effective (context design); split across both. `agent/prompt_caching.py:L298--L308`, `agent/system_prompt.py:L730--L793`
- **Gateway session hygiene (0.85 compaction):** compaction performed by the gateway outside the agent turn; still context (fits the window) but hosted in a different process layer. `gateway/run_turn_hygiene.py:L94--L102`
- **Codex app-server mode — DOES NOT FIT cleanly:** with `api_mode == "codex_app_server"` the whole turn (loop, tool execution, model calls) is handed to an external OpenAI Codex subprocess and Hermes only projects its events into rows and arms a fallback on failure; the five primitives collapse into one opaque external harness that is not a sub-agent (no delegation, no outer loop). `agent/conversation_loop.py:L1650--L1651`, `agent/conversation_loop.py:L1537--L1552`
- **ACP / external-process "providers":** `acp://` base URLs make the "model" another agent process; streaming is disabled and the model interface boundary is another harness. `agent/turn_api_call.py:L46--L56`
- **Mixture-of-Agents:** several model calls per iteration issued by the harness (control flow) whose outputs are appended to the user message (context). `agent/moa_loop.py:L1--L5`

Summary row

Input	Context	Model interface	Output	Control flow
One MessageEvent seam: CLI, 25 gateways, HTTP, ACP, cron, steer	Tiered byte-stable prompt, memory files, 50 % prune-then-summarize compaction	Pluggable transports, streaming, 3 retries, fallback chain, auto-recovery	Core tools + bridge, parallel segments, sandboxed terminal, approval gate	Phase-verdict loop, unlimited budget, stop-gate nudges, flat threaded delegation

B.5 LangGraph (prebuilt ReAct agent on the Pregel runtime)

Source: https://github.com/langchain-ai/langgraph · **Version:** commit 5eb60aef6f3837a1fb632a8abc6cf8e8da81785b (2026-10-14; packages `langgraph` 1.2.15, `langgraph-prebuilt` 1.1.1, `langgraph-checkpoint`; the local clone holds this single squashed commit, so no file history is available) · **Language:** Python **Path conventions:** `LG/` = `libs/langgraph/langgraph/`; `PB/` = `libs/prebuilt/langgraph/prebuilt/`; `CP/` = `libs/checkpoint/langgraph/`; `T/` = `libs/*/tests/`. "User-written" marks a decision the framework leaves to the person building the graph. **What it is:** LangGraph (LangChain Inc.) is a framework for writing agent control flow as a graph of nodes over a shared, checkpointed state; the Pregel runtime executes the graph in bulk-synchronous supersteps. The canonical harness deconstructed here is `create_react_agent` (`PB/chat_agent_executor.py`): an `agent` node that calls a LangChain chat model and a `tools` node (`ToolNode`) that runs the model's tool calls, cycling until the model stops calling tools. The model is reached through a `langchain_core` `BaseChatModel` (`.invoke`/`.ainvoke`), which lives outside this repository; LangGraph itself contains no provider client.

Control flow

- **ReAct loop shape (user-written graph, built by the prebuilt):** `agent` and `tools` nodes are added and the `tools` node's edge returns to the entrypoint, so the loop is an edge, not a `while`. `PB/chat_agent_executor.py:L867--L872`, `L985--L990`
- **Go again / stop decision:** `should_continue` reads the last message; no `tool_calls` → `END` (or `post_model_hook` / `generate_structured_response`), otherwise route to `tools`. `PB/chat_agent_executor.py:L831--L859`; attached as a conditional edge `L964--L968`
- **v2 fan-out, one task per tool call:** `should_continue` returns one `Send("tools", ToolCallWithContext(...))` per tool call, so each call becomes its own Pregel task. `PB/chat_agent_executor.py:L849--L859`; `Send` type `LG/types.py:L738--L826`
- **`return_direct` hand-back:** tools flagged `return_direct` end the loop after they run; `route_tool_responses` returns `END` when the newest ToolMessages came from such a tool. `PB/chat_agent_executor.py:L597`, `L970--L990`
- **post_model_hook router (user-written hook, framework routing):** after the hook, pending tool calls without ToolMessages are re-sent to `tools`; a trailing ToolMessage routes back to the entrypoint; else END/structured. `PB/chat_agent_executor.py:L919--L962`
- **Step budget inside the agent (`remaining_steps`):** a managed value computed as `scratchpad.stop - scratchpad.step`. `LG/managed/is_last_step.py:L18--L24`; `stop = step + recursion_limit + 1` `LG/pregel/_loop.py:L1858--L1859`; `_are_more_steps_needed` returns True when `< 2` steps remain and the response has tool calls `PB/chat_agent_executor.py:L620--L634`; the model's response is then replaced by an AIMessage "Sorry, need more steps to process this request." `L684--L694`
- **Recursion limit (runtime step budget):** `tick()` sets `status = "out_of_steps"` when `step > stop`. `LG/pregel/_loop.py:L640--L643`; `stream()` then raises `GraphRecursionError` `LG/pregel/main.py:L3093--L3102`; error class `LG/errors.py:L68--L88`; default `DEFAULT_RECURSION_LIMIT = int(getenv("LANGGRAPH_DEFAULT_RECURSION_LIMIT", "10007"))` `LG/_internal/_config.py:L32`
- **Pregel superstep loop:** `while loop.tick(): ... runner.tick(...) ... loop.after_tick()`; channel updates from step N are visible only in step N+1. `LG/pregel/main.py:L3050--L3079`
- **Natural stop:** the loop ends when `prepare_next_tasks` yields no tasks (`status = "done"`). `LG/pregel/_loop.py:L686--L689`
- **Which node runs next (the scheduler):** `prepare_next_tasks` first consumes pending `Send` packets from the reserved `TASKS` topic (PUSH tasks), then PULL tasks for nodes whose trigger channels were updated. `LG/pregel/_algo.py:L391--L512` (PUSH `L441--L465`, PULL `L489--L511`); a node triggers when a trigger channel's version exceeds the version it last saw `L1259--L1276`
- **Apply writes between steps:** `apply_writes` sorts tasks by path for determinism, records `versions_seen`, updates channels and bumps versions. `LG/pregel/_algo.py:L254--L258`, `L293--L344`
- **Conditional edges as routing-by-channel:** `add_conditional_edges` stores a `BranchSpec` `LG/graph/state.py:L982--L1030`; at run time `_route` reads fresh state, calls the user's path function, maps results through `path_map`, and writes to `branch:to:<node>` channels `LG/graph/_branch.py:L146--L167`, `L192--L225`; `attach_branch` installs the branch as a writer on the source node `LG/graph/state.py:L1577--L1624`; plain edges write the same way `L1551--L1575`; each node is triggered by its own `branch:to:` ephemeral channel `L1525--L1532`
- **`Command(goto=...)` from a node or tool:** `_control_branch` turns `goto` into TASKS (`Send`) or `branch:to:` writes; `Command.PARENT` raises `ParentCommand`. `LG/graph/state.py:L1749--L1775`; `ParentCommand` is bubbled to the parent graph by `run_with_retry` `LG/pregel/_retry.py:L618--L631`; the exception class `LG/errors.py:L130--L134`
- **Send packets are tasks of the next (or same) superstep:** `prepare_push_task_send` turns `TASKS[idx]` into a task for the named node `LG/pregel/_algo.py:L937--L996`; `accept_push` schedules pushes emitted mid-step `LG/pregel/_loop.py:L584--L621`; `TASKS` is a non-accumulating `Topic(Send)` `LG/pregel/main.py:L814--L819`
- **Static breakpoints (`interrupt_before`/`interrupt_after`):** `should_interrupt` returns tasks whose node is listed, if any channel changed since the last interrupt `LG/pregel/_algo.py:L156--L186`; checked before running tasks `LG/pregel/_loop.py:L696--L701` and after the step is checkpointed `L787--L792`; the prebuilt passes the user's lists straight to `compile` `PB/chat_agent_executor.py:L995--L1002`

- **Dynamic pause (`interrupt()`):** first call in a task raises `GraphInterrupt` carrying an `Interrupt` whose id hashes the task namespace `LG/types.py:L1038--L1046`; on re-run it returns the stored resume value instead `L1019--L1036`; `GraphInterrupt` `LG/errors.py:L103--L108`
- **Pause bookkeeping:** the runner saves the interrupt as an `INTERRUPT` pending write `LG/pregel/_runner.py:L586--L592`; at the root graph the exception is swallowed, state is emitted and the run returns `LG/pregel/_loop.py:L1491--L1530`
- **Resume (harness-consumed answer):** `Command(resume=...)` maps to a `RESUME` write `LG/pregel/_io.py:L56--L78`; `_first` requires a checkpointer and demands interrupt ids when several interrupts are pending `LG/pregel/_loop.py:L992--L1009`; the task scratchpad looks up its resume value `LG/pregel/_algo.py:L1279--L1344`; resume is inferred from `None` input or a `Command` `LG/pregel/_loop.py:L915--L928`
- **Approval gate is user-written:** the framework ships only the `HumanInterrupt`/`HumanResponse` shapes (`accept`/`ignore`/`response`/`edit`); no code in the harness consumes them. `PB/interrupt.py:L51--L84`, `L87--L105`
- **Model routing keyed to the work (user-written):** `model` may be a callable `(state, runtime) -> BaseChatModel` resolved on every `agent` step. `PB/chat_agent_executor.py:L563--L564`, `L599--L618`, `L674--L677`
- **Second model call for structured output:** after the loop ends, `generate_structured_response` runs as its own node. `PB/chat_agent_executor.py:L744--L767`, wiring `L899--L910`
- **Node concurrency within a superstep:** all ready tasks are submitted to the executor and awaited with `FIRST_COMPLETED`; the first non-interrupt failure cancels the rest. `LG/pregel/_runner.py:L259--L290`, `L618--L636`, `L652--L699`
- **Cooperative drain (operator stop):** `RunControl.request_drain()` is checked at the superstep boundary; the loop sets `status="draining"` and `stream()` raises `GraphDrained` after checkpointing. `LG/runtime.py:L79--L104`; `LG/pregel/_loop.py:L706--L708`; `LG/pregel/main.py:L3103--L3106`; `LG/errors.py:L55--L65`
- **Per-node error-handler nodes (user-written handler, framework routing):** `compile` assigns default/explicit handler nodes `LG/graph/state.py:L1290--L1344`; the runner reroutes a failed task to its handler instead of panicking `LG/pregel/_runner.py:L298--L324`
- **Sub-agents / subgraphs:** a compiled agent is used as a node of another graph via `name` `PB/chat_agent_executor.py:L466--L468`; nesting is detected by `CONFIG_KEY_TASK_ID` in config `LG/pregel/_loop.py:L351`; a child's `Command(graph=PARENT, goto=[Send...])` is bubbled and merged by `ToolNode` so a tool can hand work to the parent `PB/tool_node.py:L903--L927`. Results return as channel writes to the parent task; depth is unbounded (namespace `parent|child:task_id`).
- **`tools_condition` helper:** standalone routing function (`"tools"` or `"__end__"`) for user-built loops. `PB/tool_node.py:L1616--L1619`
- **Hook nodes (user-written):** `pre_model_hook` runs before every `agent` step, `post_model_hook` after it; both are ordinary nodes the user supplies. `PB/chat_agent_executor.py:L876--L881`, `L891--L896`

Input

- **Person's message enters as graph input:** `map_input` turns the input dict into writes on the input channels. `LG/pregel/_io.py:L81--L97`; applied as a null-task write set on the first tick `LG/pregel/_loop.py:L1086--L1121`; the hidden `START` node forwards it into state `LG/graph/state.py:L1508--L1514`
- **Coercion of raw messages:** `add_messages` converts dicts/tuples to `BaseMessage`s and assigns UUIDs to id-less messages. `LG/graph/message.py:L190--L211`; ids are fixed before writes are stored `LG/pregel/_loop.py:L1087--L1089`, `L461--L463`
- **Tool results captured as `ToolMessage`:** raw tool returns are normalized (`Command`, `ToolMessage`, or list) `PB/tool_node.py:L1466--L1487`; non-string outputs are JSON-dumped `L317--L344`; failures become `ToolMessage(status="error")` `L1022--L1029`
- **Human answer the model consumes:** when a tool calls `interrupt()`, the resume value is returned to the tool code on re-run (`L1023--L1036` of `LG/types.py`) and becomes input only if the tool puts it in its `ToolMessage` (user-written). Shape: `PB/interrupt.py:L87--L105`

- **State edits from outside (`Command(update=...)`):** mapped to null-task channel writes. `LG/pregel/_io.py:L76--L78`
- **Run-scoped `context` (operator-supplied, not model-visible unless user code shows it):** built into `Runtime` and passed to nodes and tools. `LG/pregel/main.py:L2963--L2973`; `ToolRuntime.context` `PB/tool_node.py:L814--L825`

Context

- **Prompt assembly (user chooses form):** `_get_prompt_runnable` prepends a `SystemMessage` (from `str`/`SystemMessage`) to `state["messages"]`, or runs a user callable/Runnable over the whole state. `PB/chat_agent_executor.py:L137--L170`
- **What the model is shown each call:** `state["messages"]`, or `llm_input_messages` produced by `pre_model_hook` (which does not alter stored history). `PB/chat_agent_executor.py:L636--L658`; contract documented `L396--L424`
- **Tool descriptions:** `bind_tools(tool_classes + llm_builtin_tools)` unless the user already bound them; `_should_bind_tools` cross-checks names (OpenAI- and Anthropic-style schemas). `PB/chat_agent_executor.py:L582--L589`, `L173--L217`
- **Conversation as reducer-merged state:** `AgentState.messages` uses `add_messages`, which upserts by id and applies `RemoveMessage`. `PB/chat_agent_executor.py:L57--L62`; `LG/graph/message.py:L218--L237`
- **Window fitting (left to the user):** the only primitives are `RemoveMessage(id=REMOVE_ALL_MESSAGES)` to replace history `LG/graph/message.py:L212--L216` and the `pre_model_hook` slot `PB/chat_agent_executor.py:L396--L424`; trimming/summarization code: not found in this repository (grep for `trim_messages`/`summariz` hits only docstrings).
- **History validation before the call:** every `AIMessage.tool_calls` id must have a `ToolMessage`, else `INVALID_CHAT_HISTORY`. `PB/chat_agent_executor.py:L243--L271`
- **Node input is read from channels:** `_proc_input` collects the node's subscribed channels plus managed values, then applies the schema mapper. `LG/pregel/_algo.py:L1347--L1391`; conditional edges read a fresh copy including the node's own writes `L189--L225`; node reads/writes go through `CONFIG_KEY_READ`/`CONFIG_KEY_SEND` `LG/pregel/_read.py:L74--L91`, `LG/pregel/_write.py:L106--L126`
- **Long-term memory for the model (cross-thread store):** `BaseStore.get/put/search` `CP/store/base/__init__.py:L708`, `L756`, `L784`, `L866`; injected into tools via `InjectedStore` `PB/tool_node.py:L1441--L1449`; retrieval and what to show the model are user-written.
- **Hidden state injected into tools (not shown to the model):** `InjectedState`/`ToolRuntime` args are filled from graph state at call time and any model-supplied values for those names are stripped. `PB/tool_node.py:L1400--L1439`, `L1455--L1463`; `ToolRuntime` fields `L1756--L1764`
- **State snapshot travels with each tool task:** `ToolCallWithContext` carries `state` inside the `Send`; `_extract_state` hydrates it (or reads channels). `PB/tool_node.py:L306--L314`, `L1315--L1347`
- **Structured-output prompt:** an optional system prompt is prepended for the final structured call. `PB/chat_agent_executor.py:L756--L758`
- **Agent name stamped on responses:** `response.name = name` (used in multi-agent transcripts). `PB/chat_agent_executor.py:L681--L682`
- **Injection of `config`/`store`/`writer`/`runtime` into node functions by signature:** `LG/_internal/_runnable.py:L168--L219`, `L392--L419`

Model interface

- **The call:** `static_model.invoke(model_input, config)` (async `ainvoke`); the model is a `langchain_core` `BaseChatModel`, outside this repo. `PB/chat_agent_executor.py:L672--L679`, `L699--L707`; string ids resolved with `init_chat_model` `L569--L580`
- **Structured output call:** `model.with_structured_output(schema).invoke(messages, config)`. `PB/chat_agent_executor.py:L761--L766`
- **Receiving a token stream:** `StreamMessagesHandler` is a LangChain callback: `on_chat_model_start` records namespace/metadata, `on_llm_new_token` forwards each `ChatGenerationChunk`, `on_llm_end`

emits the final message once. LG/pregel/_messages.py:L139--L158, L160--L173, L175--L188, L106--L113; installed when "messages" is in stream_mode LG/pregel/main.py:L2902--L2918; modes LG/types.py:L131--L133

- **Retries of a failed request (per node, user-configured):** RetryPolicy(initial_interval=0.5, backoff_factor=2.0, max_interval=128.0, max_attempts=3, jitter=True, retry_on=default_retry_on) LG/types.py:L427--L446; default predicate retries ConnectionError and HTTP 5xx, not ValueError/TypeError/OSError etc. LG/_internal/_retry.py:L1--L29; loop with exponential backoff and jitter LG/pregel/_retry.py:L641--L682; policy comes from the node or the graph L579; create_react_agent sets none and Pregel defaults to () LG/pregel/main.py:L786, L832--L834
- **Backup model when the request cannot be served:** not found. The dynamic-model callable is keyed to state/context, not to failure (PB/chat_agent_executor.py:L599--L606).
- **Timeouts per attempt:** TimeoutPolicy(run_timeout, idle_timeout, refresh_on) LG/types.py:L461--L523; async watchdog tasks race the node LG/pregel/_retry.py:L422--L470; NodeTimeoutError is deliberately retryable LG/errors.py:L191--L207; sync nodes cannot use timeouts LG/pregel/_retry.py:L580--L583
- **Sync/async mismatch guard:** an async model callable invoked synchronously raises. PB/chat_agent_executor.py:L664--L670

Output

- **Tool executor (ToolNode):** parses the last AIMessage.tool_calls (or a single ToolCallWithContext) PB/tool_node.py:L1258--L1300; runs all calls concurrently with a thread pool executor.map L829--L832 or asyncio.gather L863--L866; results combined into {"messages": [...]} L870--L895
- **One call's execution:** tool.invoke(call_args, config) after injection PB/tool_node.py:L960--L966; unknown tool name → error ToolMessage listing valid tools L1302--L1313, template L116--L118
- **Error-to-message policy (user-configurable):** default handler only converts argument-validation errors (ToolInvocationError) and re-raises everything else PB/tool_node.py:L391--L399; handle_tool_errors may be bool/str/callable/exception types L1002--L1029; templates L119--L129
- **Interceptor around each call (user-written):** wrap_tool_call(request, execute) can retry, cache, rewrite or short-circuit a call. PB/tool_node.py:L1061--L1088
- **Tool returning Command:** validated and passed through so the tool's EFFECT (state update / goto) reaches the scheduler. PB/tool_node.py:L1473--L1474, L897--L928
- **Interrupt raised inside a tool is never caught:** GraphBubbleUp passes through the error handling. PB/tool_node.py:L986--L996
- **Rendering to the person (stream modes):** values after each step LG/pregel/_loop.py:L751--L759; updates/tasks per task L1571--L1621; _emit filters by subscribed modes L1535--L1569; final value of invoke is the last values chunk plus collected interrupts LG/pregel/main.py:L4000--L4043
- **Custom and tool-progress streams:** stream_writer for "custom" LG/pregel/main.py:L2932--L2951; "tools" mode installs StreamToolCallHandler (on_tool_start/on_tool_end) L2921--L2929, LG/pregel/_tools.py:L35, L228--L258; tools push partial output with ToolRuntime.emit_output_delta PB/tool_node.py:L1766--L1784
- **Sandboxing of tool execution:** not found; tools run in-process in the caller's executor (PB/tool_node.py:L829--L832).
- **Argument validation without execution:** ValidationNode checks tool calls against schemas and returns error ToolMessages for the model to fix. PB/tool_validator.py:L47

Production concerns

Concern	Where it lives (primitive)	Mechanism	Evidence
Persistence	Control flow (loop state) and Context (conversation state)	`Checkpoint` = channel values + versions + versions_seen per thread; `BaseCheckpointSaver.put` after every superstep and `put_writes` per task; `durability` `sync`/`async`/`exit` (default `async`); `thread_id` required; backends `InMemorySaver`, `SqliteSaver`, `PostgresSaver`; resume loads latest checkpoint or a given `checkpoint_id` (time travel); cross-thread `BaseStore`	`CP/checkpoint/base/__init__.py:L93--L124, L301--L342; LG/pregel/_loop.py:L785--L786, L498--L528, L1261--L1263, L1479--L1489; LG/types.py:L98--L104; LG/pregel/main.py:L2682--L2686, L2695--L2696, L3078--L3079; LG/pregel/_loop.py:L1783--L1813; CP/checkpoint/memory/__init__.py:L34; libs/checkpoint-sqlite/langgraph/checkpoint/sqlite/__init__.py:L49; libs/checkpoint-postgres/langgraph/checkpoint/postgres/__init__.py:L45`
Safety & security	Output (tool args), Control flow (pauses), Persistence layer	No sandbox (not found). Approval = user-written `interrupt()` before/inside a tool. Model cannot forge `InjectedState`/`InjectedStore` args (stripped). Resume values failing an interrupt's `response_schema` are never fed back to the model. Checkpoint deserializer warns it must not see untrusted data; `LANGGRAPH_STRICT_MSGPACK` / msgpack allowlist; `EncryptedSerializer` wraps payloads; `TAG_HIDDEN` keeps nodes out of streams/breakpoints	`PB/tool_node.py:L1455--L1463, L997--L1001; CP/checkpoint/serde/jsonplus.py:L104--L135; CP/checkpoint/base/__init__.py:L742--L771; CP/checkpoint/serde/encrypted.py:L8--L35; LG/pregel/_algo.py:L175--L182`
Observability	Across (traces via LangChain callbacks)	`on_chain_start/end/error` per run with `ls_integration: langgraph` metadata; per-task metadata `langgraph_step/node/triggers/path/checkpoint_ns`; `checkpoints/tasks/debug` stream modes; retry attempts logged; `get_state/get_state_history`; `ExecutionInfo` (thread/run/task ids, attempt). Token accounting: not found in LangGraph (left to `langchain_core`/LangSmith).	`LG/pregel/main.py:L2865--L2890, L3108--L3112; LG/pregel/_algo.py:L653--L659; LG/pregel/_loop.py:L665--L684, L711; LG/pregel/_retry.py:L676--L680; LG/pregel/main.py:L1464, L1514; LG/runtime.py:L27--L52`

Concern	Where it lives (primitive)	Mechanism	Evidence
Resilience	Model interface (retry/timeout), Control flow (resume, error handlers), Output (tool errors)	`RetryPolicy` with backoff; `TimeoutPolicy` with idle/run watchdogs; `step_timeout` on a whole superstep; on resume, finished tasks' pending writes are re-applied so only unfinished tasks re-run; failed tasks routed to error-handler nodes; first failure cancels sibling tasks; `GraphDrained` on SIGTERM-style drain; tool errors returned to the model as messages. Compaction on overflow: not in framework (user `pre_model_hook`).	`LG/pregel/_retry.py:L641--L682, L422--L470; LG/pregel/main.py:L781, L3060; LG/pregel/_loop.py:L805--L821, L691--L694; LG/pregel/_runner.py:L298--L324, L652--L699; LG/pregel/_loop.py:L706--L708; PB/tool_node.py:L1022--L1029`
Performance	Output (parallel tools), Control flow (parallel nodes, caching), Model interface (streaming)	Tool calls run concurrently (thread pool / gather); nodes in a superstep run concurrently; node-level `CachePolicy(key_func, ttl)` with `BaseCache` lookup before execution; per-step input cache; `trigger_to_nodes` avoids scanning all nodes; token streaming; `durability="async"/"exit"` defers checkpoint writes	`PB/tool_node.py:L829--L832, L863--L866; LG/pregel/_runner.py:L259--L277; LG/types.py:L529--L538; LG/pregel/_loop.py:L1701--L1714; LG/pregel/main.py:L3056--L3057; LG/pregel/_algo.py:L667--L686, L1357--L1359, L474--L481; CP/cache/base/__init__.py:L15`
Evaluation	Across	Unit/integration test suites per package (prebuilt: `test_react_agent.py` 46 tests, `test_tool_node.py` 42; runtime: `test_pregel.py` 172, `test_retry.py` 105, `test_time_travel.py` 50, `test_interruption.py` 6); a `checkpoint-conformance` package for savers. Agent-quality evals: not found in this repository.	`libs/prebuilt/tests/test_react_agent.py; libs/prebuilt/tests/test_tool_node.py; libs/langgraph/tests/test_pregel.py; libs/langgraph/tests/test_retry.py; libs/checkpoint-conformance/tests/test_validate_memory.py`

Design decisions (one line each: the choice this harness made within a primitive)

- Input: everything (user message, tool result, resume value, external `Command(update)`) enters as a write to a named channel of a typed state, merged by a user-chosen reducer (`add_messages` upserts by message id).
- Context: the model sees exactly `state["messages"]` (optionally prefixed by one system message); fitting to the window is delegated entirely to a user-written `pre_model_hook`, with `REMOVE_ALL_MESSAGES` as the only built-in rewrite primitive.
- Model interface: no client of its own; a LangChain `BaseChatModel` is invoked once per `agent` node, with retries/timeouts expressed as per-node policies that are off by default.
- Output: `ToolNode` runs every tool call of a response concurrently in-process, converts results and (configurable) errors into `ToolMessage`s, and lets a tool return a `Command` to reach the scheduler.
- Control flow: the loop is a user-written graph executed as Pregel supersteps; "go again" is a conditional edge, "stop" is an empty task set or `recursion_limit`, pauses are exceptions persisted as checkpoint

writes and resumed by re-running the node.

Hard cases

- **Conditional edge `should_continue`:** user code, but it decides go-again vs. stop and is executed by the scheduler as a writer on the node; placed in control flow by function (rule: assign by function). Note the whole graph is user-authored control flow; the prebuilt merely authors it for you.
- **`remaining_steps` managed value:** it is injected into the node's *state* (so it looks like context), but no model ever sees it; `call_model` uses it to replace the model's answer with a canned message. Placed in control flow (step budget). Its side effect — a synthetic AIMessage written into the conversation — is a context effect of a control-flow decision.
- **`Send("tools", ToolCallWithContext(state=...))`:** a scheduling packet (control flow) that also carries a state snapshot (context) to the tool task. Split: the `Send` is control flow; the snapshot and `_extract_state` are context.
- **Tool returning `Command(goto=...)` or `Command(graph=PARENT)`:** the tool call is output; its effect is routing, so `_combine_tool_outputs` and `_control_branch` are control flow (rule: a tool's EFFECT is assigned to what it changes).
- **`interrupt()` inside a tool and `HumanResponse`:** the pause/resume machinery is control flow, but the harness never consumes an approval; the framework hands the raw resume value to user code in the tool, which decides whether to execute. So the approval gate is user-written, and the value is input only if the tool shows it to the model.
- **Dynamic model callable:** a model change keyed to state/context (planning vs execution) is control flow by the rules, but the selection logic is user-written; the framework only re-resolves it before each call. No failure-keyed fallback exists, so "backup model" is absent from the model-interface primitive.
- **Checkpointer as conversation memory:** the same mechanism that makes the loop resumable (persistence of control-flow state) is what makes a `thread_id` a chat history (persistence of context). Recorded once under persistence, spanning both primitives.
- **Pregel channel/version scheduler:** it is a general dataflow scheduler (nodes fire when subscribed channels change), not a loop with explicit go-again/hand-back/stop. It fits control flow, but the "shape of the loop" is emergent from edges and versions rather than coded; the only explicit stop conditions are "no tasks" and the step budget.
- **`DEFAULT_RECURSION_LIMIT = "10007"`:** the value found at this commit (`LG/_internal/_config.py:L32`) differs from the widely documented default of 25; the clone has a single squashed commit, so the change cannot be traced. Recorded as found, not interpreted.
- **Run `context` / `Runtime`:** operator-supplied static data that is neither a person's message nor a tool result; it is closest to configuration (out of scope) but is delivered on the request path to tools, so it is noted under Input without a strong claim.
- **Error-handler nodes and `wrap_tool_call`:** both are user-written code that the framework schedules; they are resilience mechanisms whose primitive depends on what the user's code does (a retry inside `wrap_tool_call` is output-side; an error handler that routes is control flow).

Summary row

Input	Context	Model interface	Output	Control flow
Messages and tool results become reducer-merged channel writes	System prompt plus state messages; fitting left to user hook	One LangChain chat-model invoke per step; per-node retries optional	ToolNode runs tool calls concurrently, errors become ToolMessages	User-written graph run as Pregel supersteps; interrupts checkpointed